



US 20250284653A1

(19) **United States**

(12) **Patent Application Publication**
AGARWAL et al.

(10) **Pub. No.: US 2025/0284653 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **INTERCONNECT ARCHITECTURE
ENABLING PATH DIVERSITY FOR
STRONGLY ORDERED MESSAGES**

Publication Classification

(51) **Int. Cl.**
G06F 13/40 (2006.01)
G06F 15/173 (2006.01)
(52) **U.S. Cl.**
CPC G06F 13/4059 (2013.01); **G06F 15/17381**
(2013.01)

(71) Applicant: **Intel Corporation**, Santa Clara, CA
(US)

(72) Inventors: **Monam AGARWAL**, Lynnwood, WA
(US); **Swadesh CHOUDHARY**,
Mountain View, CA (US); **Stephen R.
VAN DOREN**, Portland, OR (US);
Yen-Cheng Liu, Portland, OR (US);
Vinit MATHEW ABRAHAM,
Hillsboro, OR (US); **Robert G.
BLANKENSHIP**, Tacoma, WA (US);
Amandeep SHARAN, Karnataka (IN);
Anindya Sundar DHAR, Karnataka
(IN); **Akhilesh TIWARY**, West Bengal
(IN)

(21) Appl. No.: **18/757,457**

(22) Filed: **Jun. 27, 2024**

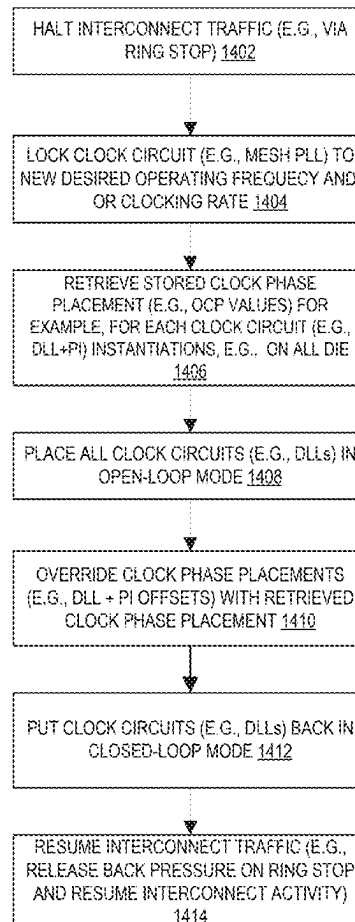
(30) **Foreign Application Priority Data**

Mar. 8, 2024 (IN) 202441016809

(57) **ABSTRACT**

Methods and apparatuses related to efficient fabric usage. One embodiment of a method comprises: decoding, by a first bridge device associated with a plurality of source fabric agents, a first plurality of packets received from the plurality of source fabric agents of an interconnect fabric comprising a plurality of vertical interconnects coupled to a plurality of horizontal interconnects, wherein decoding is to identify one or more destination fabric agents associated with a second bridge device; routing, by first routing circuitry, the first plurality of packets across the interconnect fabric to the second bridge device, the first routing circuitry to distribute the first plurality of packets across at least one of: multiple vertical interconnects of the plurality of vertical interconnects and multiple horizontal interconnects of the plurality of horizontal interconnects.

1400 ↗



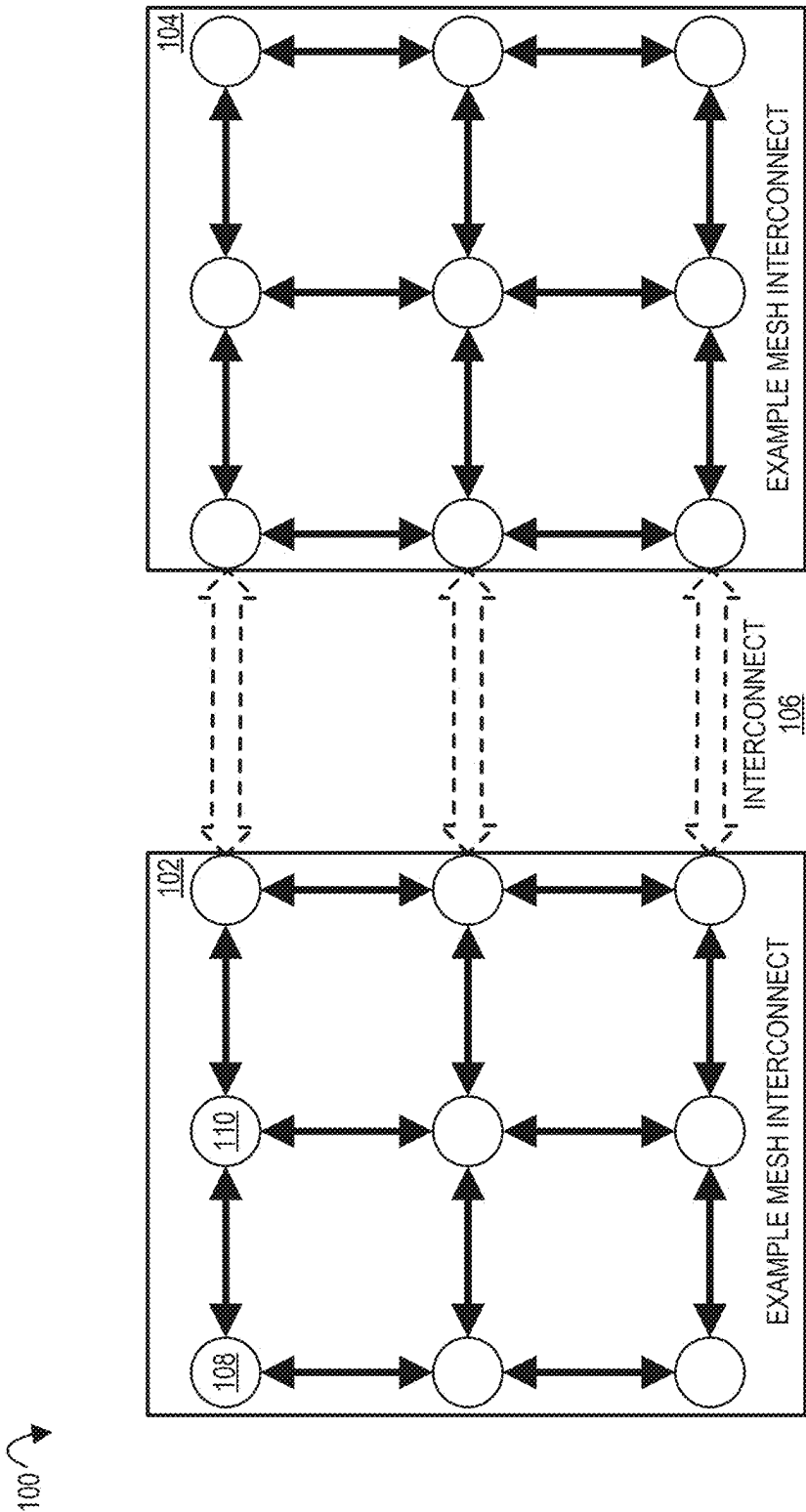


FIG. 1

200A ↗

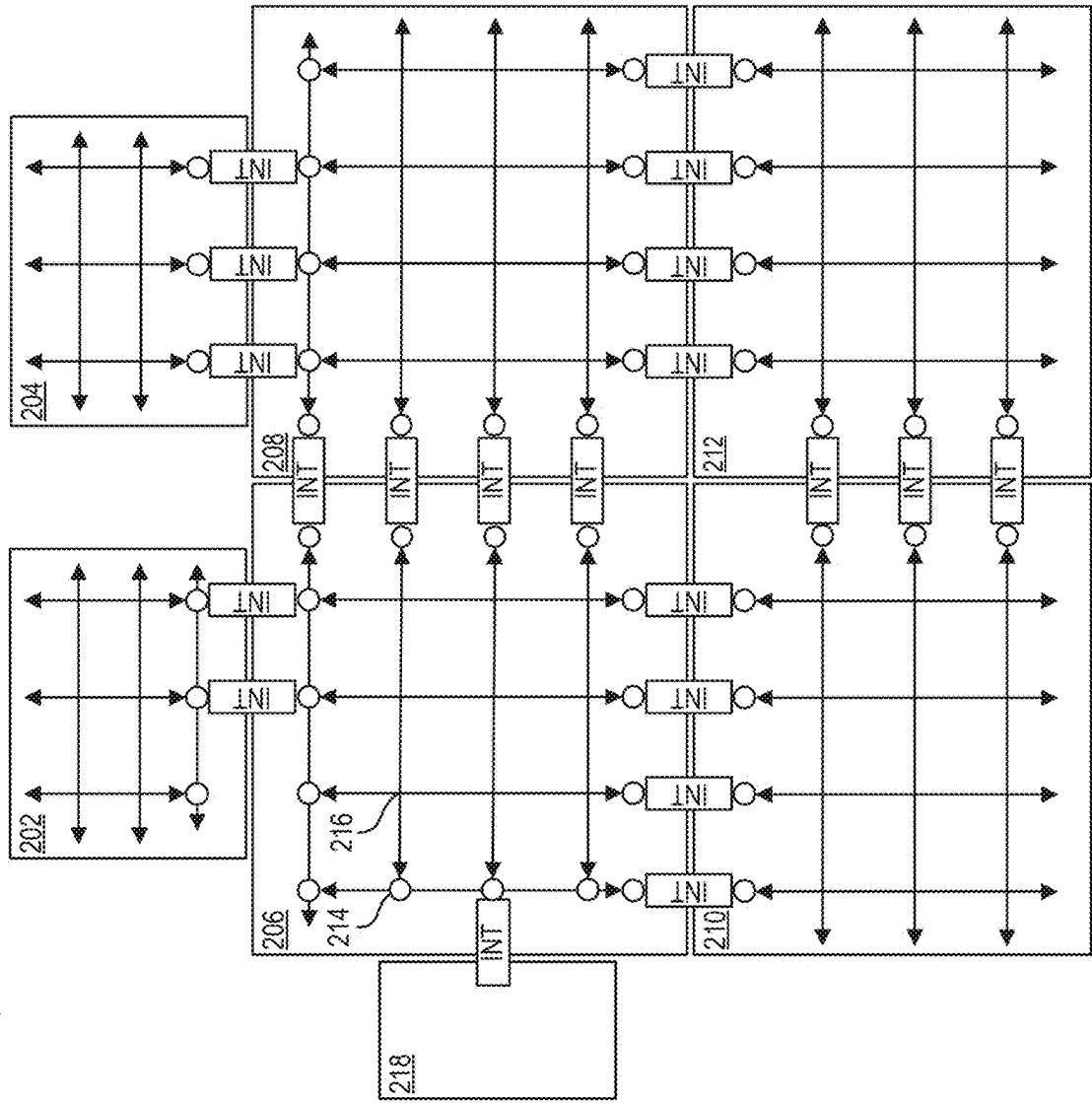


FIG. 2A

200B

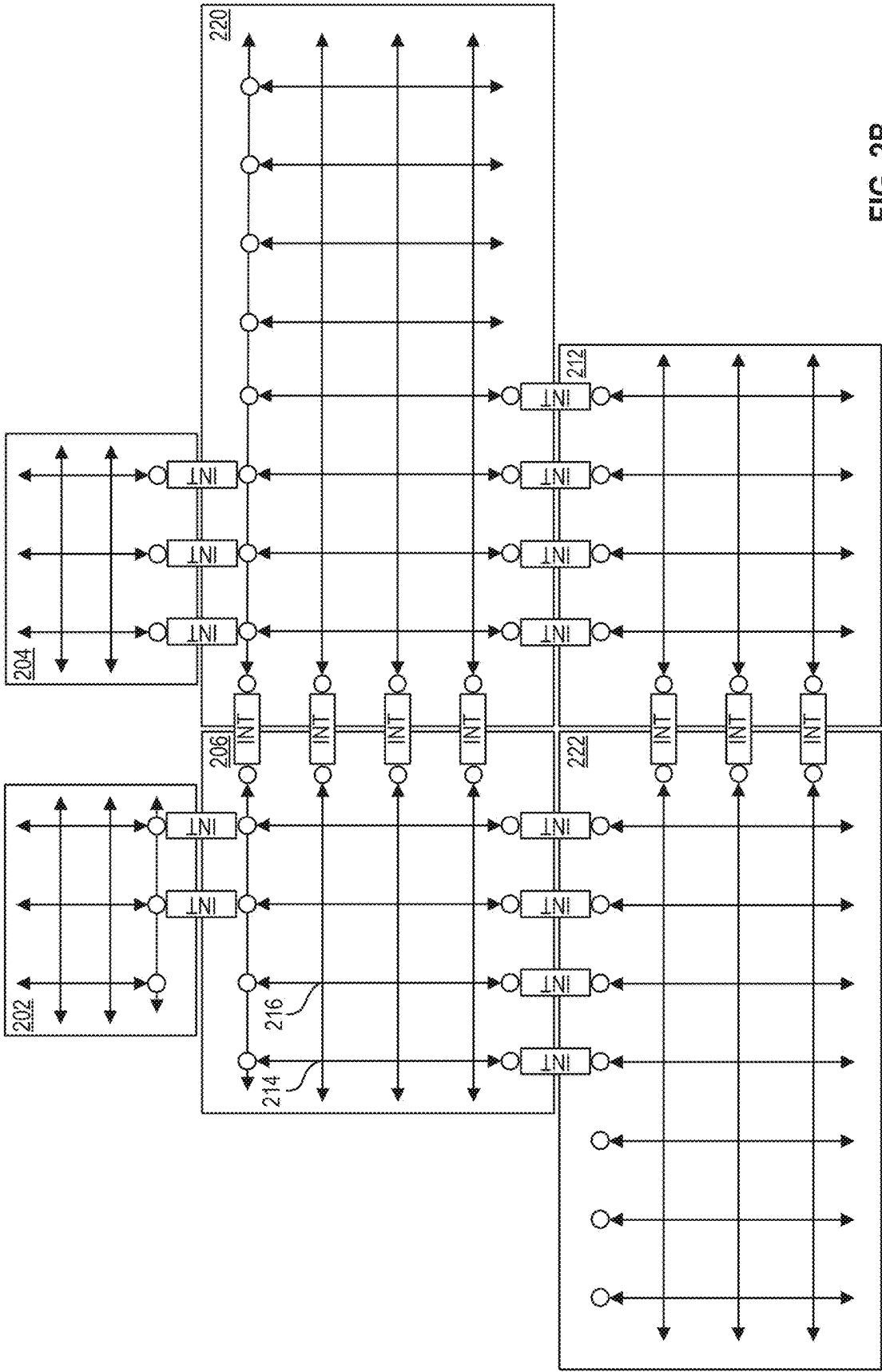
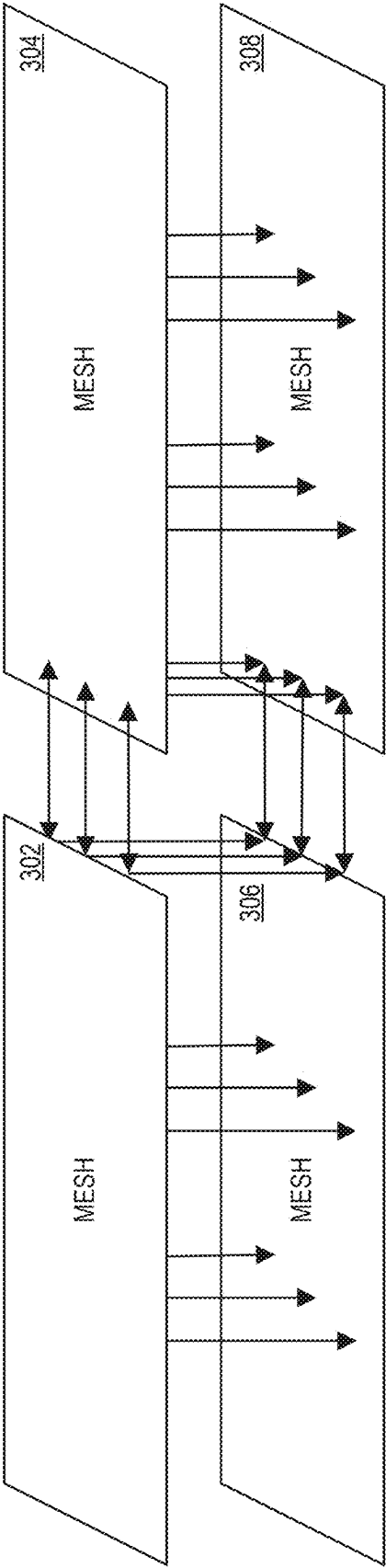


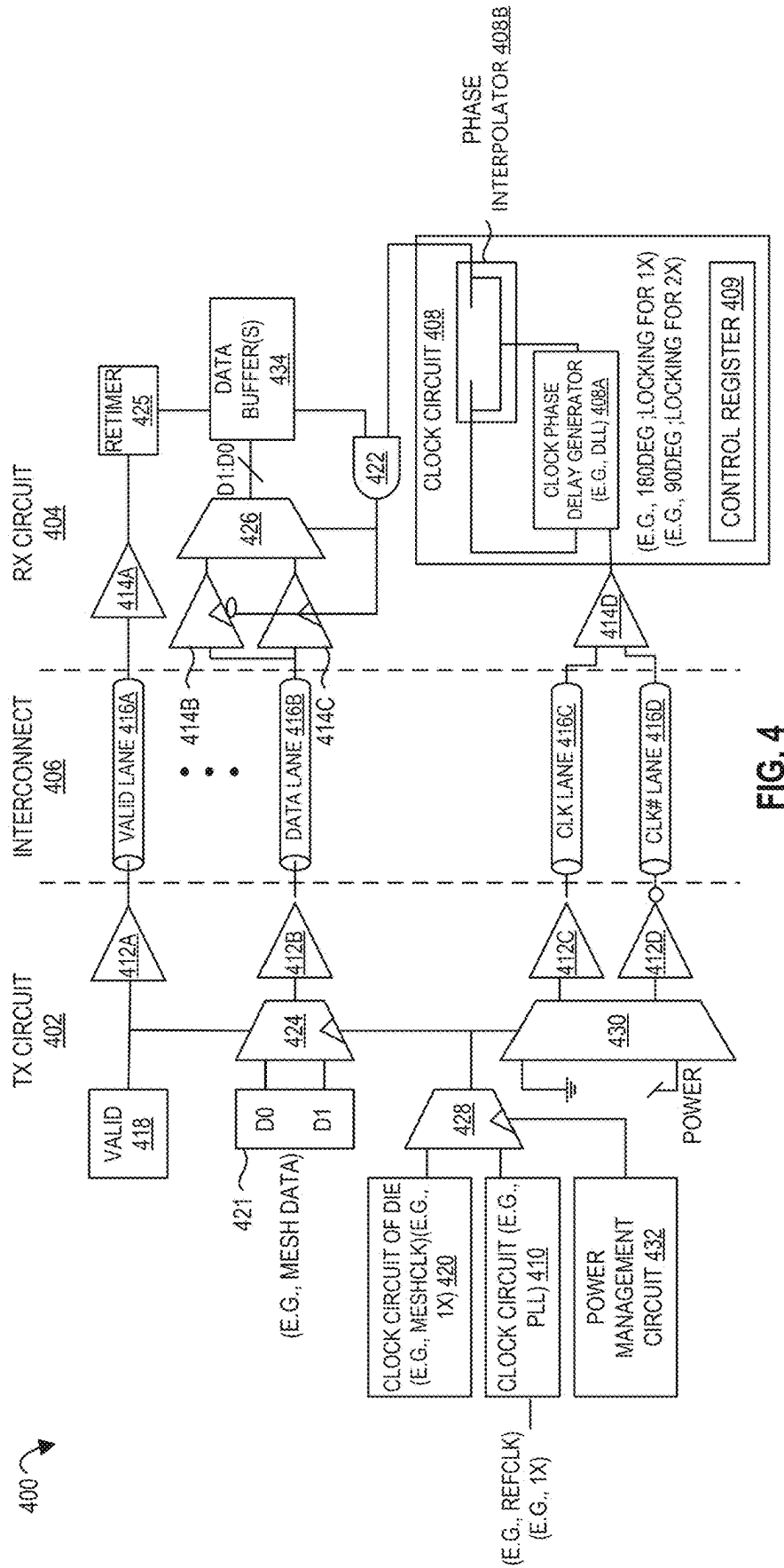
FIG. 2B

300 ↗



EXAMPLE 3D STACKED ARCHITECTURE

FIG. 3



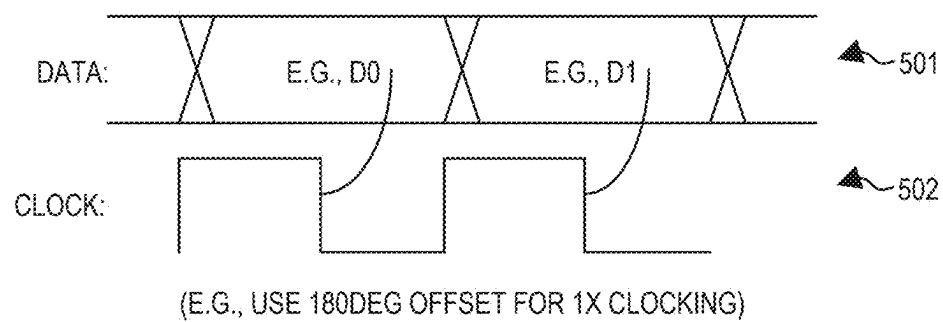


FIG. 5

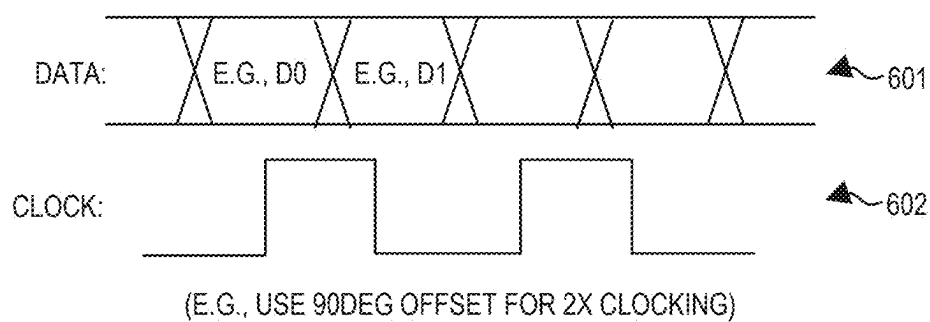


FIG. 6

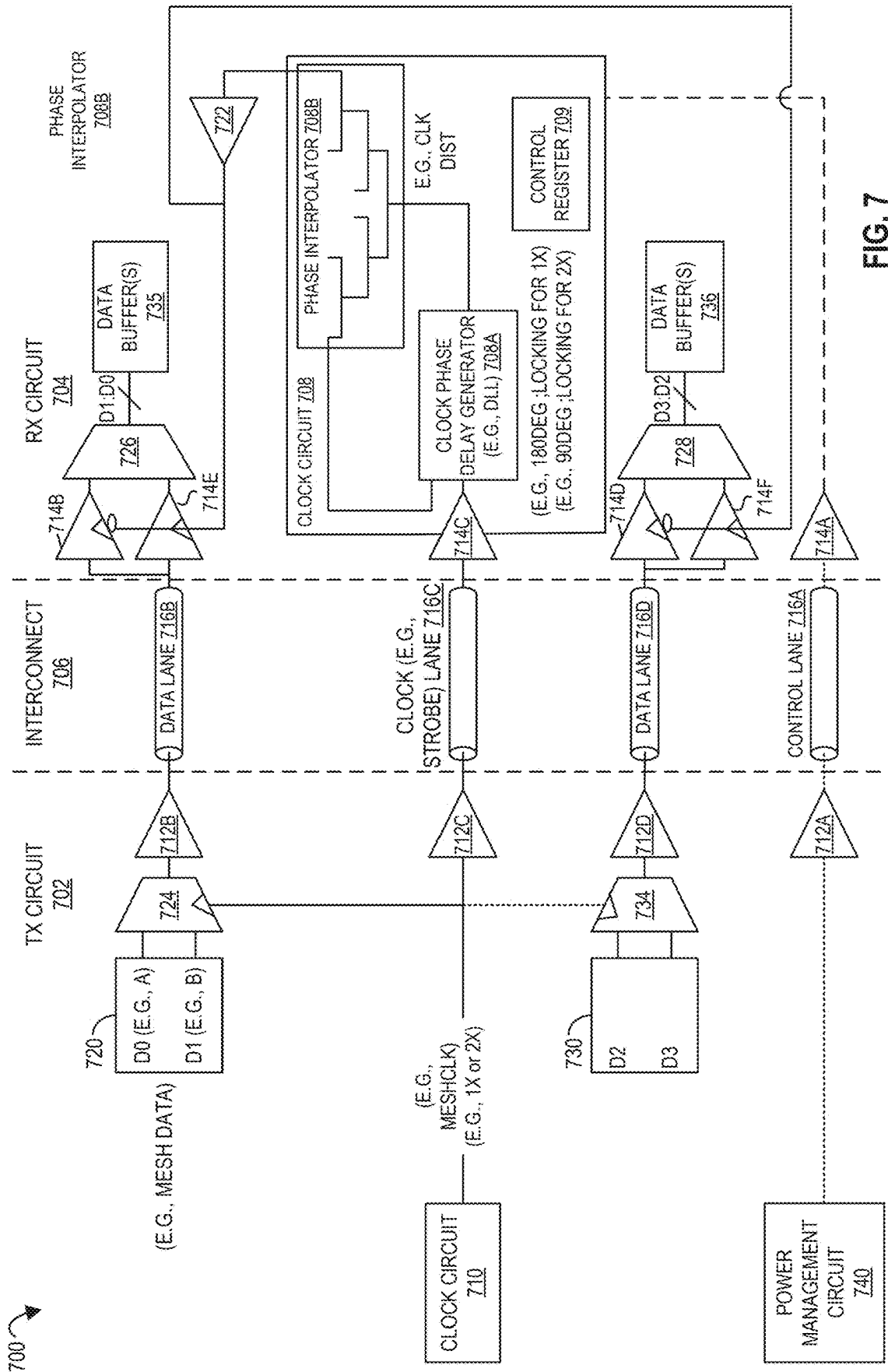


FIG. 7

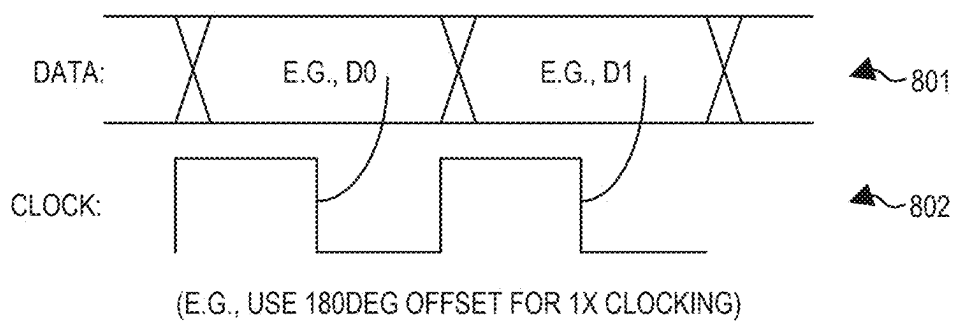


FIG. 8

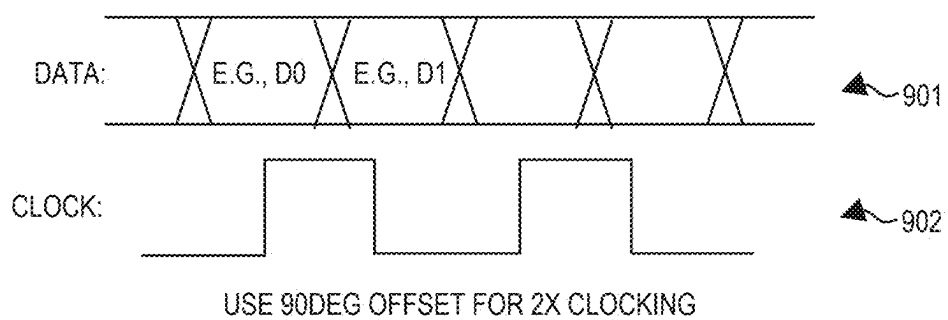


FIG. 9

1000 ↘

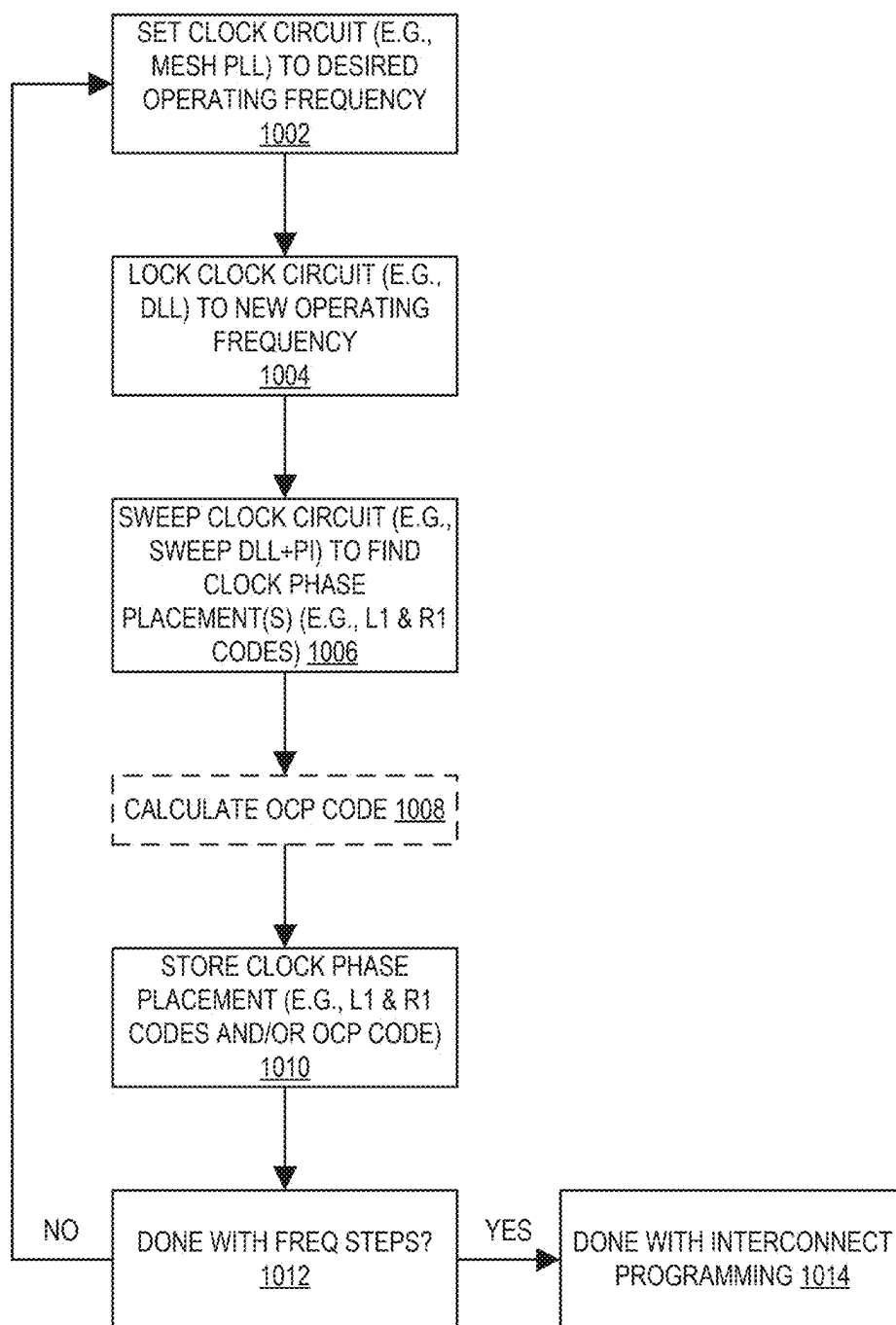


FIG. 10

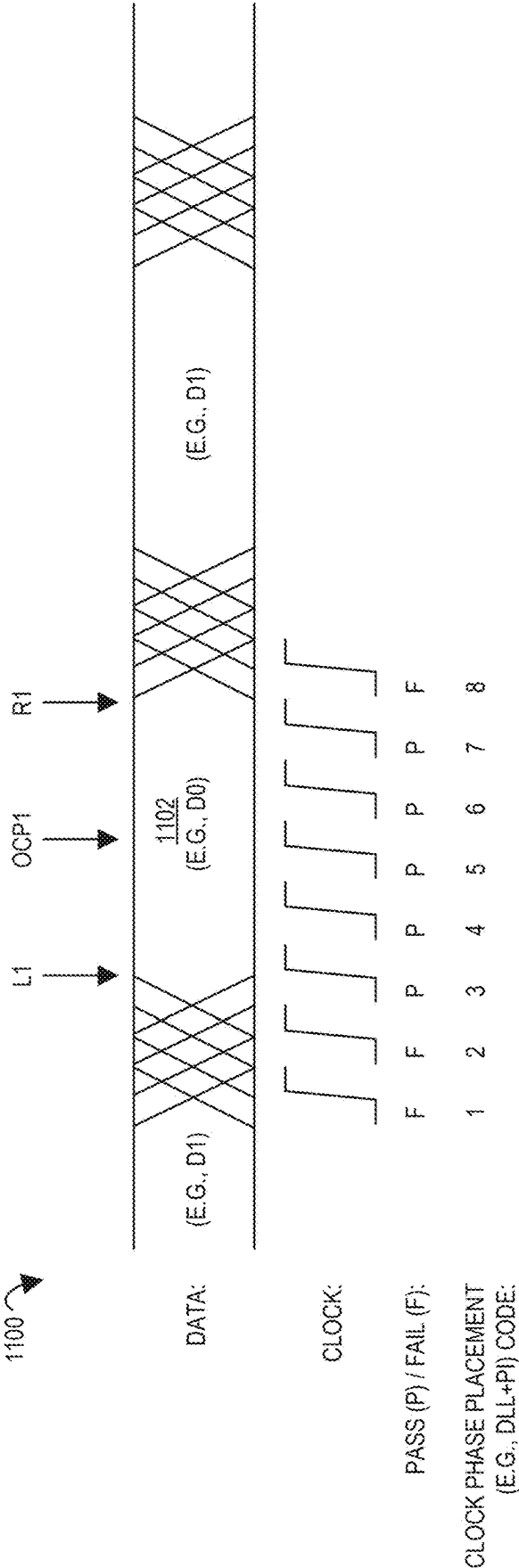


FIG. 11

1200

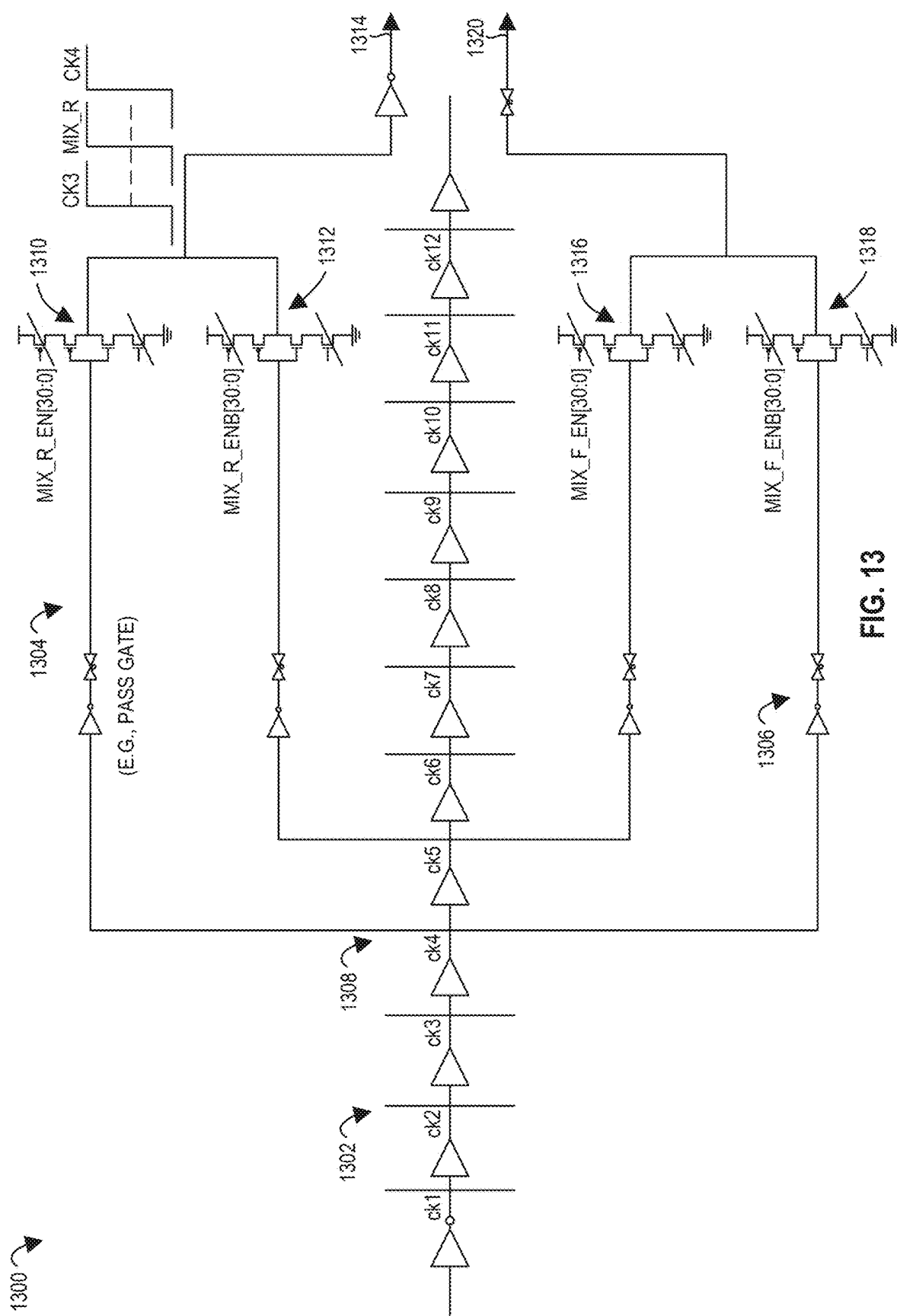
1201

1202

1203

TX TO RX	FREQUENCY	CLOCKING RATE 1 (E.G., 1X)	CLOCKING RATE 2 (E.G., 2X)	CLOCKING RATE M
D_1 TO D_2	f_1	L = VALUE 1 (E.G., 3) OCP = VALUE 2 (E.G., 5) R = VALUE 3 (E.G., 7)	L = VALUE 4 OCP = VALUE 5 R = VALUE 6	L = VALUE 7 OCP = VALUE 8 R = VALUE 9
D_1 TO D_2	f_2	L = VALUE 10 OCP = VALUE 11 R = VALUE 12	L = VALUE 13 OCP = VALUE 14 R = VALUE 15	L = VALUE 16 OCP = VALUE 17 R = VALUE 18
D_2 TO D_1	f_1	L = VALUE 19 OCP = VALUE 20 R = VALUE 21	L = VALUE 22 OCP = VALUE 23 R = VALUE 24	L = VALUE 25 OCP = VALUE 26 R = VALUE 27
D_2 TO D_1	f_2	L = VALUE 28 OCP = VALUE 29 R = VALUE 30	L = VALUE 31 OCP = VALUE 32 R = VALUE 33	L = VALUE 34 OCP = VALUE 35 R = VALUE 36
$D_{(a)}$ TO $D_{(b)}$	f_y	L = VALUE 37 OCP = VALUE 38 R = VALUE 39	L = VALUE 40 OCP = VALUE 41 R = VALUE 42	L = VALUE 43 OCP = VALUE 44 R = VALUE 45
$D_{(b)}$ TO $D_{(a)}$	f_y	L = VALUE 46 OCP = VALUE 47 R = VALUE 48	L = VALUE 49 OCP = VALUE 50 R = VALUE 51	L = VALUE 52 OCP = VALUE 53 R = VALUE 54

FIG. 12



1400 ↘

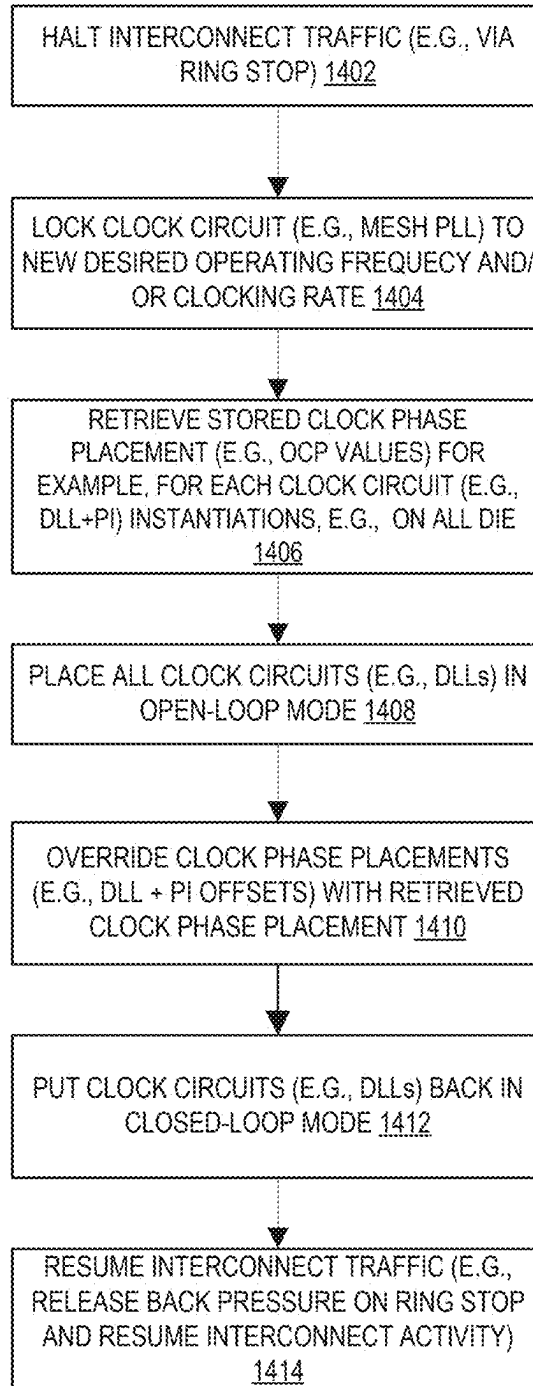
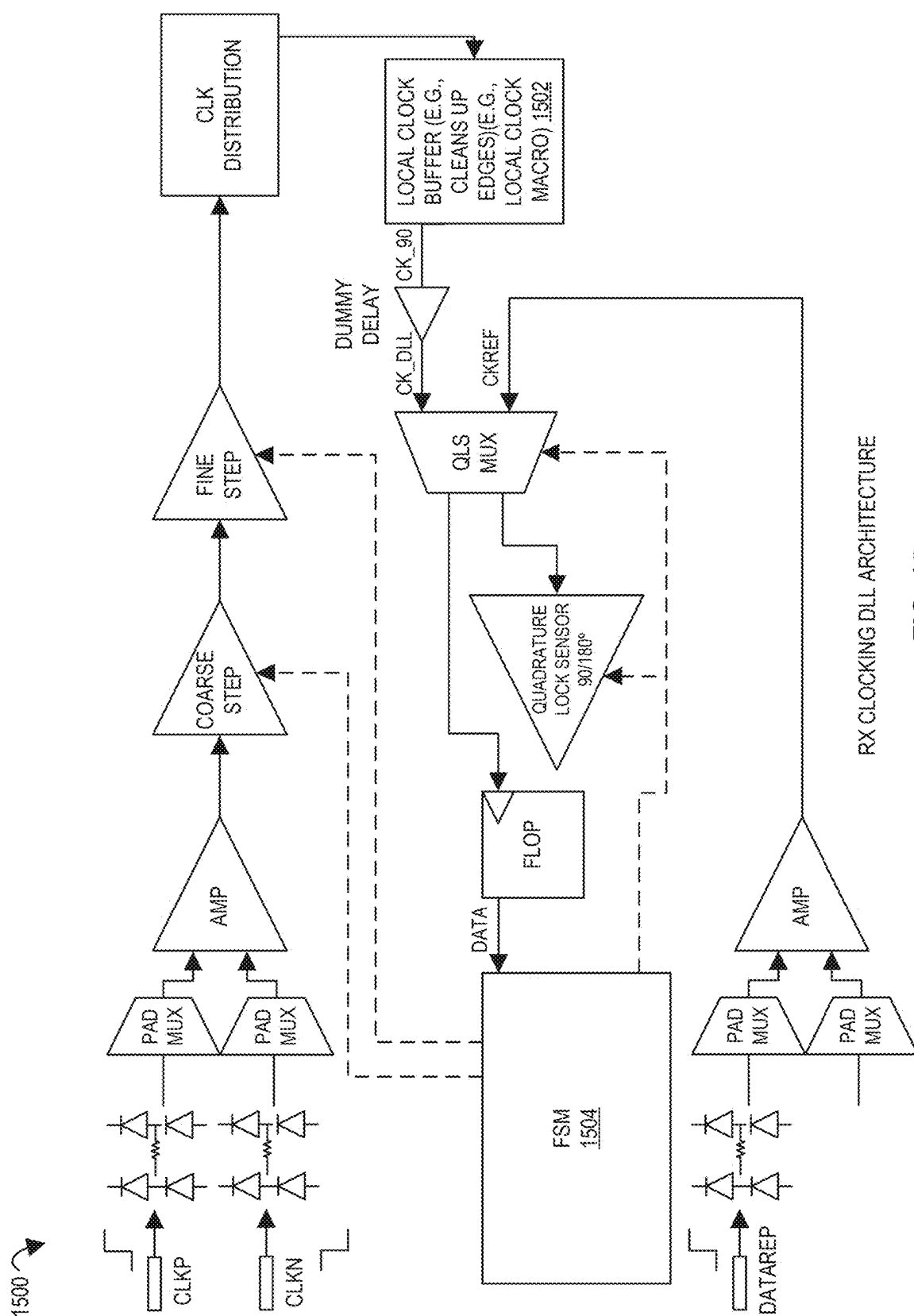
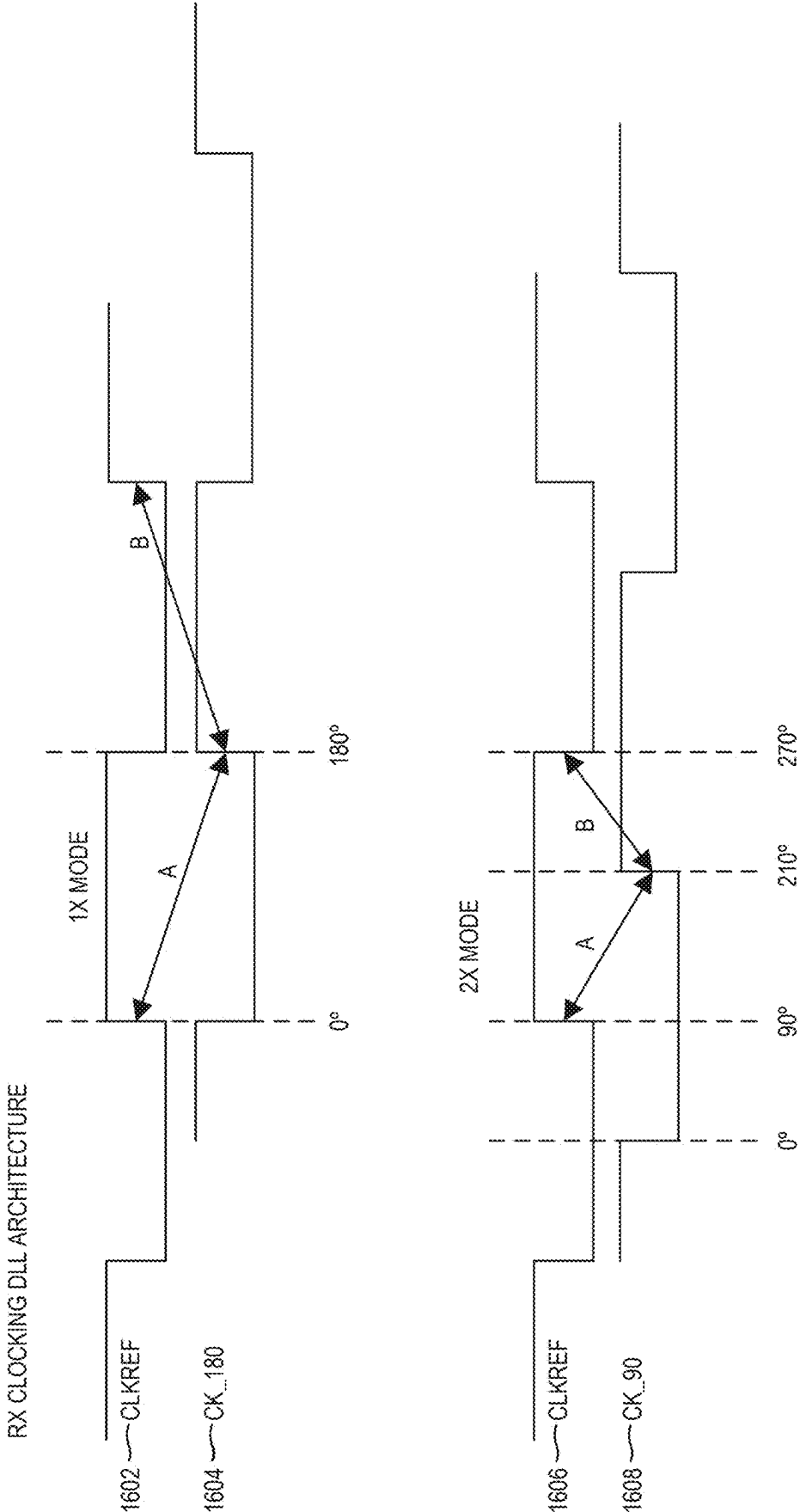


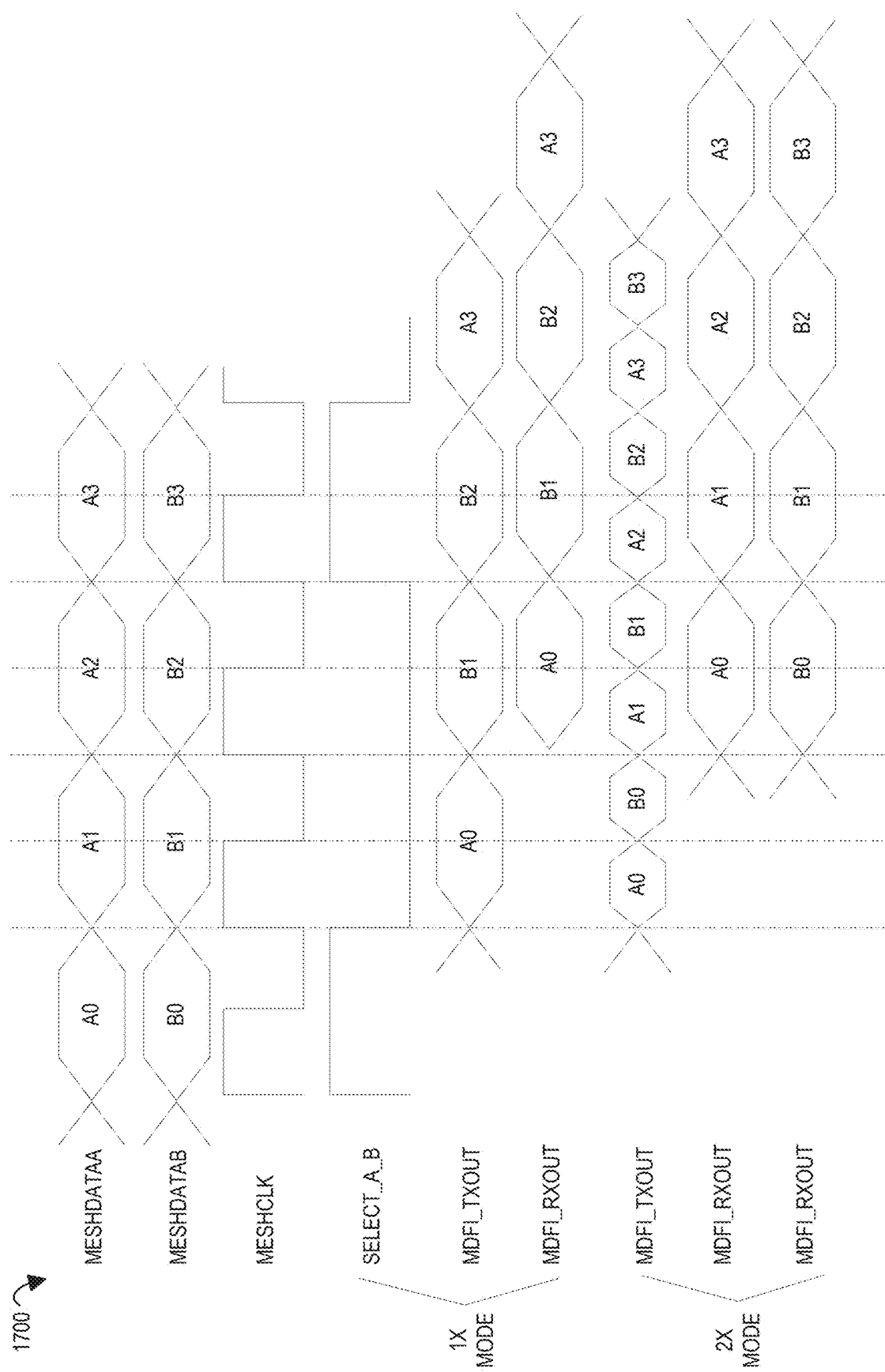
FIG. 14



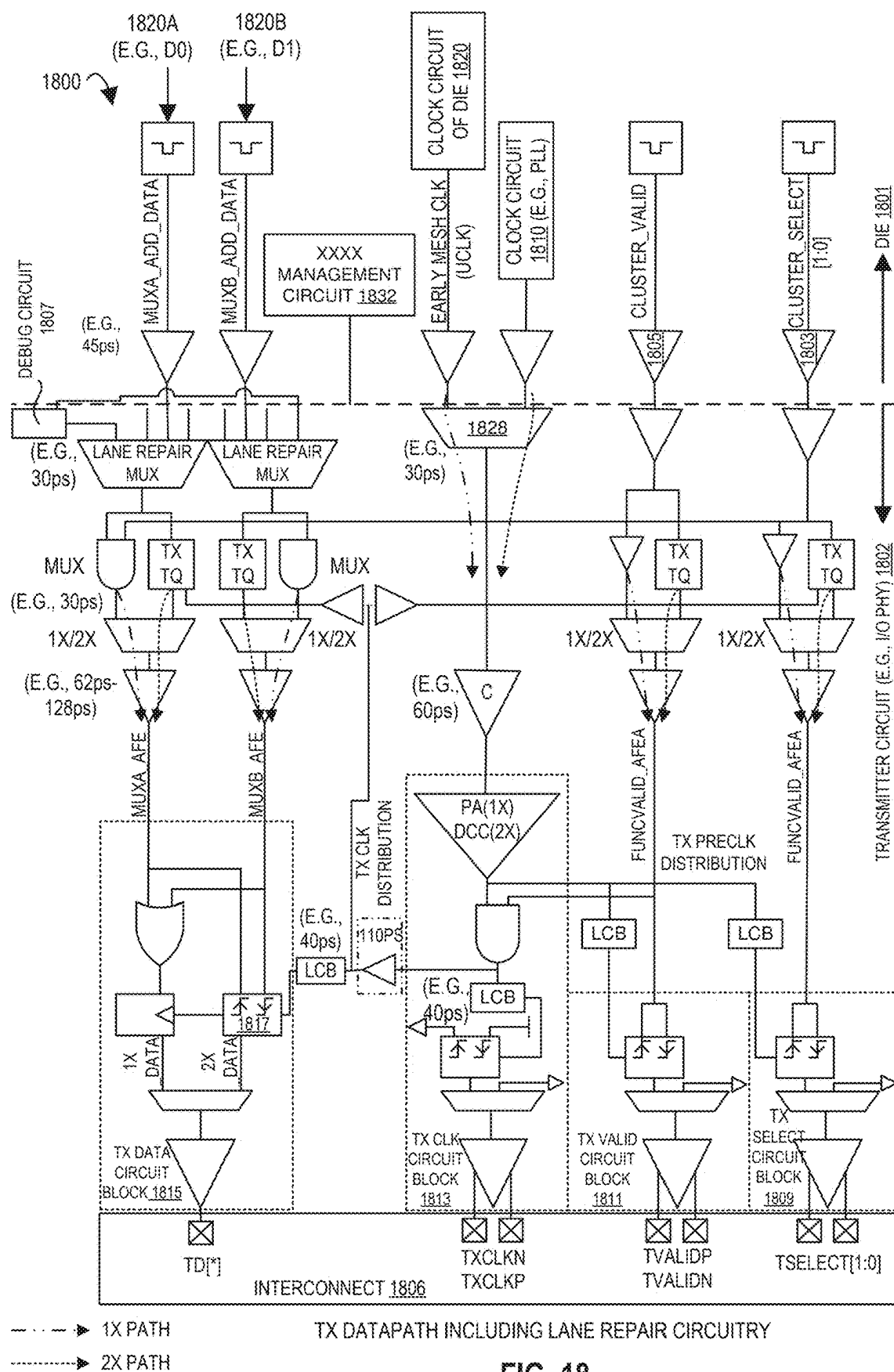


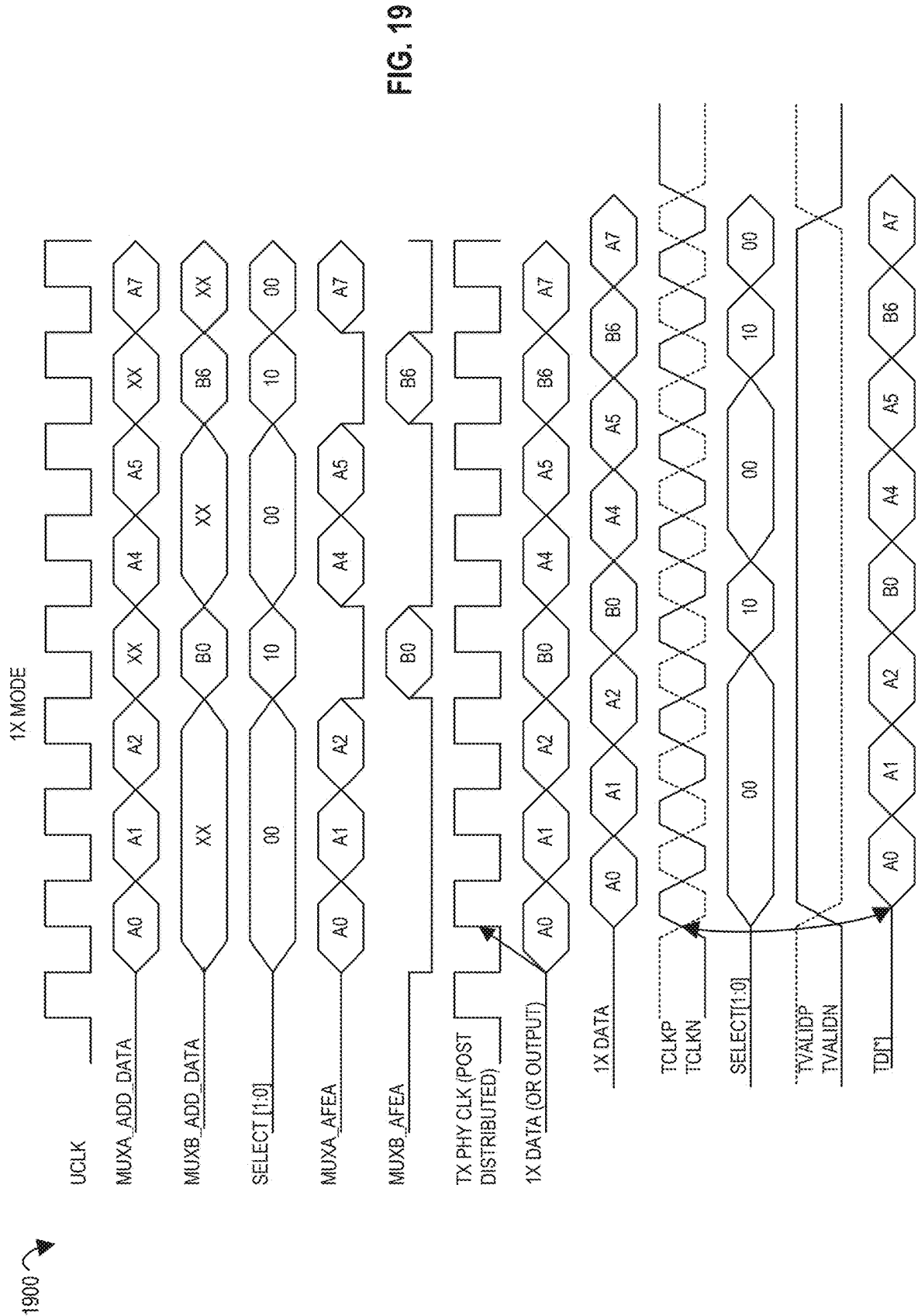
DLL LOCK TIMING DIAGRAMS FOR 1X AND 2X CLOCKING RATE (E.G., DATA RATE) MODES

FIG. 16



76





2000 

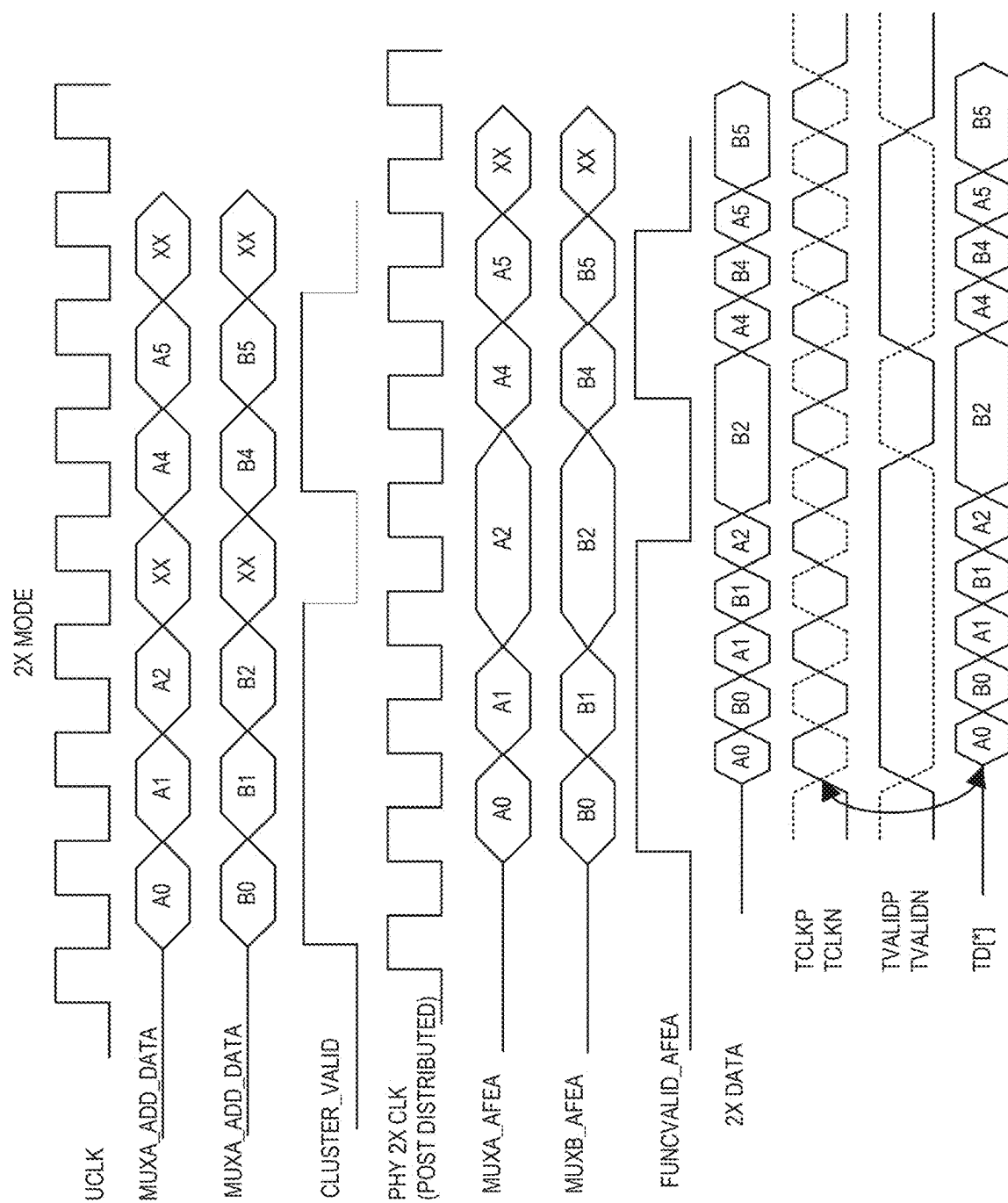
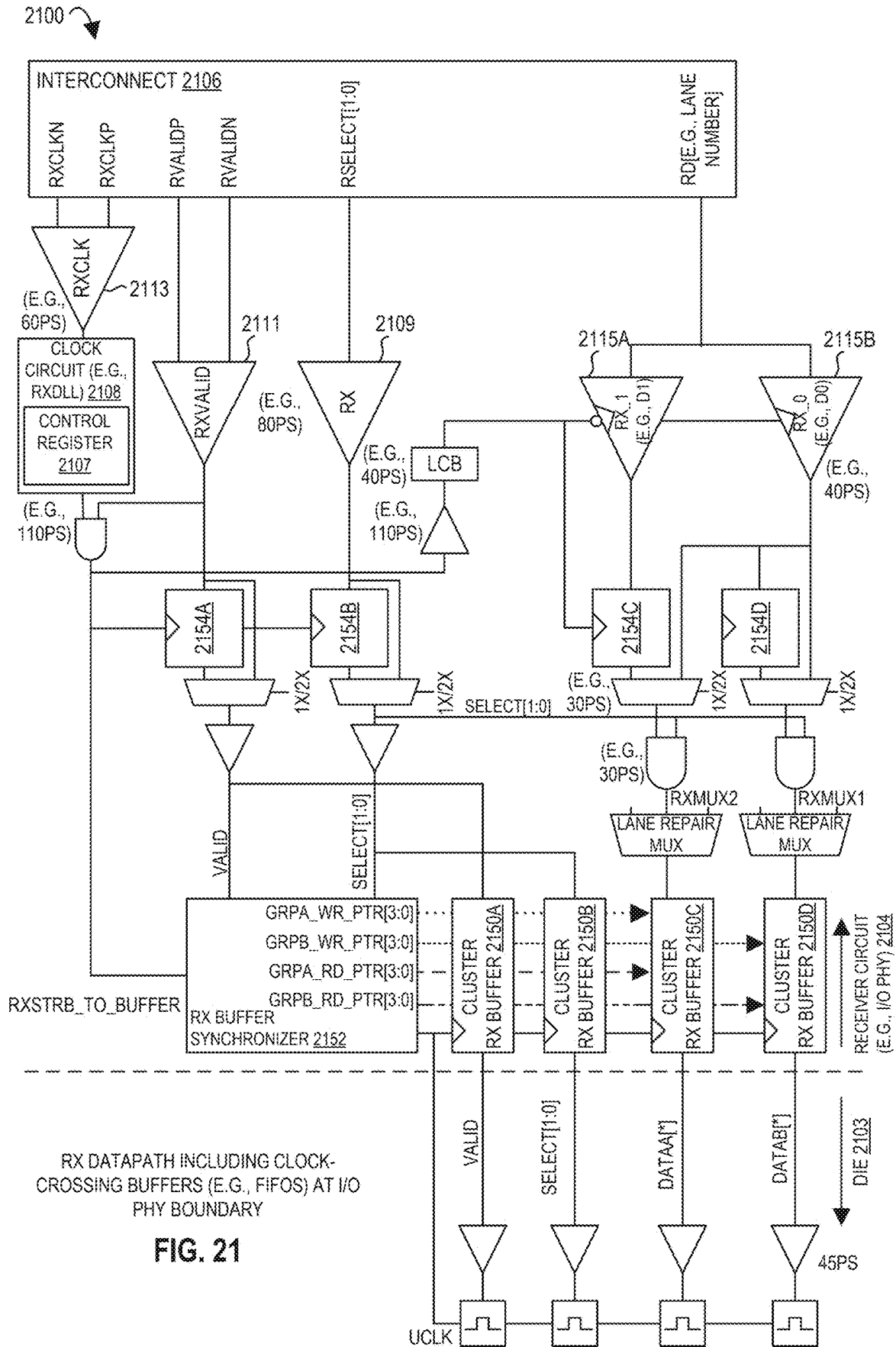
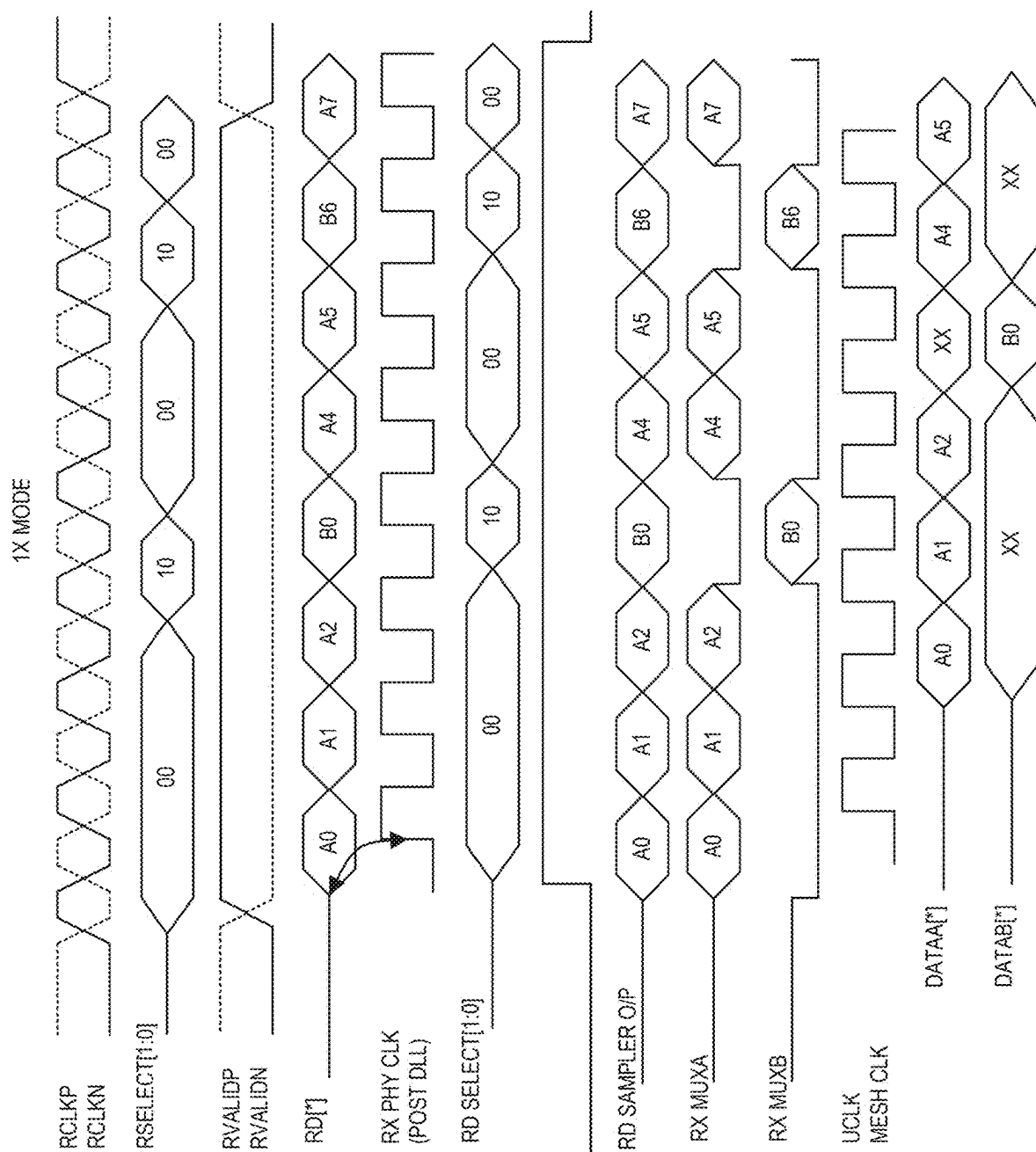


FIG. 20





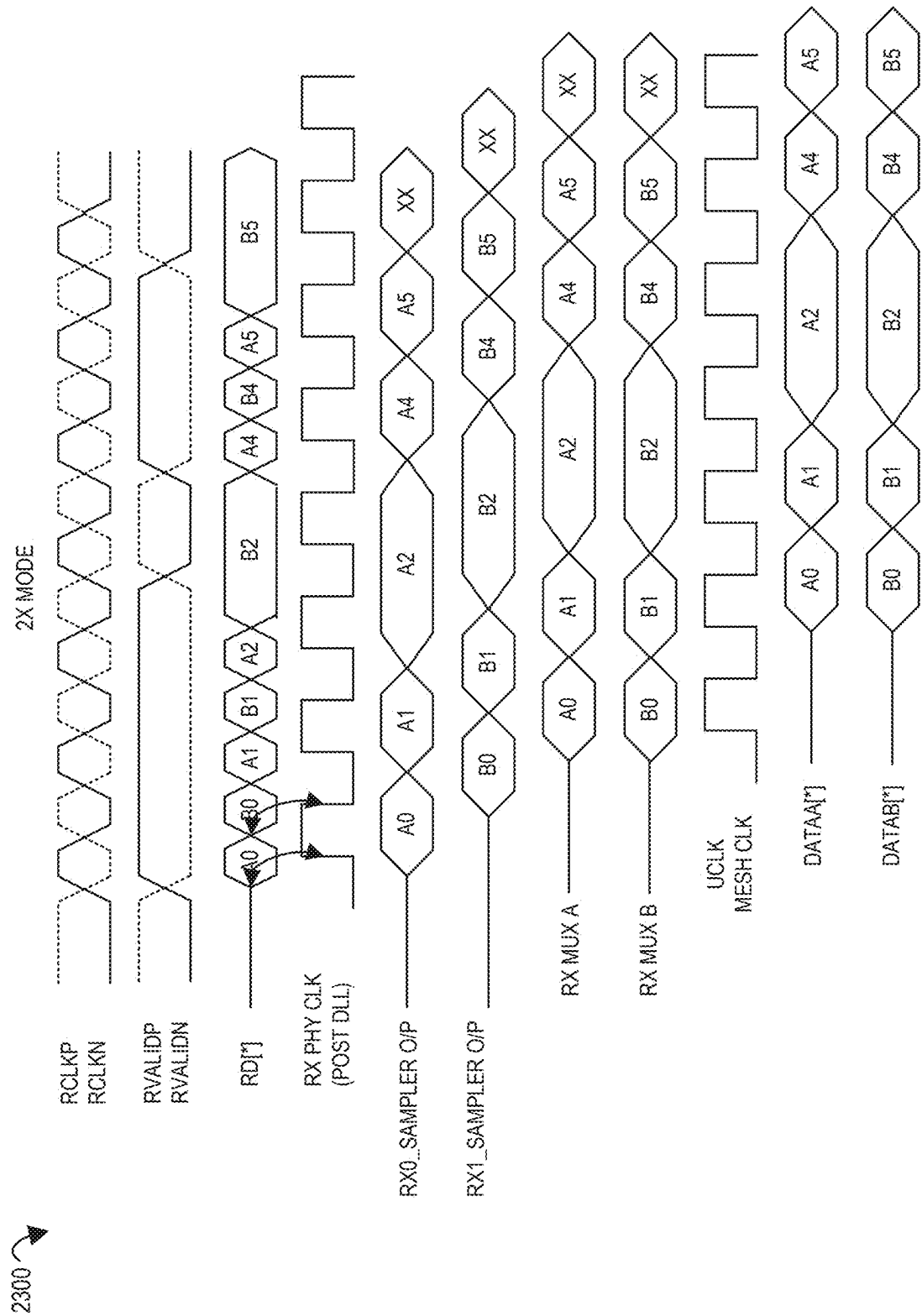
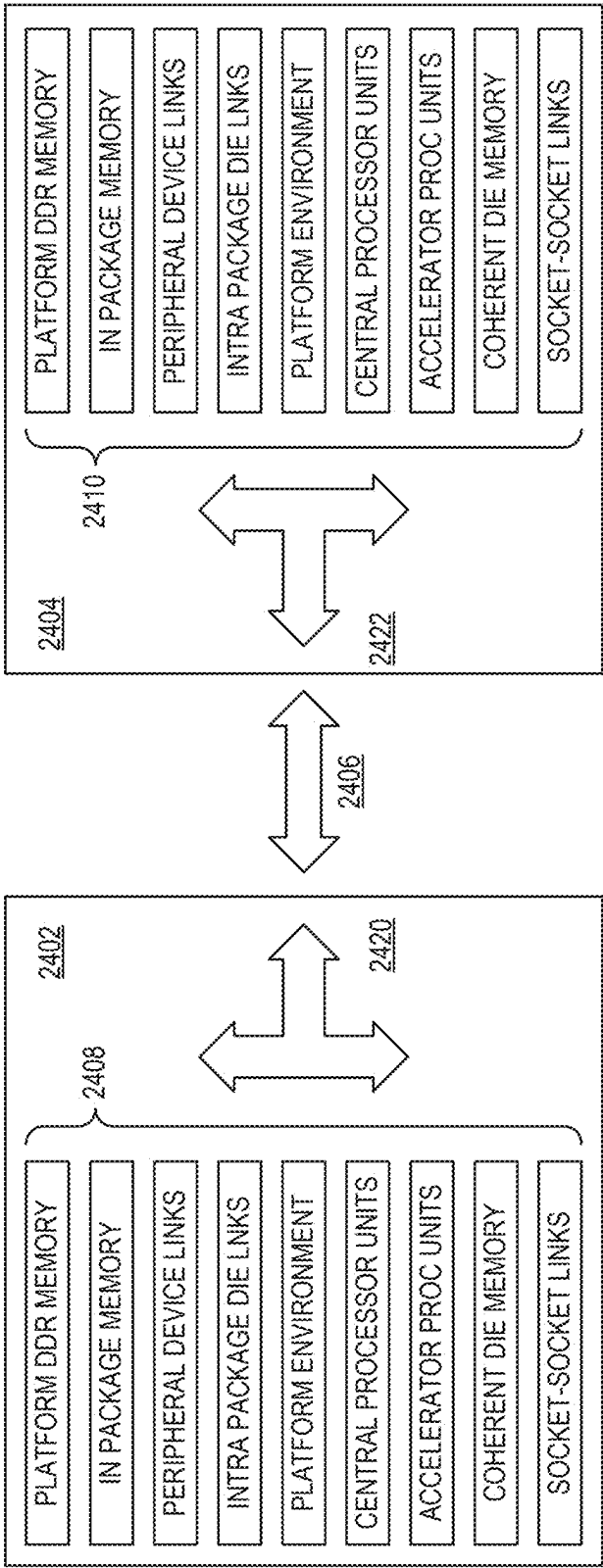


FIG. 23

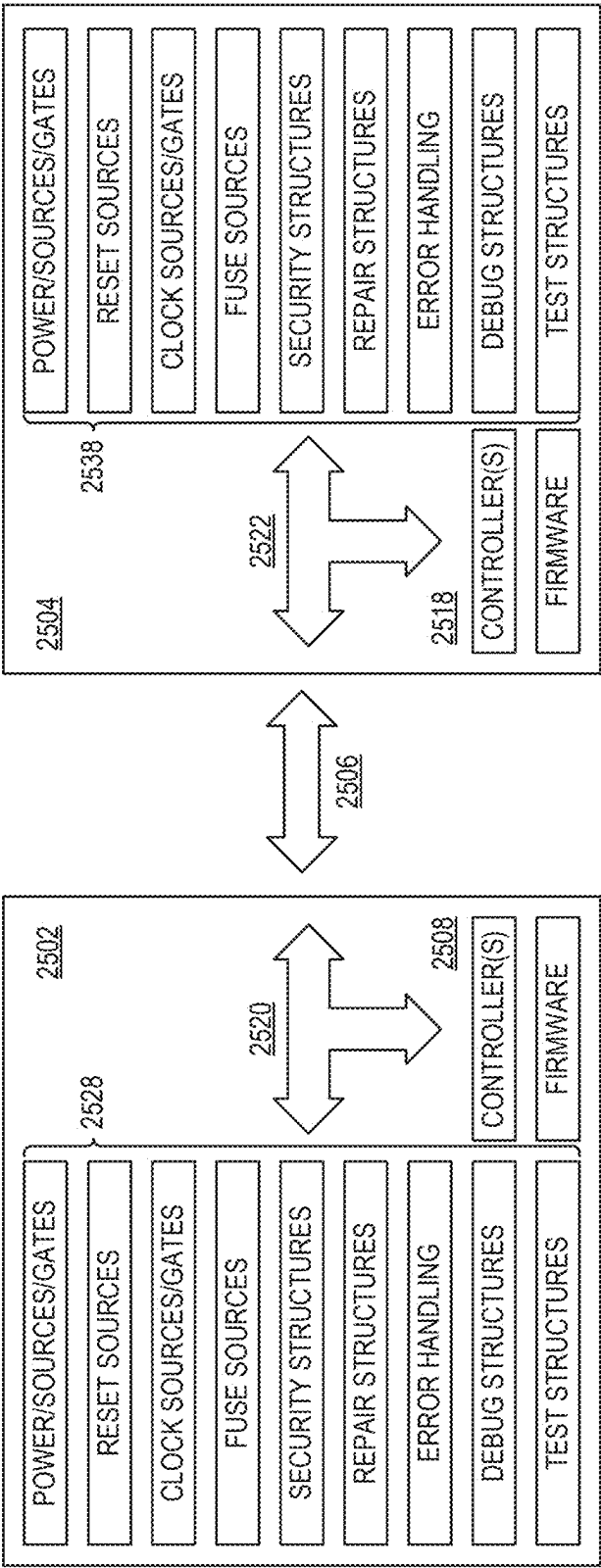
2400 ↗



EXAMPLE DIE RESOURCE SHARING

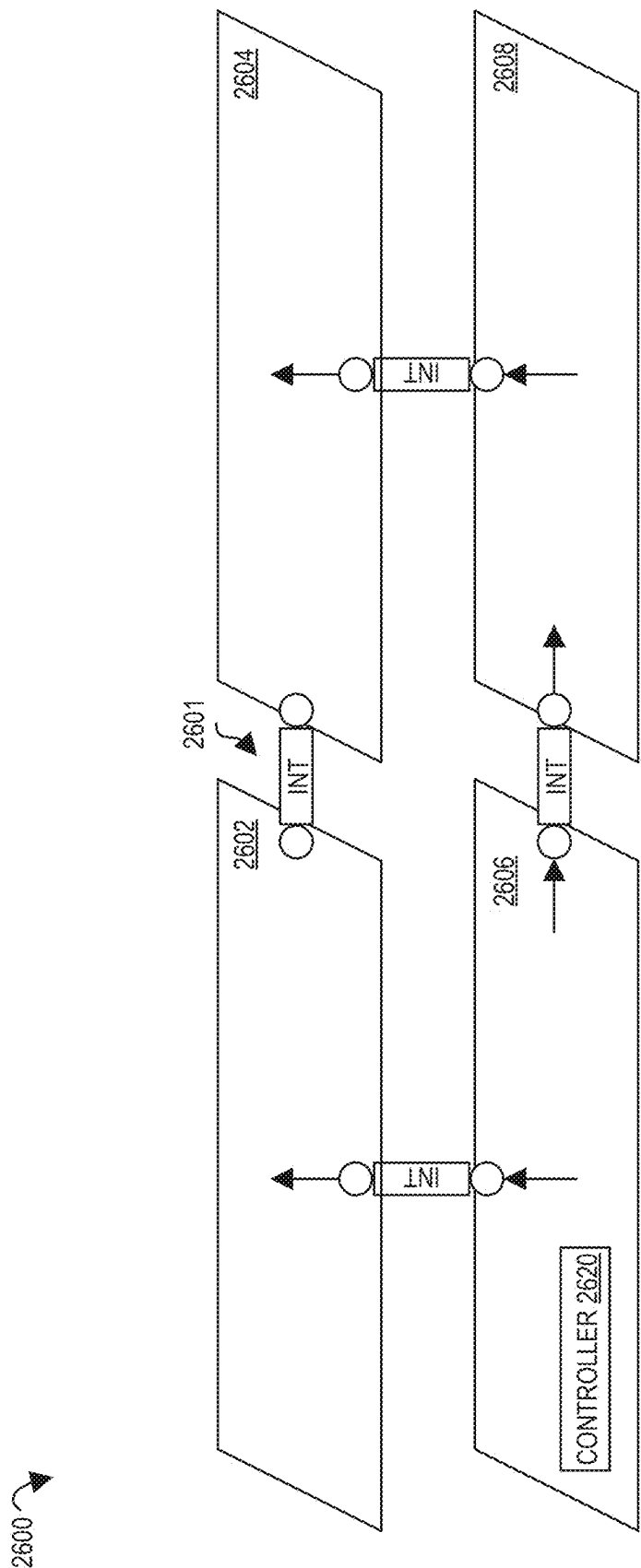
FIG. 24

2500 ↗



EXAMPLE INFRASTRUCTURE MANAGEMENT

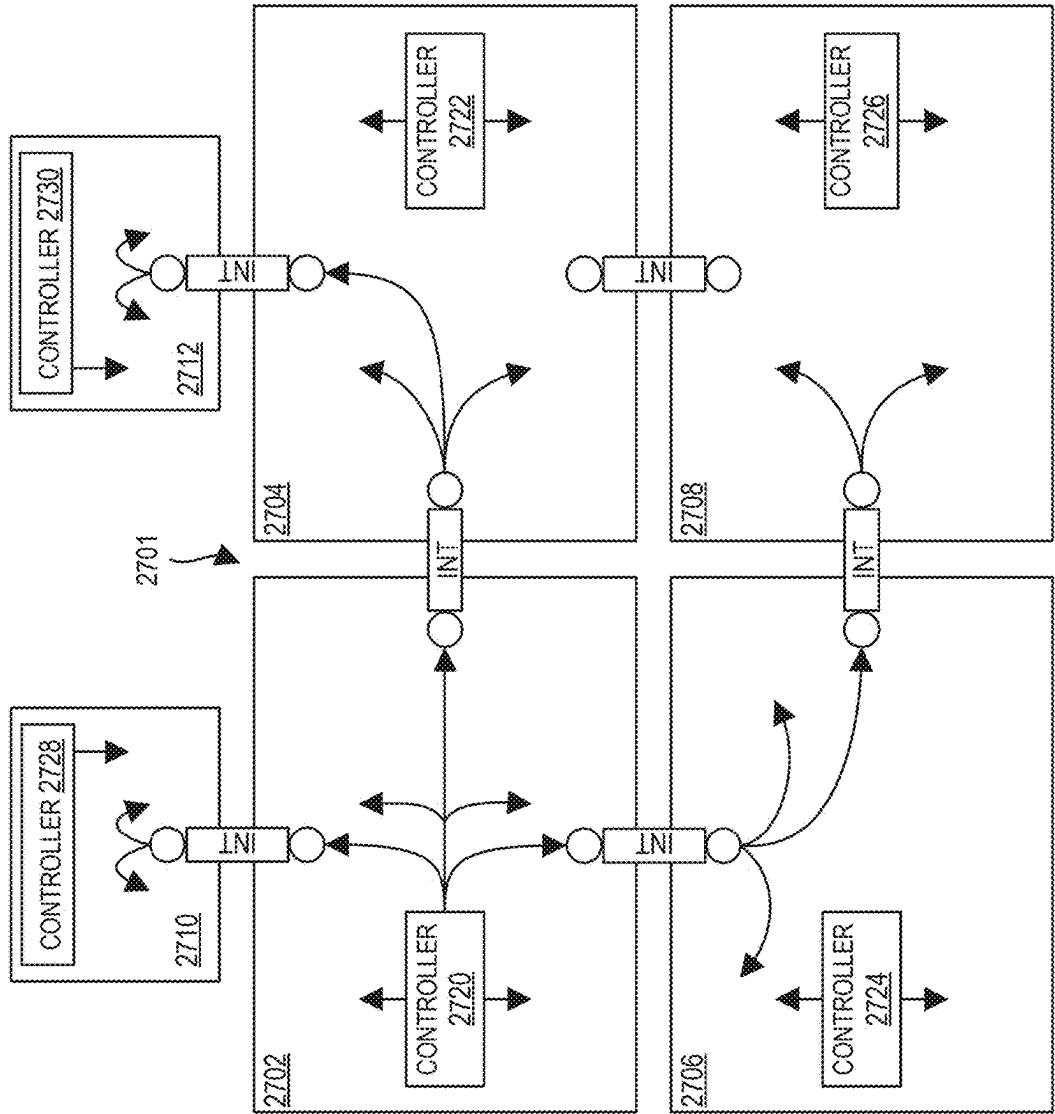
FIG. 25



EXAMPLE 3D STACKED INFRASTRUCTURE MANAGEMENT

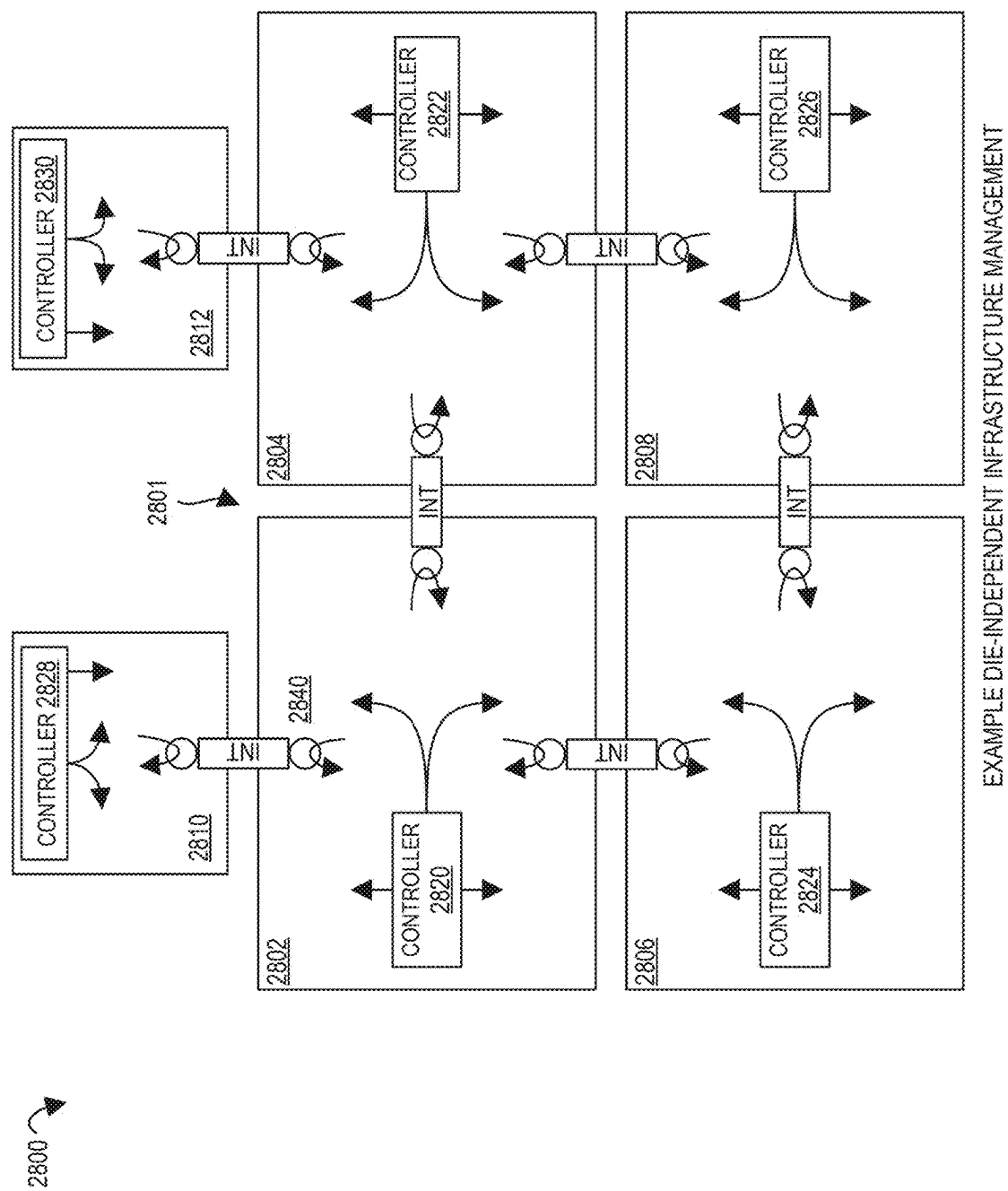
FIG. 26

2700 ↗

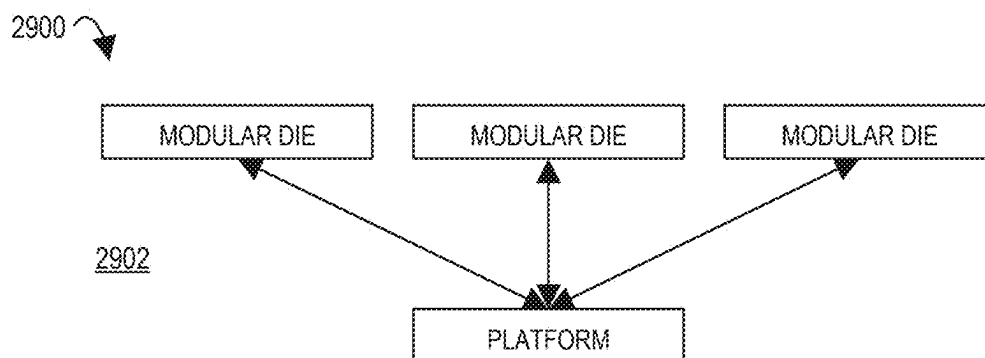


EXAMPLE MASTER-SLAVE INFRASTRUCTURE MANAGEMENT

FIG. 27



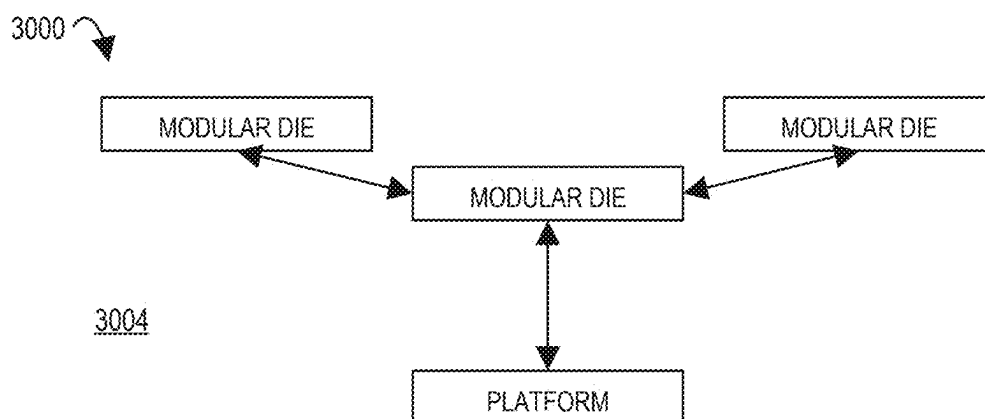
28



FOR EXAMPLE, BROADCASTS OR UNICAST POSTED WRITES FROM PLATFORM, AND SIMPLE HANDSHAKES. (E.G., ONLY EARLY BOOT USES.)

PLATFORM – MODULAR DIE EXCHANGES.

FIG. 29



FOR EXAMPLE, MORE COMPLEX EXCHANGES. (E.G., FOR LATER PHASES OF BOOT)

PLATFORM – MODULAR DIE EXCHANGES.

FIG. 30

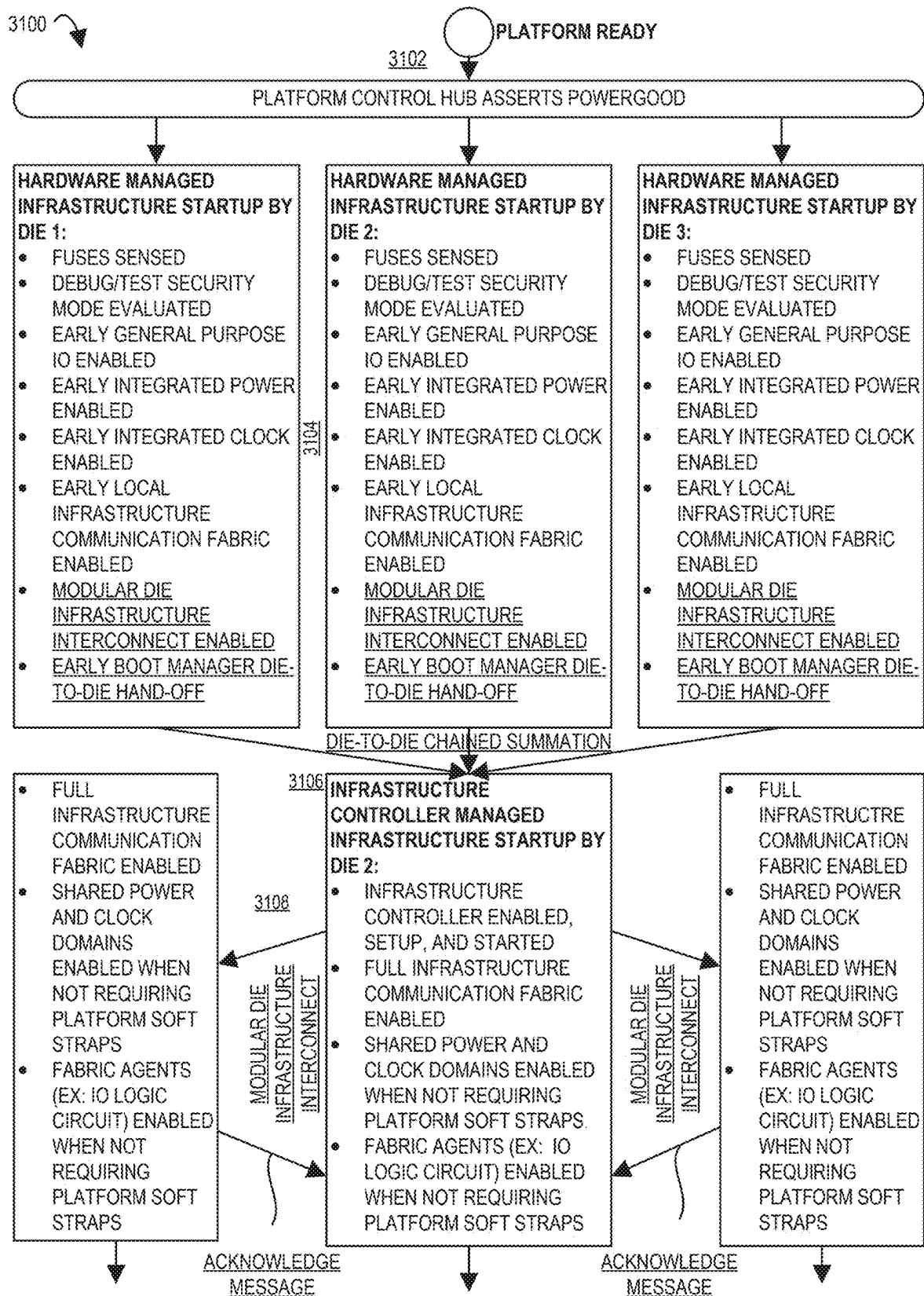
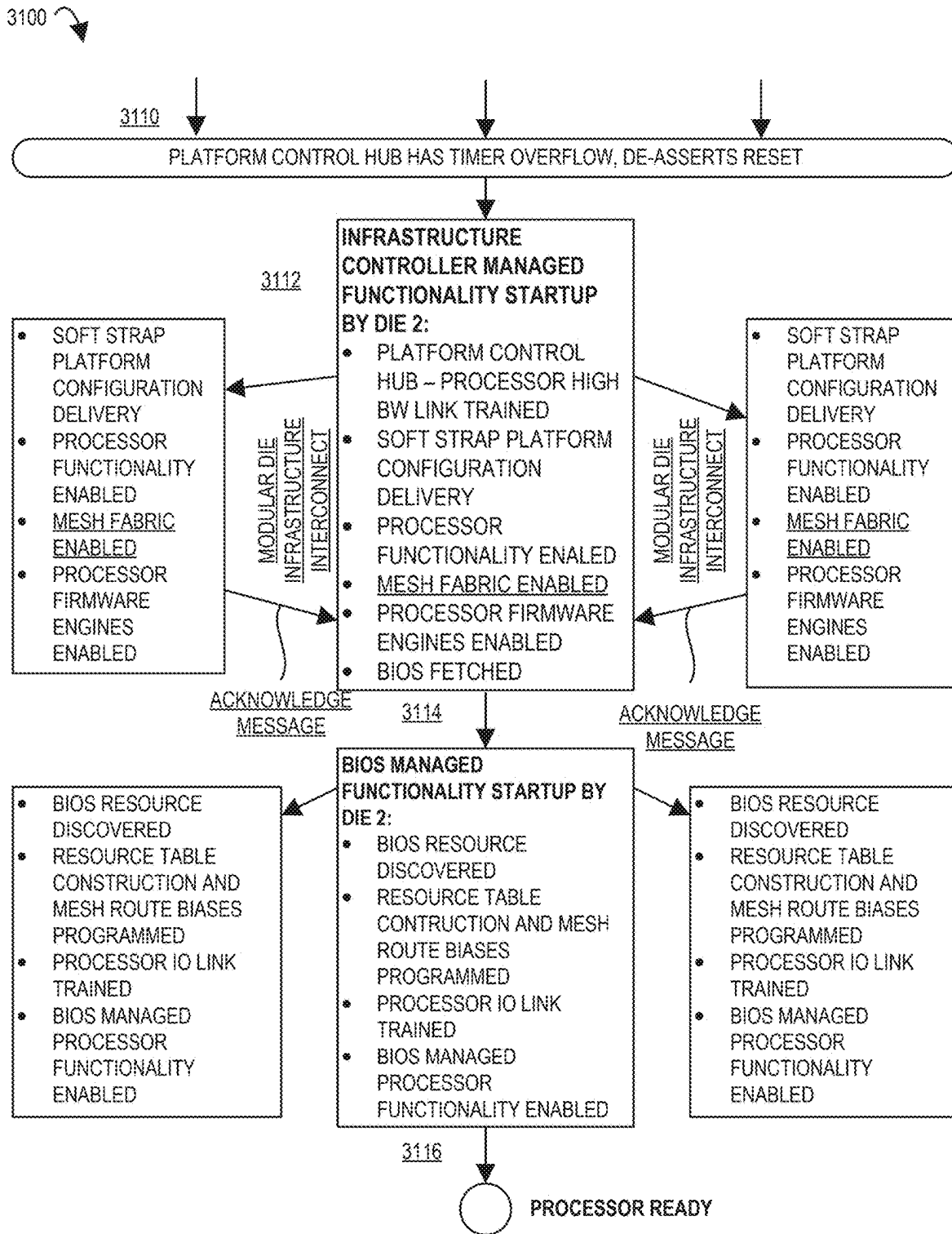


FIG. 31A



MASTER-SLAVE BOOT AND DIE-INDEPENDENT BOOT FLOWS

FIG. 31B

3200 ↗

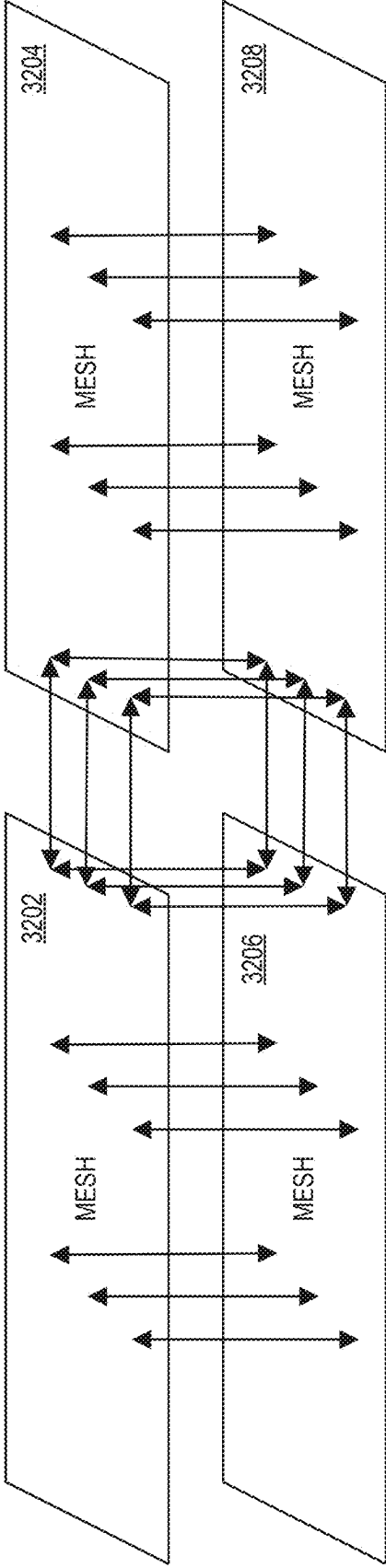
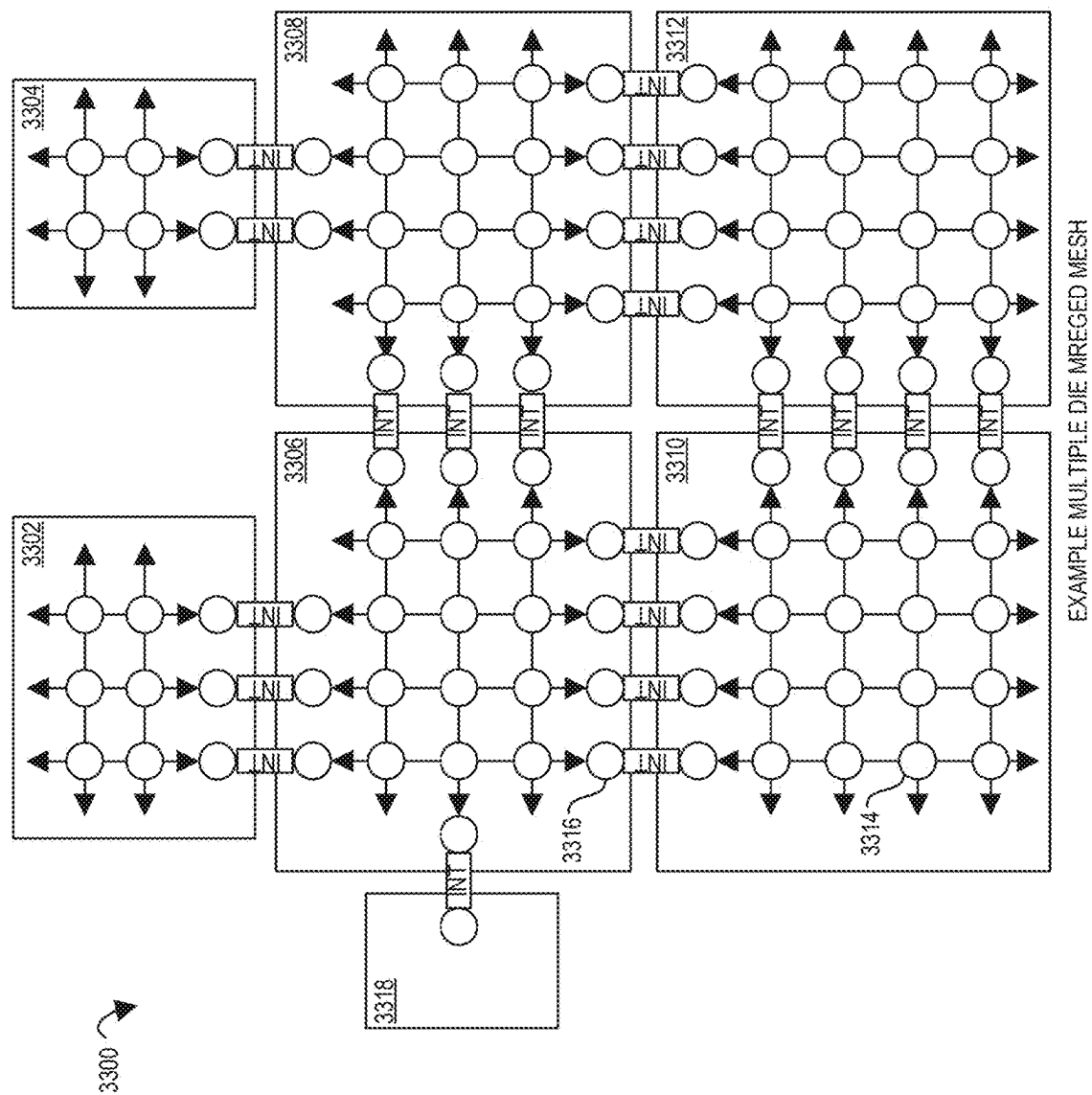


FIG. 32



EXAMPLE MULTIPLE DIE MREGED MESH

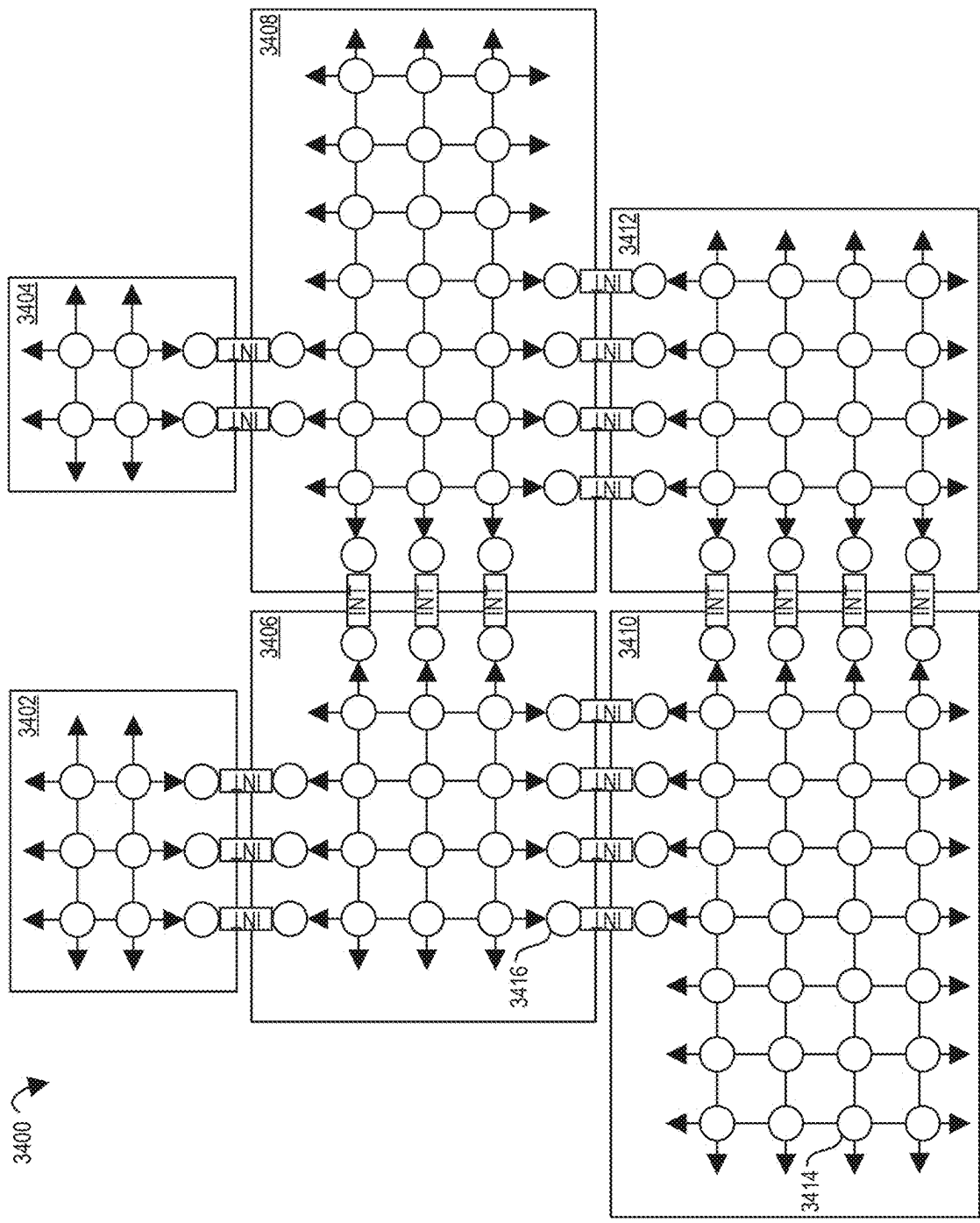


FIG. 34
EXAMPLE MULTIPLE DIE MERGED MESH

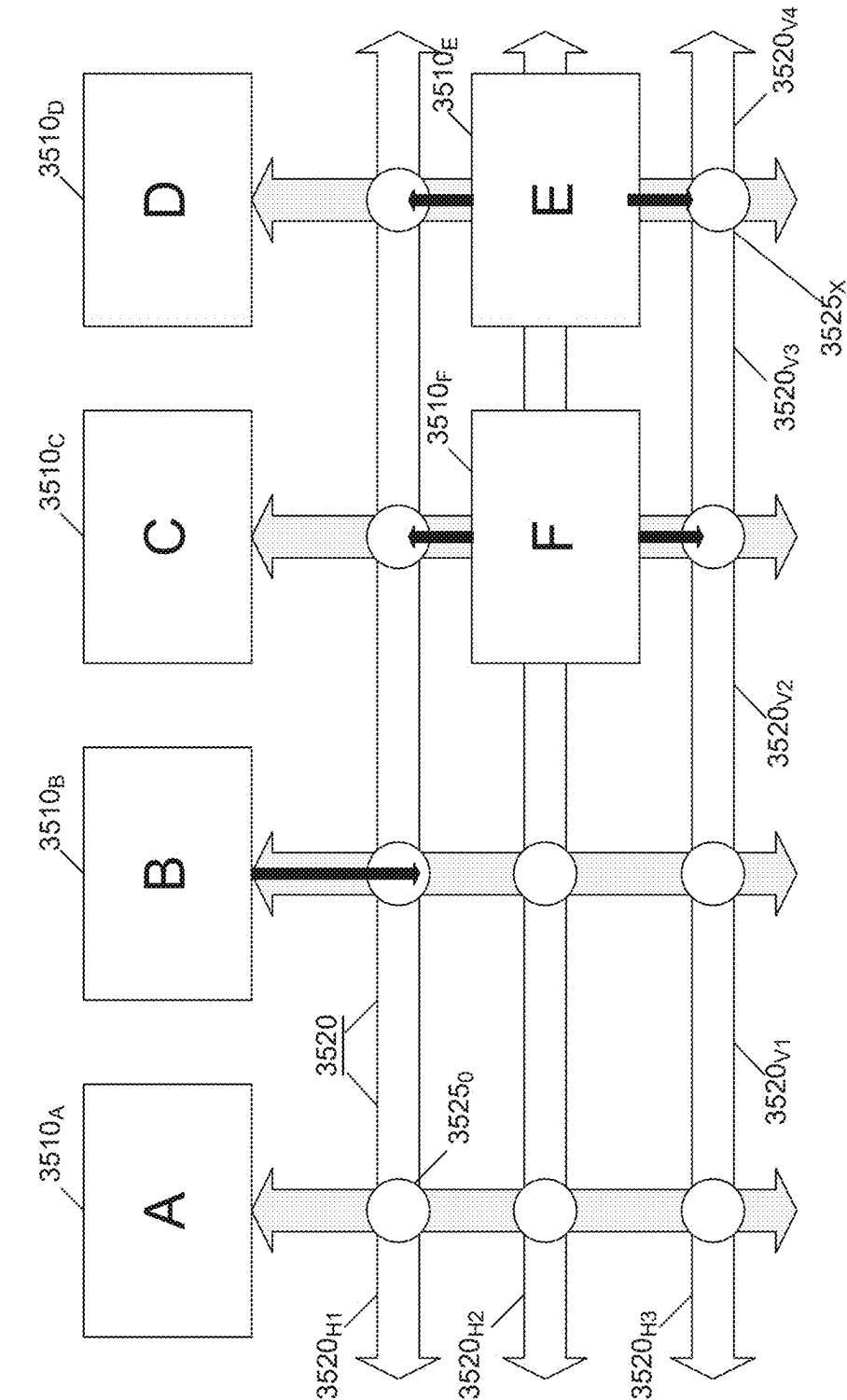


FIG. 35

3500

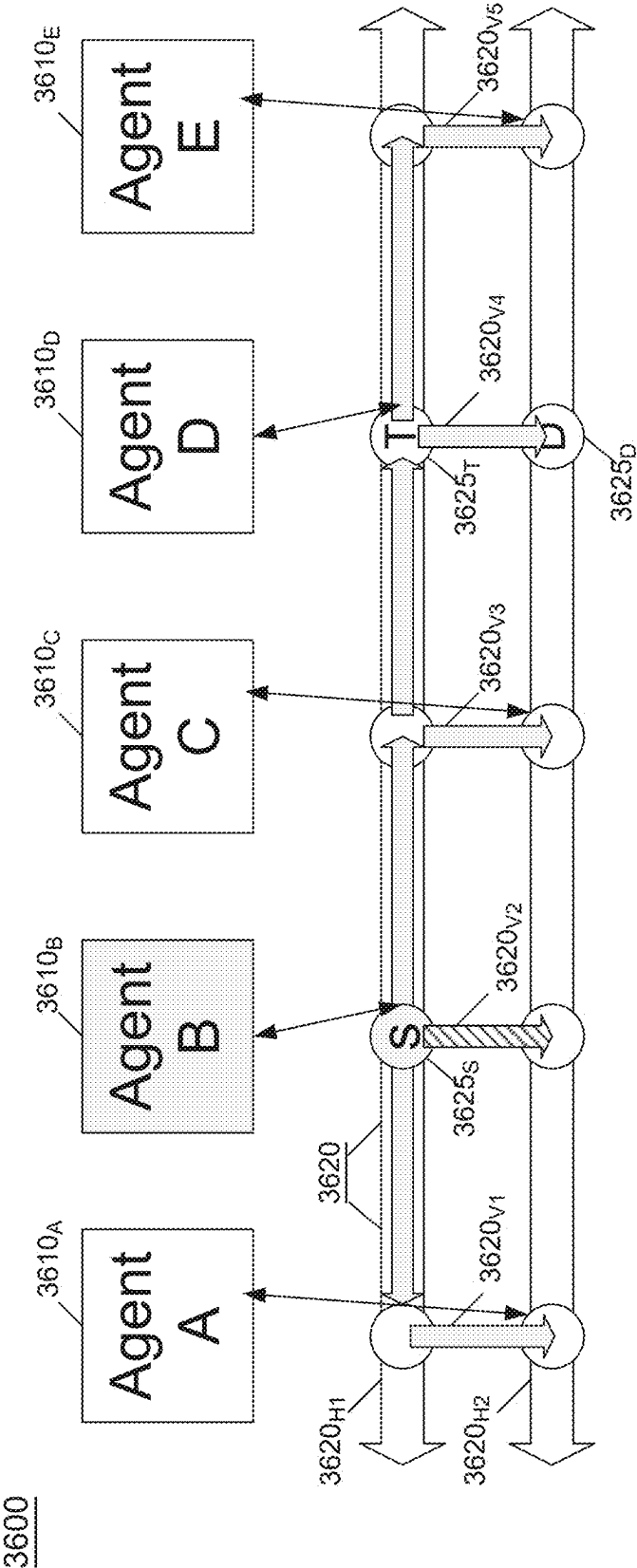


FIG. 36

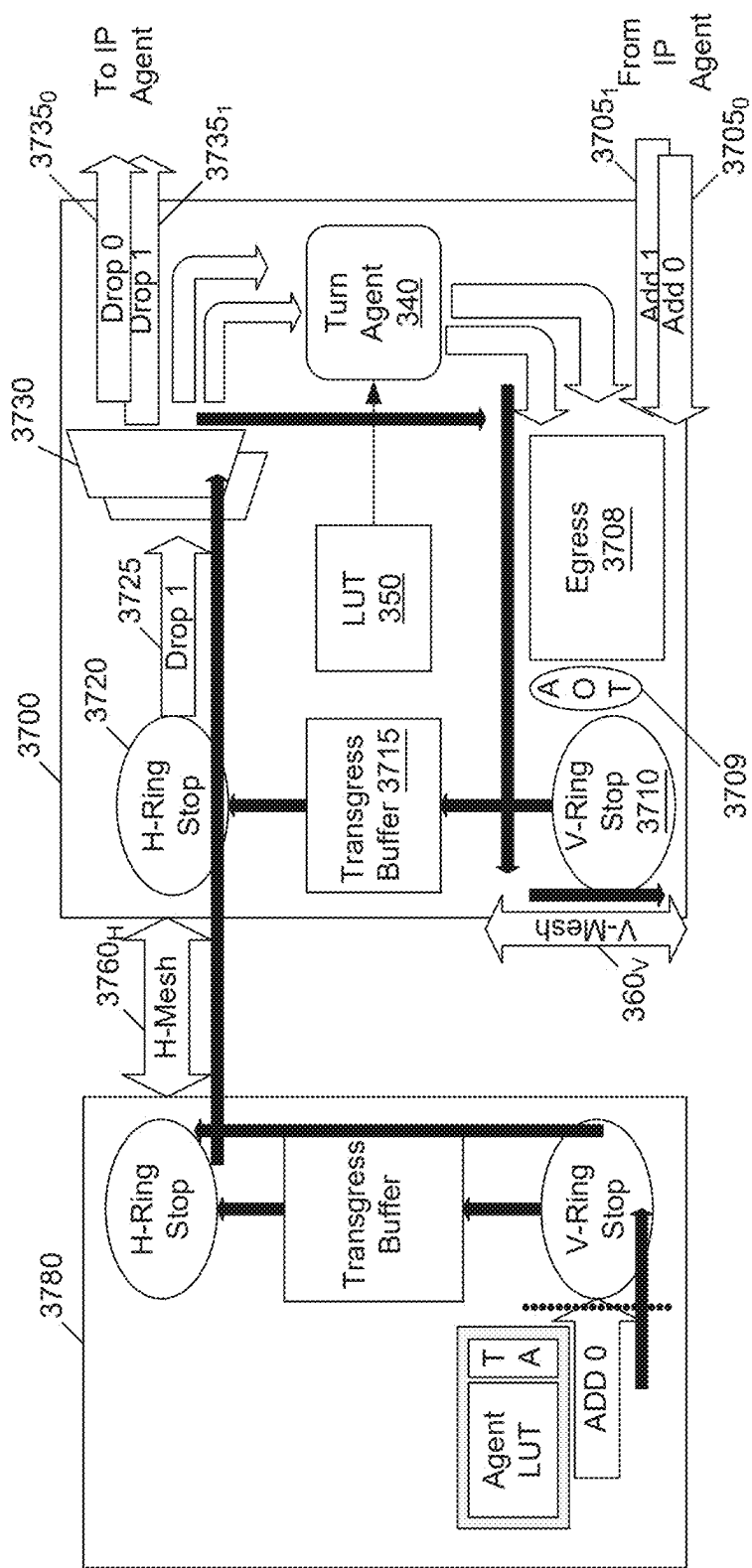


FIG. 37A

3790

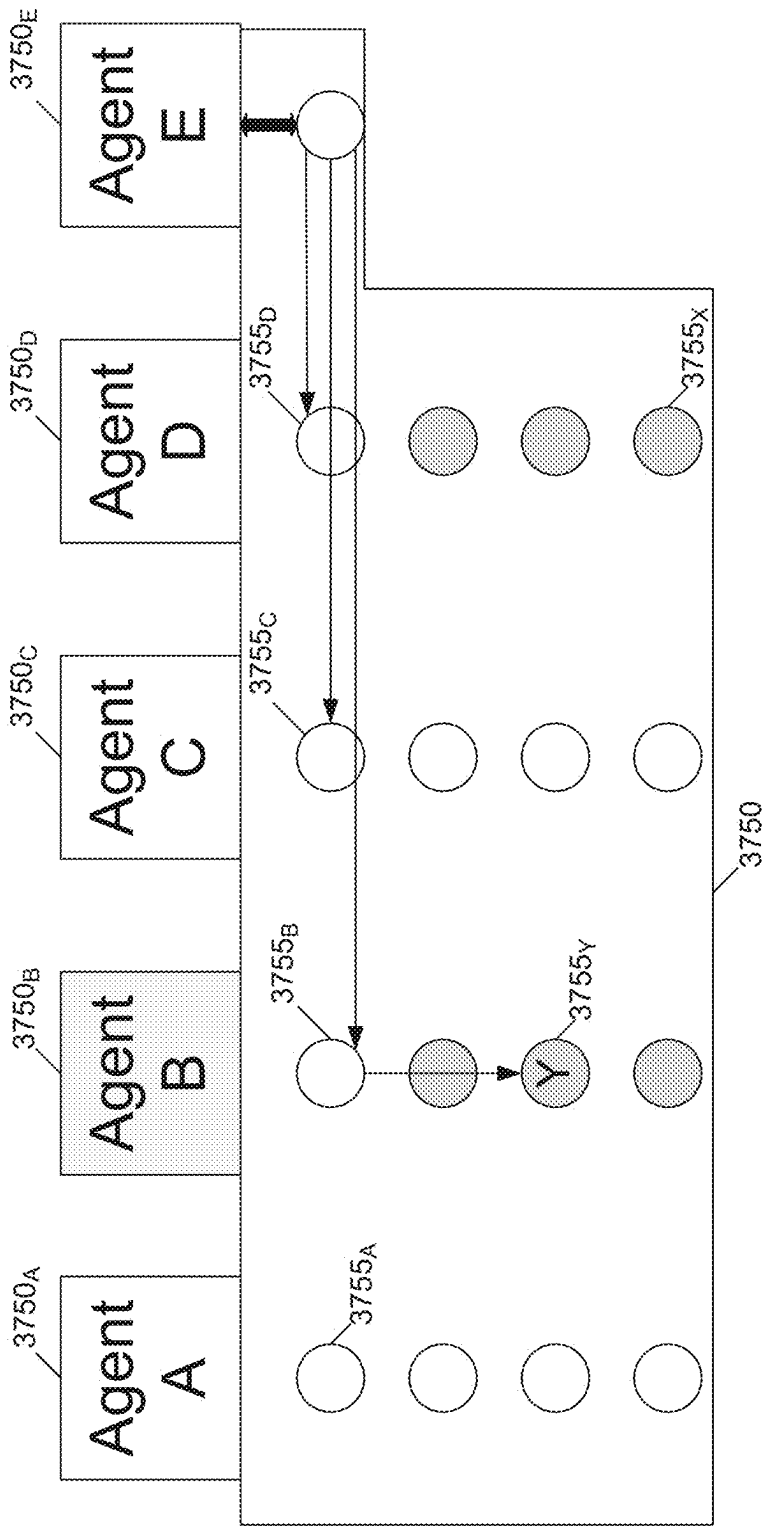
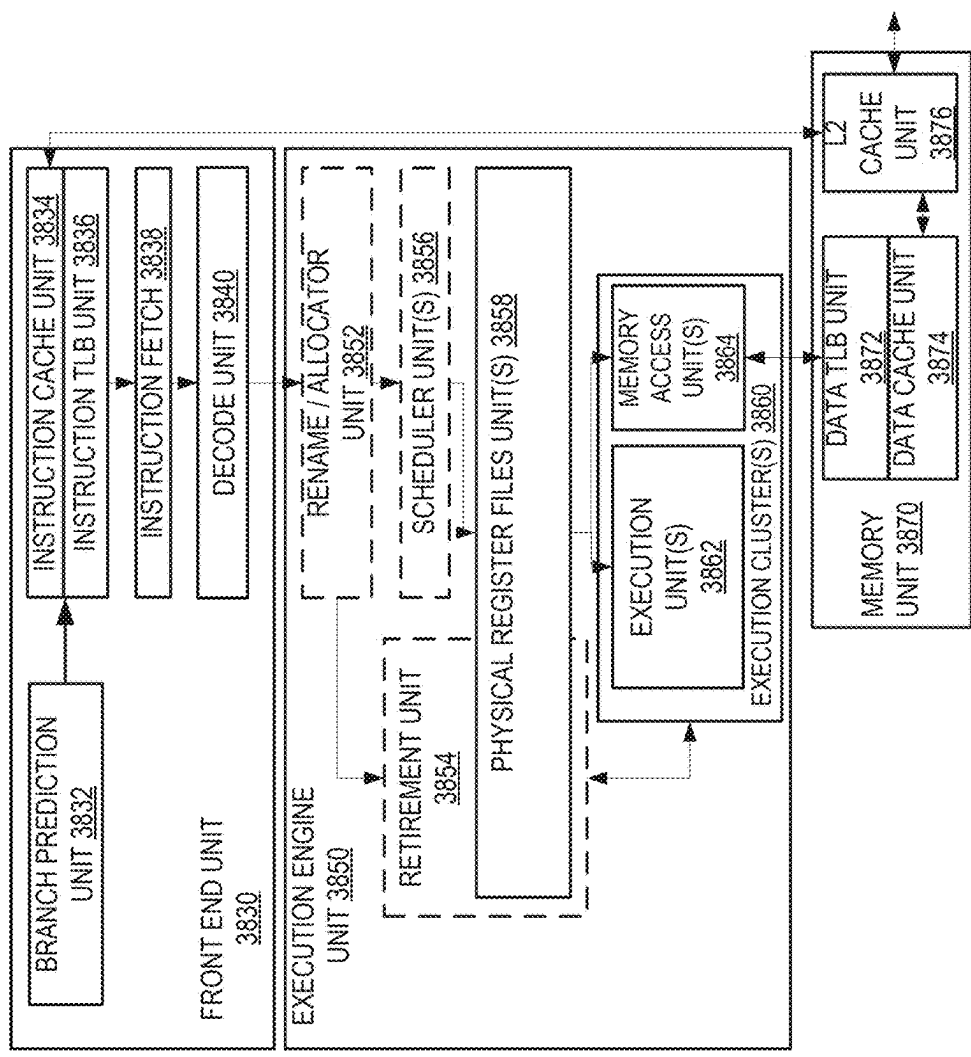


FIG. 37B



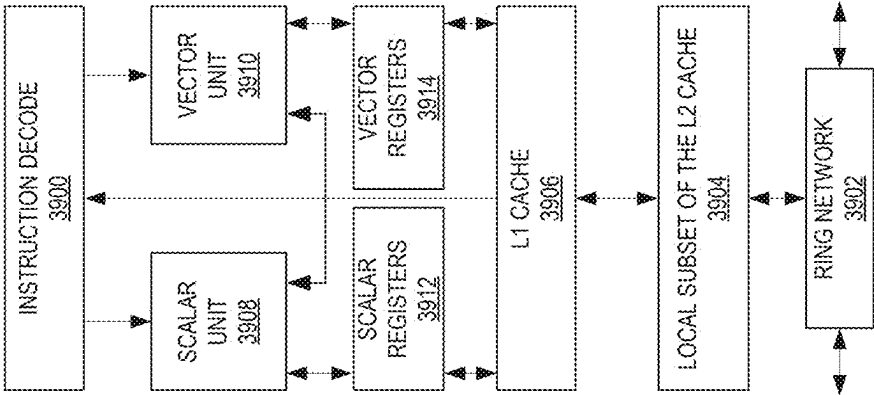


FIG. 39A

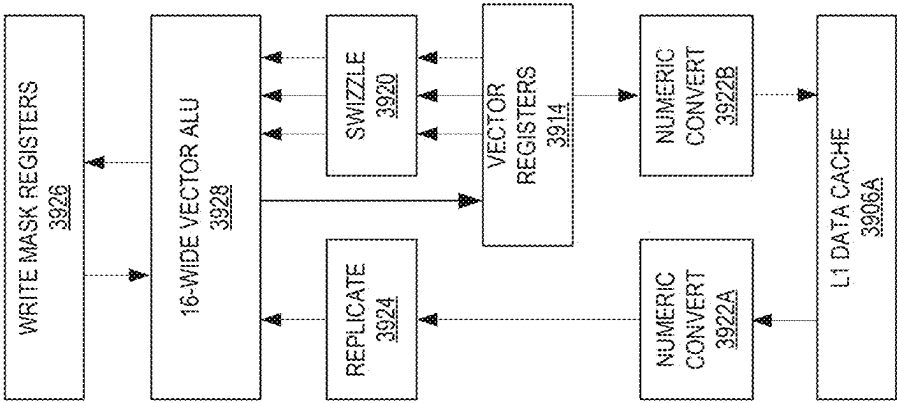


FIG. 39B

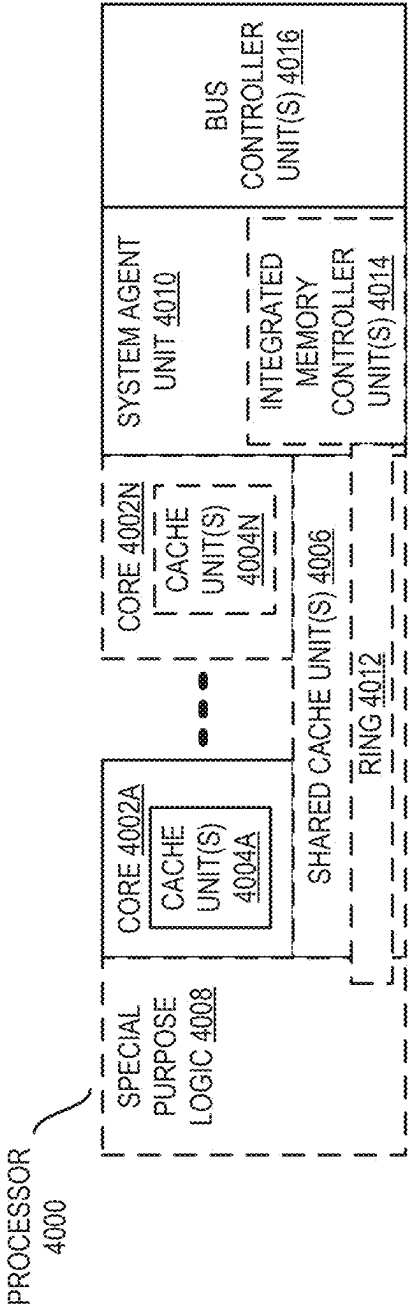


FIG. 40

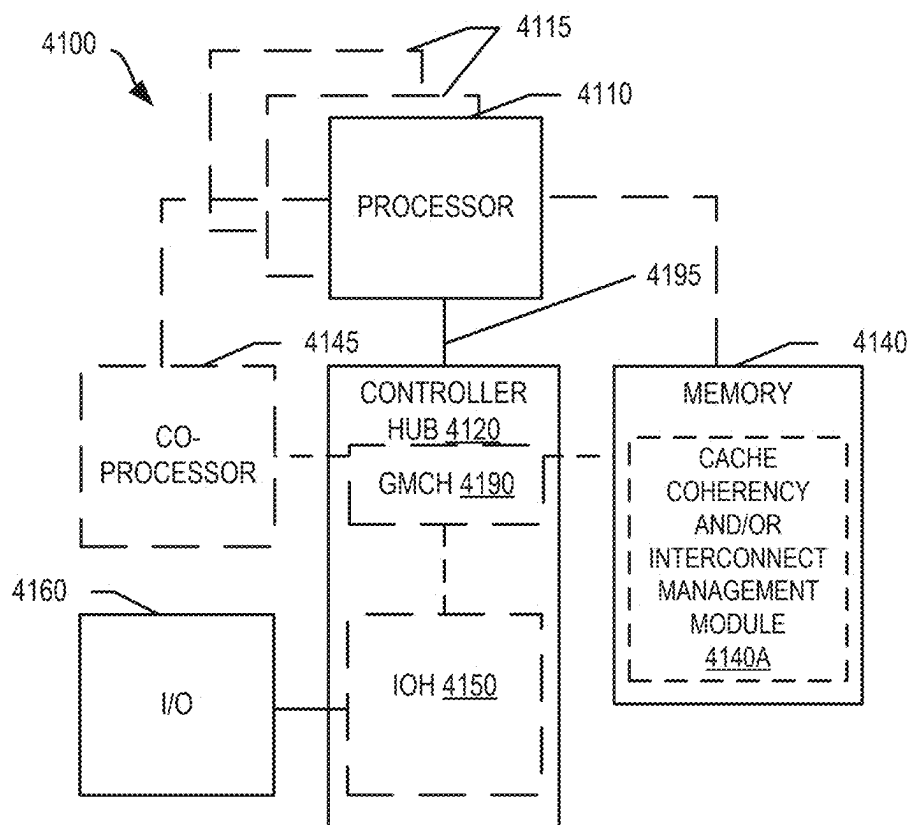


FIG. 41

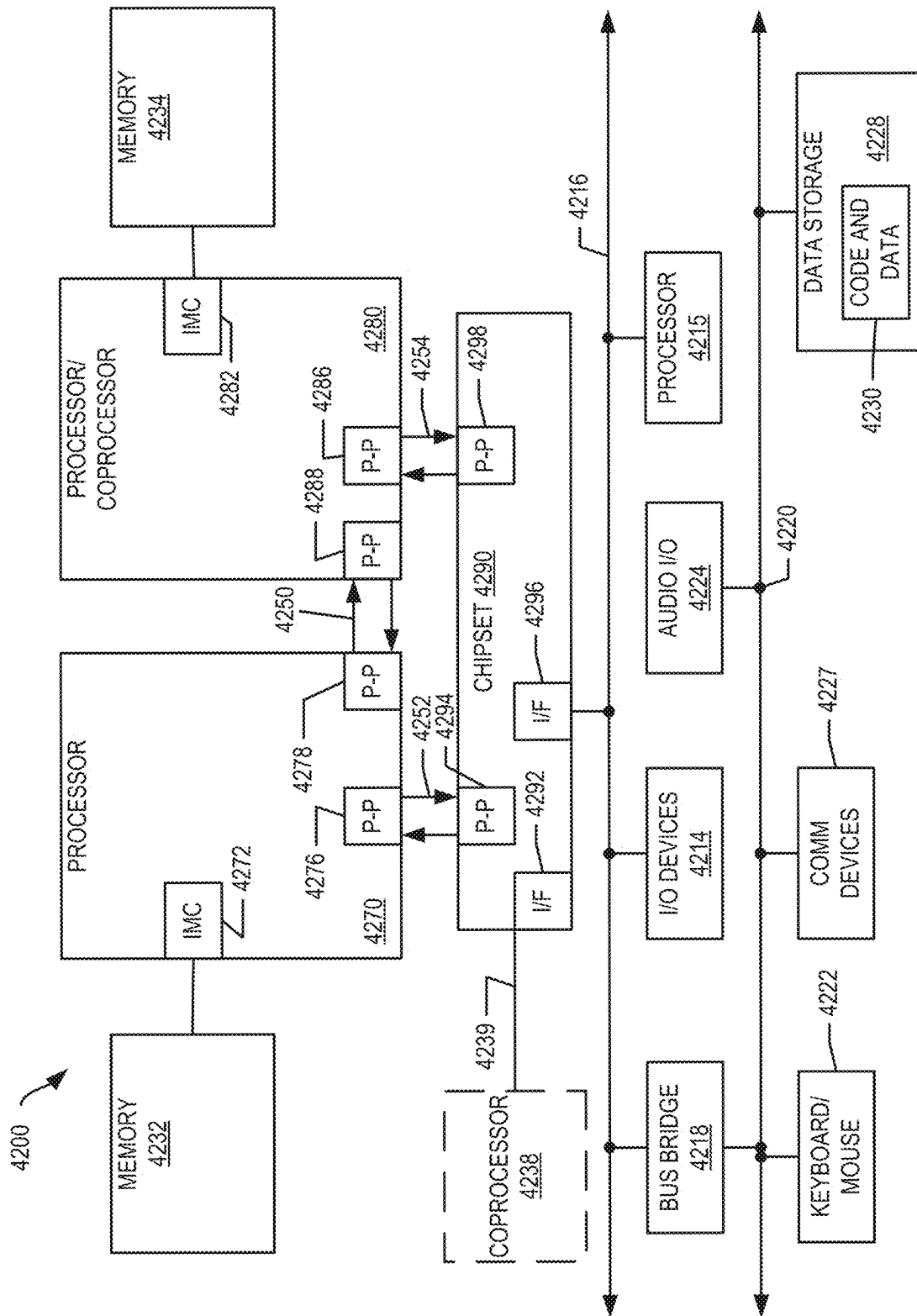


FIG. 42

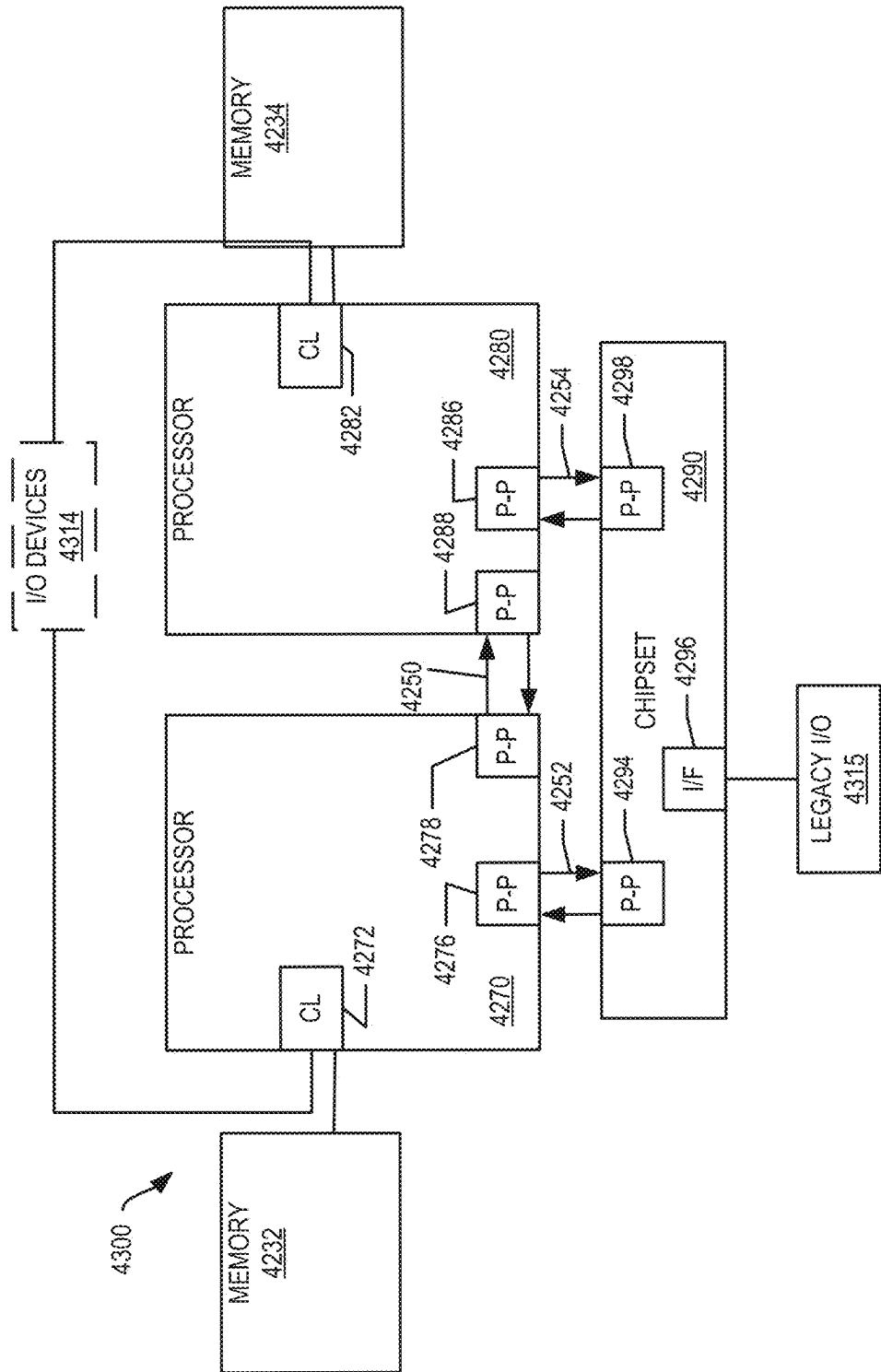


FIG. 43

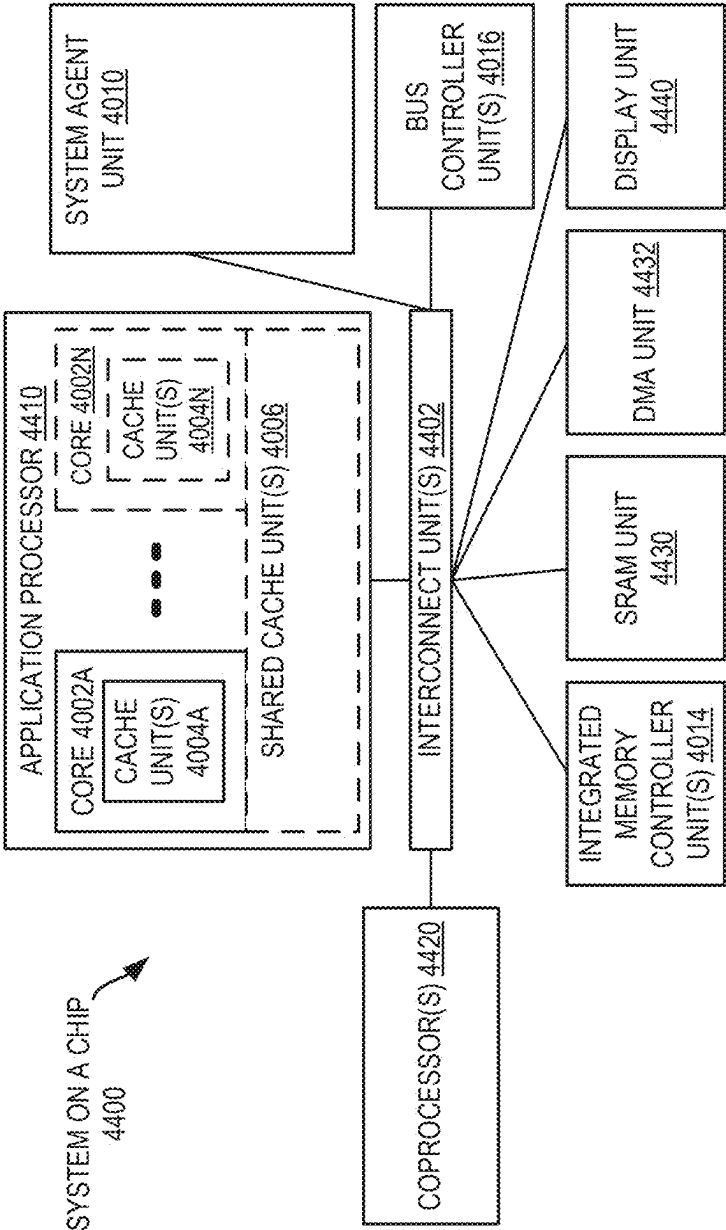


FIG. 44

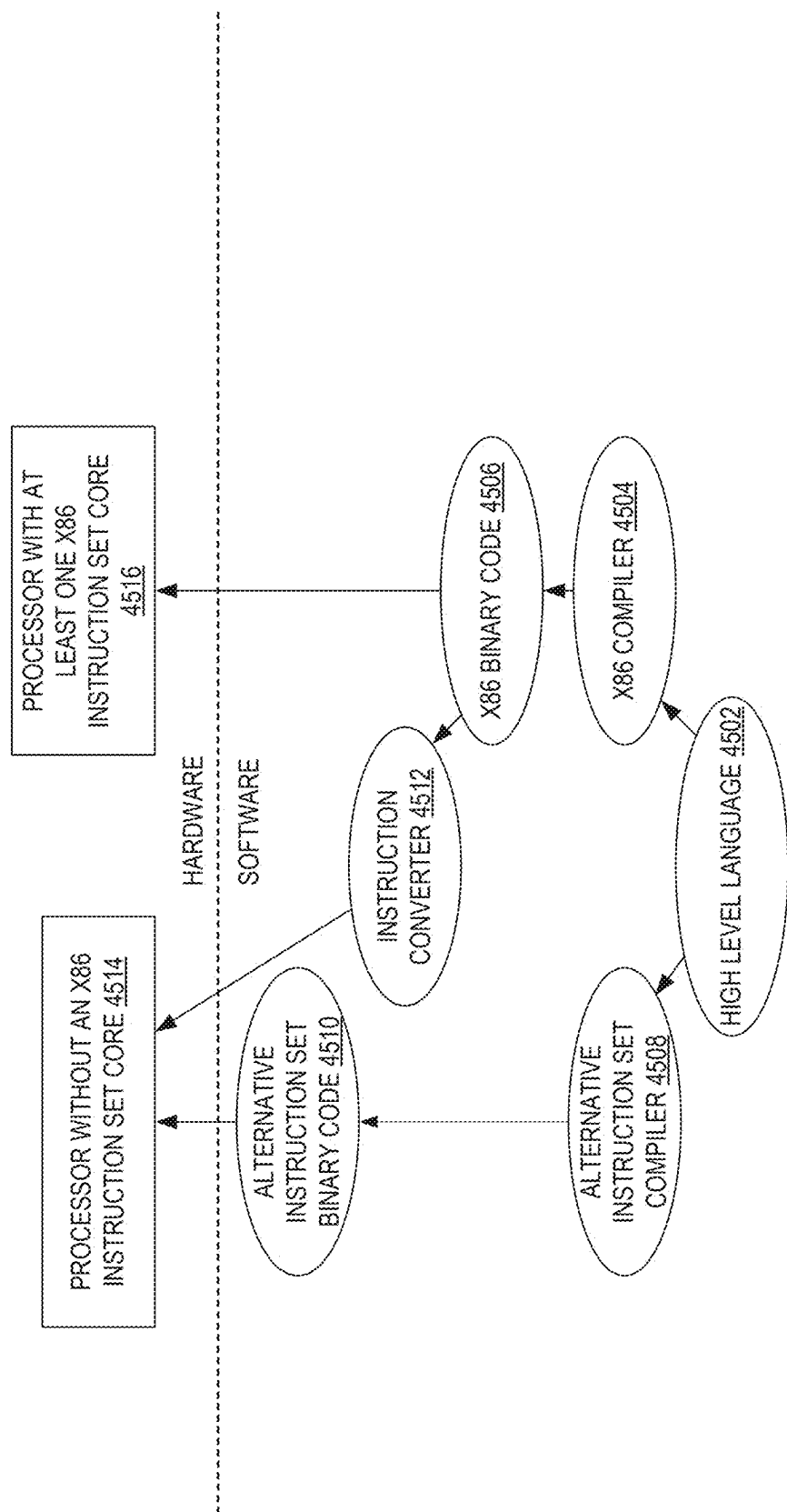


FIG. 45

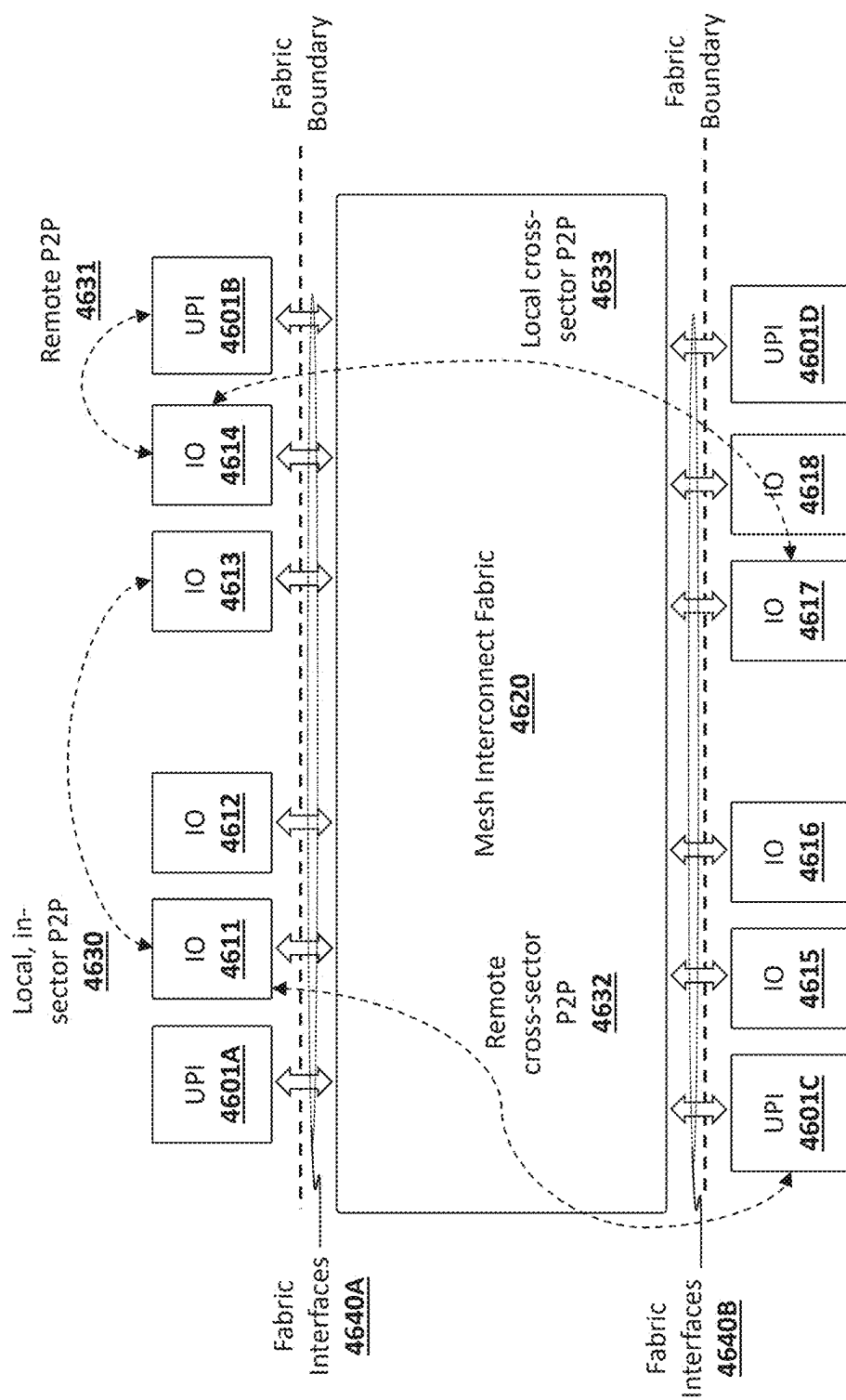


FIG. 46

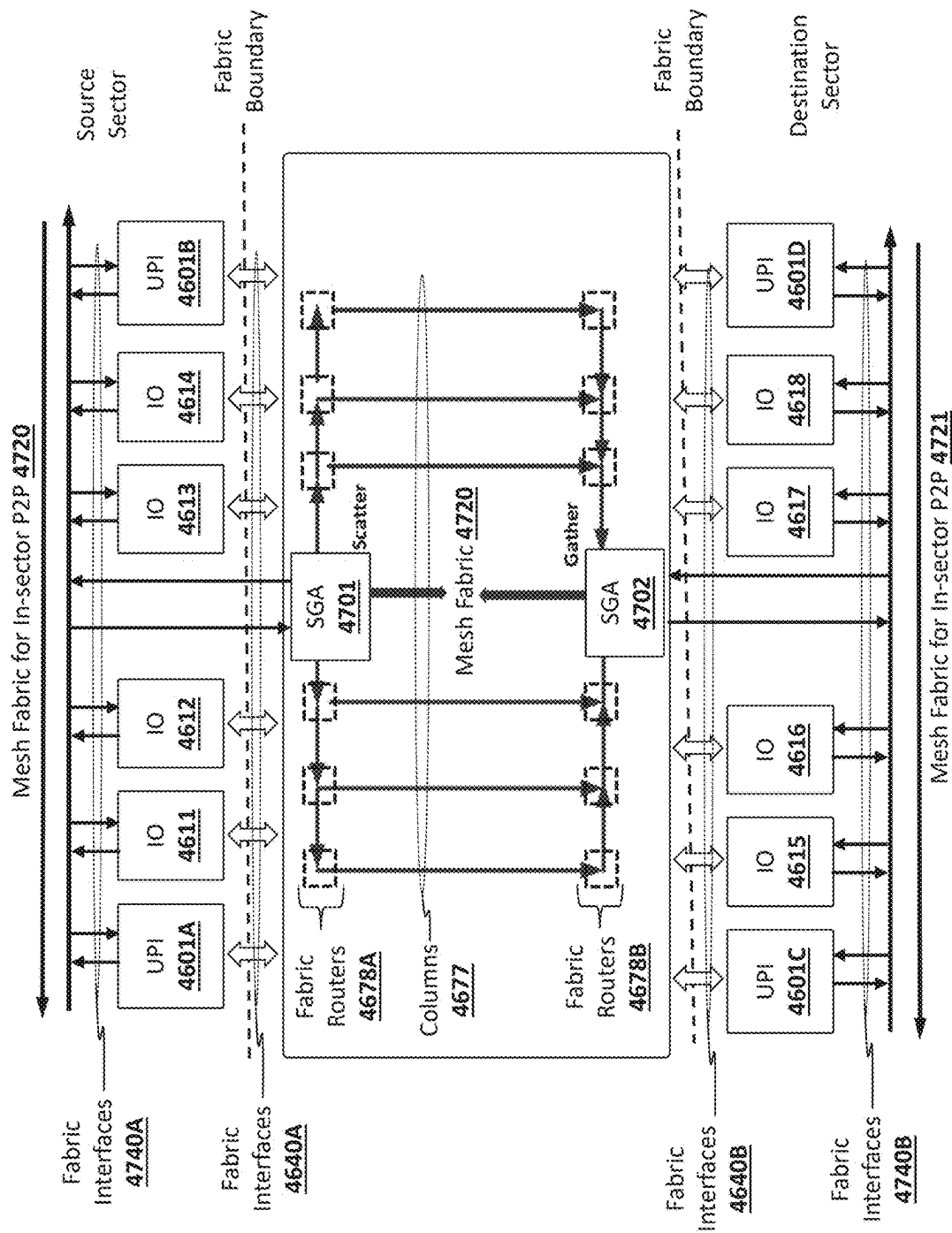
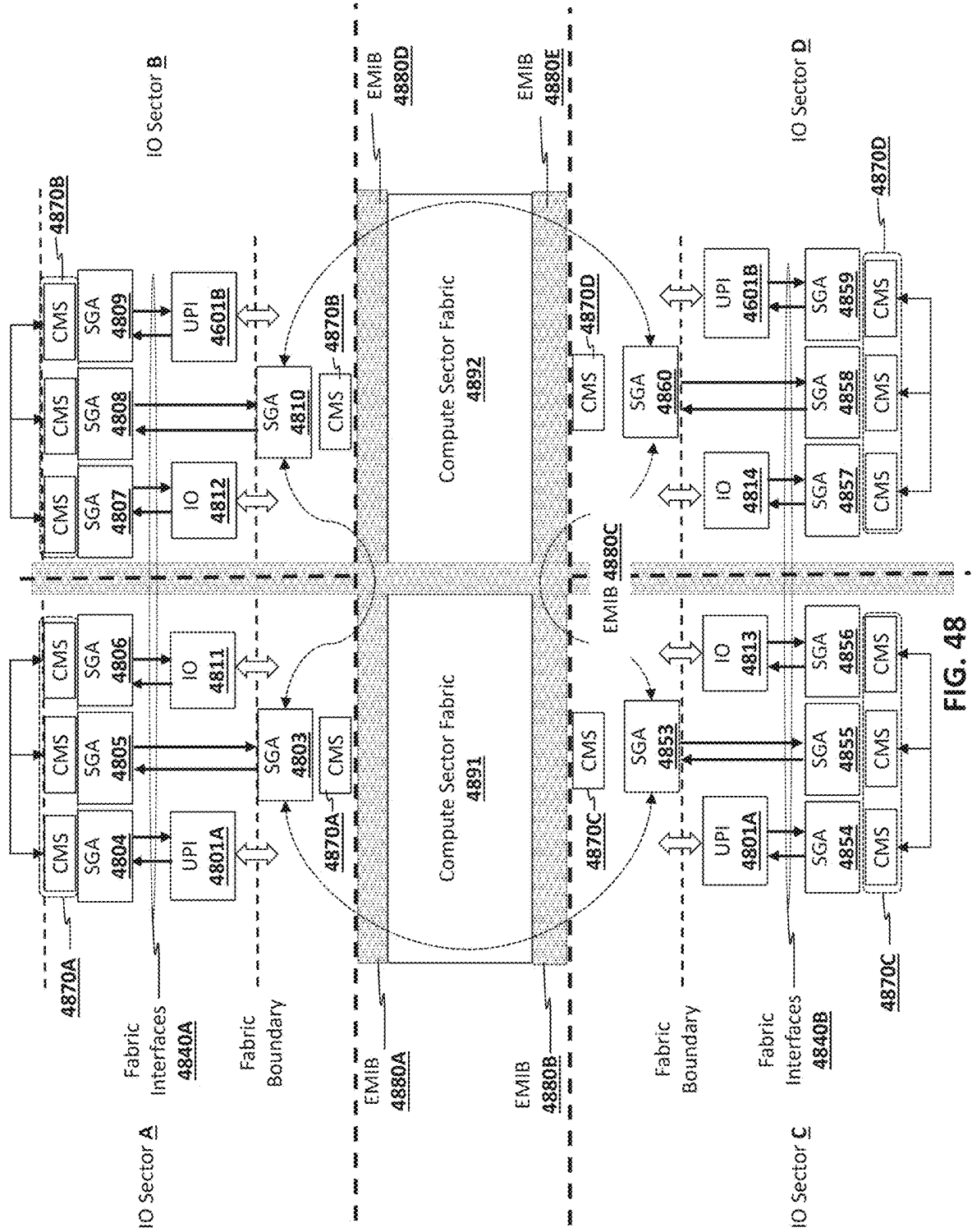
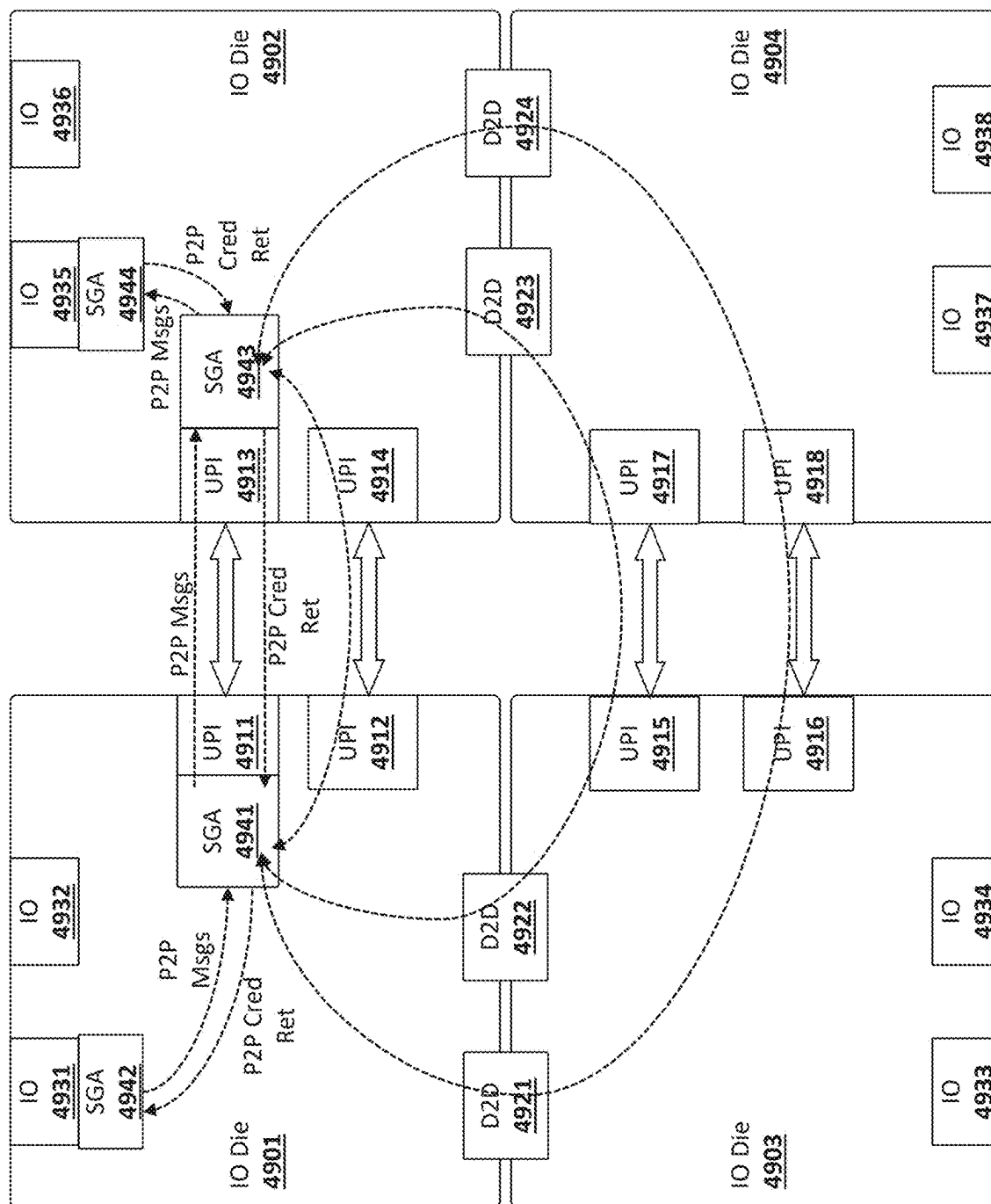


Fig. 47





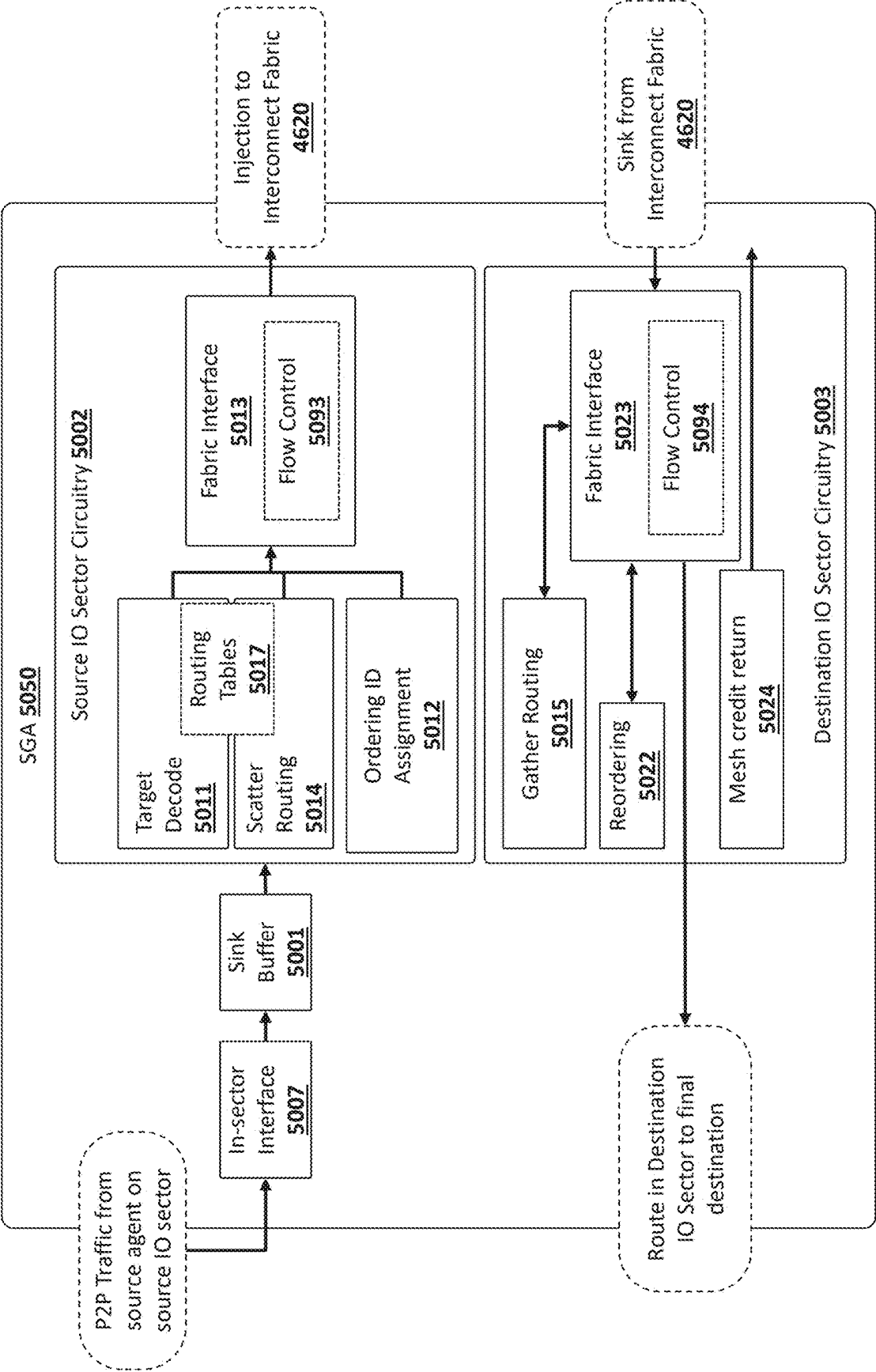
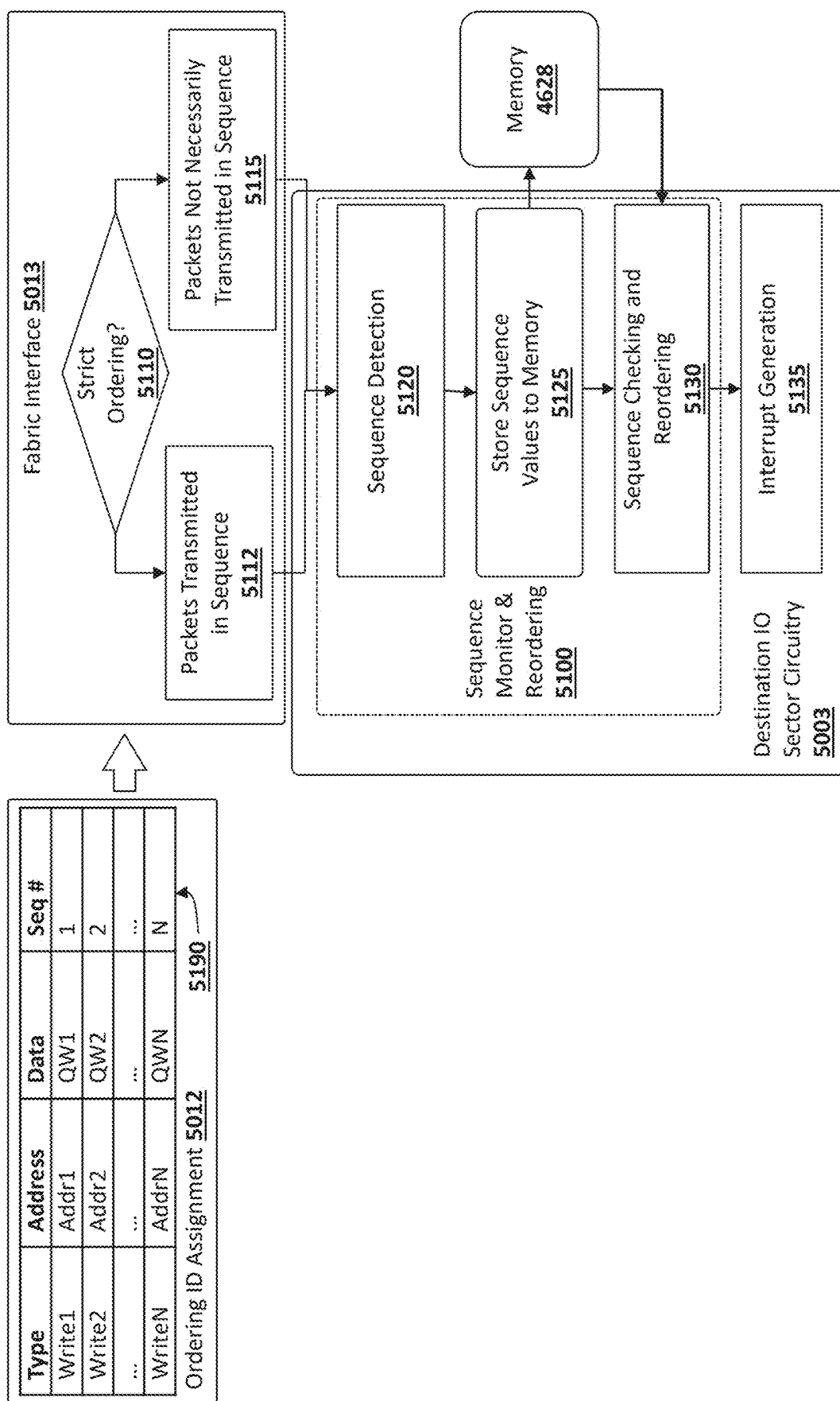
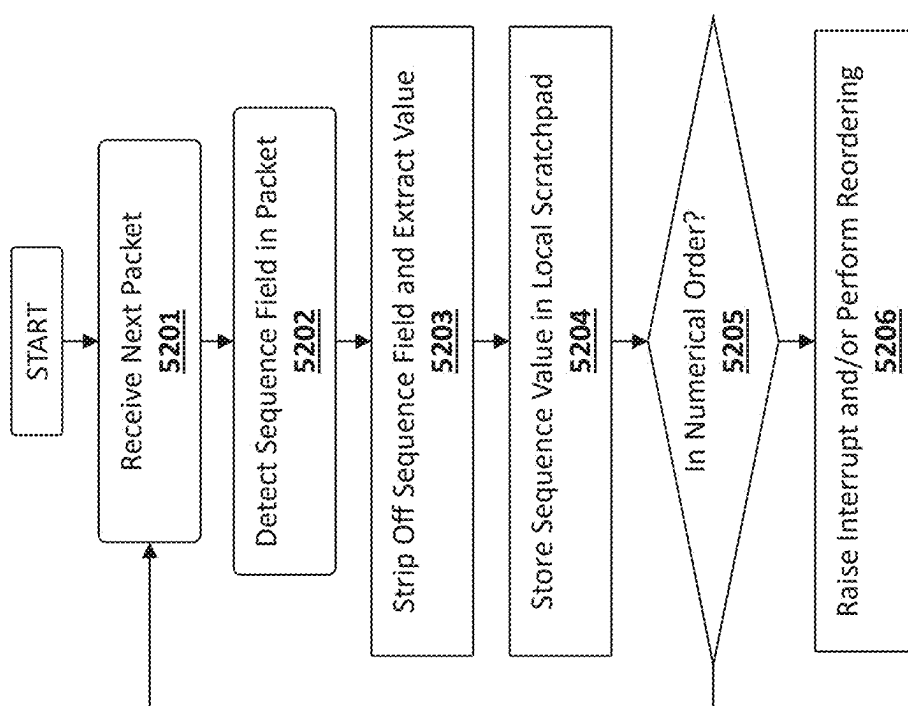


FIG. 50



156E

**FIG. 52**

INTERCONNECT ARCHITECTURE ENABLING PATH DIVERSITY FOR STRONGLY ORDERED MESSAGES

TECHNICAL FIELD

[0001] The disclosure relates generally to electronics, and, more specifically, an embodiment of the disclosure relates to an apparatus and method associated with an interconnect architecture enabling path diversity for strongly ordered messages.

BACKGROUND

[0002] A processor, or set of processors, executes instructions from an instruction set, e.g., the instruction set architecture (ISA). The instruction set is the part of the computer architecture related to programming, and generally includes the native data types, instructions, register architecture, addressing modes, memory architecture, interrupt and exception handling, and external input and output (I/O).

[0003] Mesh interconnect topologies for modern system-on-chip (SoC) processors connect to different core agents, caching agents, memory controllers, input-output (IO) agents and Ultra Path Interconnect (UPI) agents. These mesh interconnect implementations perform fixed routing, following a Y-X routing scheme. More than one route can be enabled on a mesh interconnect using agent devices in the mesh router(s).

[0004] IO agents communicate with other IO agents via ordered peer-to-peer (P2P) transactions. This communication can be between different IO agents in the same socket ("local P2P") or across sockets ("remote P2P") that require routing through UPI links. In floorplans with disaggregated IO dies present on different edges of the interconnect, the local P2P communication can take place within each IO die (In-sector P2P) or across multiple IO dies (cross-sector P2P).

BRIEF DESCRIPTION OF THE DRAWINGS

[0005] The present disclosure is illustrated by way of example and not limitation in the figures of the accompanying drawings, in which like references indicate similar elements and in which:

[0006] FIG. 1 illustrates a hardware processor according to embodiments of the disclosure.

[0007] FIG. 2A illustrates a hardware processor according to embodiments of the disclosure.

[0008] FIG. 2B illustrates a hardware processor according to embodiments of the disclosure.

[0009] FIG. 3 illustrates a hardware processor according to embodiments of the disclosure.

[0010] FIG. 4 illustrates a transmitter circuit of a first die coupled to a receiver circuit of a second die through an interconnect according to embodiments of the disclosure.

[0011] FIG. 5 illustrates a data timing diagram and a clock timing diagram for a first clocking rate according to embodiments of the disclosure.

[0012] FIG. 6 illustrates a data timing diagram and a clock timing diagram for a second clocking rate according to embodiments of the disclosure.

[0013] FIG. 7 illustrates a transmitter circuit of a first die coupled to a receiver circuit of a second die through an interconnect according to embodiments of the disclosure.

[0014] FIG. 8 illustrates a data timing diagram and a clock timing diagram for a first clocking rate according to embodiments of the disclosure.

[0015] FIG. 9 illustrates a data timing diagram and a clock timing diagram for a second clocking rate according to embodiments of the disclosure.

[0016] FIG. 10 illustrates a flow diagram for interconnect programming according to embodiments of the disclosure.

[0017] FIG. 11 illustrates clock phase placement according to embodiments of the disclosure.

[0018] FIG. 12 illustrates a table including clock phase placements according to embodiments of the disclosure.

[0019] FIG. 13 illustrates a digital delay-locked loop (DLL) delay line and digital phase interpolator circuit according to embodiments of the disclosure.

[0020] FIG. 14 illustrates a flow diagram for a frequency transition through an interconnect according to embodiments of the disclosure.

[0021] FIG. 15 illustrates clocking architecture of a receiver circuit according to embodiments of the disclosure.

[0022] FIG. 16 illustrates clock timing diagrams for 1× and 2× clocking rate modes according to embodiments of the disclosure.

[0023] FIG. 17 illustrates clock timing diagrams for 1× and 2× clocking rate modes according to embodiments of the disclosure.

[0024] FIG. 18 illustrates a transmission datapath of a transmitter circuit that includes lane repair circuitry according to embodiments of the disclosure.

[0025] FIG. 19 illustrates clock timing diagrams for a 1× clocking rate mode of a transmitter circuit according to embodiments of the disclosure.

[0026] FIG. 20 illustrates clock timing diagrams for a 2× clocking rate mode of a transmitter circuit according to embodiments of the disclosure.

[0027] FIG. 21 illustrates a receiver datapath of a receiver circuit that includes clock-crossing buffers according to embodiments of the disclosure.

[0028] FIG. 22 illustrates clock timing diagrams for a 1× clocking rate mode of a receiver circuit according to embodiments of the disclosure.

[0029] FIG. 23 illustrates clock timing diagrams for a 2× clocking rate mode of a receiver circuit according to embodiments of the disclosure.

[0030] FIG. 24 illustrates a hardware processor having two dies that share resources via an interconnect according to embodiments of the disclosure.

[0031] FIG. 25 illustrates infrastructure management controllers for a hardware processor having two dies that share resources via an interconnect according to embodiments of the disclosure.

[0032] FIG. 26 illustrates an infrastructure management controller for a hardware processor having four dies that share resources via an interconnect according to embodiments of the disclosure.

[0033] FIG. 27 illustrates infrastructure management controllers for a hardware processor having six dies that share resources via an interconnect according to embodiments of the disclosure.

[0034] FIG. 28 illustrates infrastructure management controllers for a hardware processor having six dies coupled via an interconnect according to embodiments of the disclosure.

[0035] FIG. 29 illustrates a flat communication topology for data exchanges in a multiple die processor according to embodiments of the disclosure.

[0036] FIG. 30 illustrates a hierarchical master and slave communication topology for data exchanges in a multiple die processor according to embodiments of the disclosure.

[0037] FIGS. 31A-31B illustrate a flow diagram for a master and slave boot and a die-independent boot according to embodiments of the disclosure.

[0038] FIG. 32 illustrates a hardware processor according to embodiments of the disclosure.

[0039] FIG. 33 illustrates a hardware processor according to embodiments of the disclosure.

[0040] FIG. 34 illustrates a hardware processor according to embodiments of the disclosure.

[0041] FIG. 35 is a block diagram of an integrated circuit in accordance with an embodiment of the present invention.

[0042] FIG. 36 illustrates a flow diagram according to embodiments of the disclosure.

[0043] FIGS. 37A-B illustrate interconnections between agents over a mesh fabric in accordance with some embodiments.

[0044] FIG. 38A is a block diagram illustrating both an exemplary in-order pipeline and an exemplary register renaming, out-of-order issue/execution pipeline according to embodiments of the disclosure.

[0045] FIG. 38B is a block diagram illustrating both an exemplary embodiment of an in-order architecture core and an exemplary register renaming, out-of-order issue/execution architecture core to be included in a processor according to embodiments of the disclosure.

[0046] FIG. 39A is a block diagram of a single processor core, along with its connection to the on-die interconnect network and with its local subset of the Level 2 (L2) cache, according to embodiments of the disclosure.

[0047] FIG. 39B is an expanded view of part of the processor core in FIG. 39A according to embodiments of the disclosure.

[0048] FIG. 40 is a block diagram of a processor that may have more than one core, may have an integrated memory controller, and may have integrated graphics according to embodiments of the disclosure.

[0049] FIG. 41 is a block diagram of a system in accordance with one embodiment of the present disclosure.

[0050] FIG. 42 is a block diagram of a more specific exemplary system in accordance with an embodiment of the present disclosure.

[0051] FIG. 43, shown is a block diagram of a second more specific exemplary system in accordance with an embodiment of the present disclosure.

[0052] FIG. 44, shown is a block diagram of a system on a chip (SoC) in accordance with an embodiment of the present disclosure.

[0053] FIG. 45 is a block diagram contrasting the use of a software instruction converter to convert binary instructions in a source instruction set to binary instructions in a target instruction set according to embodiments of the disclosure.

[0054] FIG. 46 illustrates an example mesh interconnect fabric with different interconnections between fabric agents;

[0055] FIG. 47 illustrates fabric bridges for efficiently utilizing columns within a mesh fabric in accordance with embodiments of the invention;

[0056] FIG. 48 illustrates a plurality of fabric bridges arranged in sectors with corresponding fabric agents in accordance with some embodiments;

[0057] FIG. 49 illustrates fabric bridges incorporated within a plurality of IO dies in accordance with some embodiments;

[0058] FIG. 50 illustrates embodiments of a fabric bridge including source IO sector circuitry and destination IO sector circuitry;

[0059] FIG. 51 illustrates packet sequencing and reordering implemented in some embodiments; and

[0060] FIG. 52 illustrates a method in accordance with embodiments of the invention.

DETAILED DESCRIPTION

[0061] In the following description, numerous specific details are set forth. However, it is understood that embodiments of the disclosure may be practiced without these specific details. In other instances, well-known circuits, structures and techniques have not been shown in detail in order not to obscure the understanding of this description.

[0062] References in the specification to “one embodiment,” “an embodiment,” “an example embodiment,” etc., indicate that the embodiment described may include a particular feature, structure, or characteristic, but every embodiment may not necessarily include the particular feature, structure, or characteristic. Moreover, such phrases are not necessarily referring to the same embodiment. Further, when a particular feature, structure, or characteristic is described in connection with an embodiment, it is submitted that it is within the knowledge of one skilled in the art to affect such feature, structure, or characteristic in connection with other embodiments whether or not explicitly described.

[0063] A (e.g., hardware) processor, or set of processors, executes instructions from an instruction set, e.g., the instruction set architecture (ISA). The instruction set is the part of the computer architecture related to programming, and generally includes the native data types, instructions, register architecture, addressing modes, memory architecture, interrupt and exception handling, and external input and output (I/O). It should be noted that the term instruction herein may refer to a macro-instruction, e.g., an instruction that is provided to the processor for execution, or to a micro-instruction, e.g., an instruction that results from a processor's decode unit (decoder) decoding macro-instructions. A processor (e.g., having one or more cores to decode and/or execute instructions) may operate on data, for example, in performing arithmetic, logic, or other functions.

[0064] A processor may be formed on a single die, e.g., a single (semiconductor) block of integrated circuits. In one embodiment, a single die may have (e.g., manufacturing) errors or defects that impede or remove certain functionality of the die. This liability to process defect may increase with the die area, as does the fabrication investment at risk of loss in construction of (e.g., large) processors. A processor may be formed on a single die (e.g., fabrication) having all hardware functionality at one design release, e.g., and not have hardware supported features added, enhanced, or optimized where those new capabilities were not in the original design release.

[0065] Certain embodiments herein provide for multiple physically separate (e.g., discrete) dies to be (e.g., electrically) connected together by an interconnect to form a processor. Certain embodiments herein provide for a single

(e.g., monolithic) cache coherency domain over that interconnect. Certain embodiments herein include not packetizing and/or not serializing the data (e.g., transmitted and/or received) over an interconnect (e.g., between dies). Certain embodiments herein reduce the risk associated with a single (e.g., large) die size. Certain embodiments herein allow for the forming of a processor from the same (and/or a mirrored version of a) die duplicated multiple times to create a (e.g., larger) monolithic domain. Certain embodiments herein allow redundancy for yield recovery and/or die testability. For example, different dies and/or different groupings of dies may allow a wide variety of unique processors (e.g., SKUs) with minimal or without re-design efforts. Certain embodiments herein allow a late decision on design cycle whether to manufacture a monolithic design of a die or multiple dies (e.g., a 2 way or 4 way split of the single die). Certain interconnects herein include a transparent queue to cross clock and/or power domains, for example, that may be tuned post silicon. In certain embodiments, an interconnect (e.g., with transparent queue) may have no latency impact, e.g., if both domain are running at the same frequency but running on different power sources. In certain embodiments, a transceiver circuit (e.g., a transmitter circuit and a receiver circuit) includes a transparent queue on both transmitter and receiver circuits, for example, where data is crossing a physical die boundary, e.g., crossing a power domain where each die has a different power source.

[0066] Certain embodiments herein provide a monolithic cache domain across multiple dies (e.g., allowing very large cross bandwidth but also having minimal latency and power impact). Certain embodiments herein allow a scale up in two dimensions (e.g., X-Y) and/or three dimensions (e.g., X-Y-Z). Certain embodiments herein provide for a larger die to connect to smaller die (e.g., multiple dies having a different number of physical connections on their die). Certain embodiments herein allow transportation according to multiple (e.g., any) protocols between dies (e.g., not restricted to a single protocol). Certain embodiments herein provide for a mesh loopback (e.g., micro) architecture, e.g., to tolerate die to die differences. Certain embodiments herein add an entry into a look-up table (LUT) to indicate if data (e.g., a cache line) is to cross a physical die boundary, e.g., to pass through an interconnect between two die. Certain embodiments herein allow for independent (e.g., power and/or cache) domains as needed, e.g., to help yield recovery by disabling row and/or column of an (e.g., mesh) interconnect. Certain embodiments herein allow for one die to run at a different frequency than another die of that hardware processor. Certain transport protocols herein enable a high speed interconnect between multiple dies and/or seamless crossing of the die boundaries. Alternatively to using those protocols as die to die connection, certain embodiments herein may use other solutions, e.g., utilizing an interposer.

[0067] Certain embodiments of an interconnect between multiple dies provides one or more of: (e.g., very high) increased bandwidth (BW), reduced pin count but allowing full cross sectional BW, $\frac{1}{4}$ pins used with $4\times$ frequency of a die, $\frac{1}{2}$ pins used with dynamic $1\times/2\times$ modes, for example, $1\times$: half BW (e.g., operating frequency matching the die, since $\frac{1}{2}$ pin, $\frac{1}{2}$ BW) with low power and/or latency impact, no packetization (e.g., for any die to die connection) for minimal latency impact, lower frequency and/or lower error rate (e.g., an error rate similar or less than the error rate on silicon) (e.g., to allow no error protection utilized on a

between dies interconnect link or error protection for an on die interconnect utilized on a between dies interconnect link), and, for example, $2\times$: full BW full performance with increased power and/or latency, double the operating frequency versus die frequency, and algorithm(s) for switching between the two modes. Certain embodiments herein of an interconnect between multiple dies provides decreased latency and/or increased BW of the interconnect, e.g., much less than current die to die interconnect technology and/or equal or substantially equal to an on die interconnect.

[0068] Certain embodiments herein provide sharing processor primary resources over a high bandwidth and low-latency electrical interconnect such that the performance in accessing remote die resources is substantially similar or very near the performance of a monolithically fabricated integrated die. Certain embodiments herein provide sharing processor infrastructure resources to enable intimate management of power, thermal, clocking, reset, configuration, error handling, etc. with an electrical interconnect such that the performance in accessing remote die resources is substantially similar or very near the performance of a monolithically fabricated integrated die. Certain embodiments herein reduce the fabrication yield risk associated with a single large die size. Certain embodiments herein allow scaling to (e.g., larger) numbers of functional logic circuit components to offer redundancy for yield recovery and/or special uses such as die testability. Certain embodiments herein allow a late (e.g., or any time) decision on design cycle whether to manufacture a monolithic design of a die or multiple dies (e.g., a 2 way or 4 way split of the single die).

[0069] Certain embodiments herein allow combinations of dissimilar dies to enable staging over time design completion for some dies or for some dies to be manufactured in more matured or special fabrication process, as well as better monetizing some older dies from previous products. Certain embodiments herein allow combinations of dissimilar dies and/or quantities of dies to enable a wide variety of unique processors products (e.g., SKUs) with minimal or without re-design efforts.

[0070] Certain embodiments herein provide for a larger die to connect to smaller die and/or multiple dies having a different number of physical connections on their die. Certain embodiments herein allow for the forming of a processor from the same and/or a mirrored version of a die duplicated multiple times to create a larger monolithic domain. Certain embodiments herein allow a scale up in two dimensions (e.g., X and Y axes in Cartesian coordinates) and/or three dimensions (e.g., X, Y, and Z axes in Cartesian coordinates).

[0071] Certain embodiments herein provide circuitry (e.g., PHY) to deliver a low-latency high-bandwidth die-to-die coherent connection, e.g., substantially similar to the monolithic experience. Certain embodiments herein provide for performance neutrality and power saving capabilities equivalency to the monolithic case. Certain embodiments herein provide for the cohesive flow of individual dies in wafers into packaged modular die products. Certain embodiments herein provide for modularity and extensibility of tiling several modular dies (e.g., heterogeneous modular dies). Certain embodiments herein allow dies to influence each other seamlessly and unencumbered with security protection despite die exposure of private sideband messaging between them.

[0072] FIG. 1 illustrates a hardware processor 100 according to embodiments of the disclosure. Although not depicted, certain circuitry (e.g., decode unit(s), execution unit(s), core(s), cache coherency circuitry, cache(s), or other components) may be utilized, for example, as discussed below. In one embodiment, the processor components on a single die 102 may be coupled together via an interconnect, such as the mesh interconnects illustrated in FIG. 1. For example, die 102 may include component 108 and component 110 that communicate with each other through the mesh interconnect. In one embodiment, physically separate die 102 is to communicate with physically separate die 104 through interconnect 106. Die and/or interconnect may include a transceiver to transmit data between die 102 and die 104. Note that a single headed arrow herein may not require one-way communication, for example, it may indicate two-way communication (e.g., to and from that component). Any or all combinations of communications paths may be utilized in certain embodiments herein.

[0073] In one embodiment, each of die 102 and die 104 are identical. In another embodiment, die 104 is a mirror image of die 102. In one embodiment, die 102 and die 104 are different, for example, each representing a portion of a single die design that has been cleaved into multiple physical dies that are then joined together (e.g., electrically coupled) via an interconnect.

[0074] In one embodiment, a mesh interconnect of a die does not depend on a connection to another die to function, for example, the data signals (e.g., requests and/or answers) may loop back into that die, e.g., if interconnect 106 is not functioning or present. In one embodiment, such data signals are not blocking signals (e.g., not fences).

[0075] Cache coherency circuitry in each of the plurality of physically separate dies may be switchable between a master mode and a slave mode. In one embodiment, a management circuit (e.g., a controller) is to set one of the cache coherency circuits in each of the plurality of physically separate dies as master, e.g., and the rest as slave to the master. Cache coherency circuitry may be within a controller, e.g., controller(s) in FIGS. 25-28.

[0076] FIG. 2A illustrates a hardware processor 200A according to embodiments of the disclosure. In the depicted embodiment, die 202 and 204 are smaller than die 206, die 208, die 210, and die 212. Each of the depicted dies is coupled to an adjacent die via an interconnect (INT). Die 202 is depicted as having two connections (e.g., discrete interconnects) with die 206. Die 204 is depicted as having a different number of (e.g., three) connections (e.g., discrete interconnects) with die 208. Die 206 is depicted as having four connections (e.g., discrete interconnects) with die 208. Die 210 is depicted as having a different number of (e.g., three) connections (e.g., discrete interconnects) with die 212.

[0077] The intersection of mesh interconnect of a die (e.g., intersection 214 or intersection 216 of die 206) may be the access point into the mesh interconnect, e.g., by a circuit component. In one embodiment, multiple (e.g., any) mesh configurations with different sizes on their respective die are coupled together by certain embodiments herein. In one embodiment, a die with a mesh interconnect is coupled to a die without a mesh interconnect, for example, die 218 is depicted in FIG. 2A as coupled to mesh interconnect of die 206 though single interconnect (INT).

[0078] FIG. 2B illustrates a hardware processor 200B according to embodiments of the disclosure. In the depicted embodiment, die 202 and 204 are smaller than die 206, die 220, die 222, and die 212. Die 220 is depicted as including a different mesh interconnect than die 222, e.g., having a different number of intersections. FIG. 2B illustrates that certain of a plurality of dies may be different in certain embodiments (e.g., in one embodiment, they are not symmetric). FIG. 2B illustrates that a mesh interconnect on a die may be different than another mesh interconnect on a different die in certain embodiments (e.g., in one embodiment, they are not symmetric).

[0079] FIG. 3 illustrates a hardware processor 300 according to embodiments of the disclosure. A mesh interconnect is not shown in each die for clarity, but it may be utilized, e.g., as in FIG. 1 or 2. FIG. 3 illustrates a three dimensional stacked architecture. A plurality of dies may extend in any single direction (e.g., with an interconnect(s) between each die). In the depicted embodiment, die 302 and die 304 extend in a first, single plane and die 306 and die 308 extend in a second, different single plane that is laterally spaced from the first single plane. A die may be affixed to another substrate, e.g., a mounting substrate (not depicted).

[0080] In certain embodiments, a first die communicates with (e.g., to and/or from) one or more other dies, e.g., via an electrical connection therebetween. A transceiver (e.g., including a transmitter circuit and/or receiver circuit) may be utilized in one or more of the dies and/or in an interconnect between the dies. A transceiver (e.g., transceiver circuit) may include a physical transport layer (e.g., PHY) circuit (e.g., Input/Output PHY or I/O PHY). Transceivers may be used for communication between multiple dies, e.g., multiple dies that comprise a split-die processor arrangement. In one embodiment, one or more of multiple dies has one or more of its I/O ports (e.g., mesh wires) electrically coupled to the I/O ports (e.g., mesh wires) of another die or dies. In one embodiment, one or more of multiple dies includes a mesh interconnect within the die and each mesh interconnect may have one or more of its I/O ports (e.g., mesh wires) electrically coupled to the I/O ports (e.g., mesh wires) of a mesh interconnect of another die, e.g., at a die boundary crossing. An electrical coupling of dies may be customized for optimized power and latency performance. The couplings (e.g., wires) may be bi-directional, uni-directional, or a combination of both. The physical medium connecting and allowing signaling between the multiple die transceivers (e.g., I/O PHYs) may be an interconnect or other electrical connection.

[0081] The transceiver (e.g., I/O PHY) lanes and/or interconnect lanes (e.g., communication lanes) may be programmable to run in multiples of the processor (e.g., mesh interconnect) (e.g., on die) wire data transmittal rates (e.g., data rates). For example, a one times (1×) (e.g., PHY) rate of clocking of data (e.g., clocking rate) is a 1:1 ratio between the interconnect and/or transceiver (e.g., PHY I/O) (e.g., lane) data transmittal rate (e.g., data rate) and the die (e.g., mesh interconnect or mesh wire) data transmittal rate (e.g., data rate). For example, a two times (2×) (e.g., PHY) rate of clocking of data (e.g., clocking rate) is a 2:1 ratio between the interconnect and/or transceiver (e.g., PHY I/O) (e.g., lane) data transmittal rate (e.g., data rate) and the die (e.g., mesh interconnect or mesh wire) data transmittal rate (e.g., data rate). In one embodiment, the interconnect and the portions of the transceiver coupled directly to the intercon-

nect have the same data rate, e.g., different than a die's internal (e.g., intra-mesh) interconnect data rate. As another example, other ratios are possible, e.g., 3x, 4x, 5x, 6x, 7x, 8x, 9x, 10x, etc. The clocking scheme for the transceiver (e.g., PHY I/O) may be source-synchronous (e.g., for higher bandwidth performance per wire) or common-clock (e.g., for lower bandwidth targets).

[0082] FIG. 4 illustrates a transmitter circuit 402 of a first die coupled to a receiver circuit 404 of a second die through an interconnect 406 according to embodiments of the disclosure. FIG. 4 shows a high-level (e.g., source-synchronous clocking) circuit diagram for a transceiver (e.g., PHY I/O) connecting two dies together, e.g., for a data transfer therebetween. Transmitter circuit 402 includes a plurality of transmitters (412A, 412B, 412C, 412D) that produce (e.g., amplify) signals. Receiver circuit 404 includes a plurality of receivers (414A, 414B, 414C, 414D) (e.g., samplers) that receives transmitted signals. Interconnect 406 includes a plurality of lanes (416A, 416B, 416C, 416D). An interconnect may have any one or more of these lanes in certain embodiments. An interconnect may include a plurality of each of these lanes in certain embodiments. In one embodiment, each of these lanes is a discrete wire of the interconnect. Although a single data lane 416 is depicted, a plurality of data lanes (e.g., including one or more respective instances of one or more of the components of the transceiver circuit 402 and/or the receiver circuit 404) may be utilized, e.g., with a single clock lane associated with those multiple data lanes.

[0083] In certain embodiments, transceiver circuit 402, interconnect 406, and/or receiver circuit 404 (e.g., any one of those or any combination thereof) include a circuit (e.g., clock circuit) to change operating frequency and/or a clock rate for that operating frequency. In certain embodiments, a clock phase placement (e.g., as discussed herein) is determined (e.g., predetermined) for the operating frequency or frequencies and/or the clocking rate or rates for those operating frequency or frequencies. As an example, data to be transmitted from a first die to a second die may be received by transmitter circuit 402 of the first die and then sent to a second die via receiver circuit 404 through interconnect 406. The first die may be operating at an operating frequency and the second die may be operating at an (e.g., the same) operating frequency, but a clock circuit (e.g., clock circuit 408) may adjust the clock phase placement for the operating frequency (e.g., and a clocking rate for the operating frequency) from a plurality of clock phase placements (e.g., for the same clock cycle). For example, the clock phase placement for the operating frequency may be selected such that no or a minimal amount of data is lost during transmittal. In one embodiment, an intra-die interconnect operates at multiple clocking rate relative to an operating frequency of a different (e.g., inter-die) interconnect of a die or dies coupled to the intra-die interconnect.

[0084] As one example, transmitter circuit 402 may receive data from a data generator 421 of a first die that is to be transmitted to receiver circuit 404 (e.g., second die including receiver circuit 404). Data generator 421 of first die may be a processor (e.g., a processor including a decoder to decode an instruction to decode instruction in an execution unit to execute the decoded instruction to generate the data) of the first die. Data to be transmitted may include first data (e.g., data stream) (e.g., data D0) and (e.g., separate) second data (e.g., data stream) (e.g., data D1).

[0085] A clock signal (e.g., from or based on the clock signal in first die) from the transmitter circuit 402 (e.g., transmitter side) may be sent (e.g., forwarded) along with (e.g., concurrently with) the data (e.g., payload data) being sent to the receiver circuit 404. Clock circuit 420 may be the internal (e.g., main) clock of the first die (e.g., of the mesh in the first die). Clock circuit 410 may be a separate clock generator, e.g., separate from the internal (e.g., main) clock of the first die, and/or a dedicated clock circuit of the transmitter circuit 402. A multiplexer may select and output one of multiple inputs according to a control signal. Multiplexer (mux) 428 may be set to provide a clock signal from clock circuit 410 or clock circuit 420, e.g., based on a control signal. Multiplexer 428 may be controlled by power management circuit 432, e.g., based on a control signal received from power management circuit (e.g., a power management controller). A power management circuit may control the switching of an operating frequency and/or a clocking rate, for example, the operating frequency and/or a clocking rate in a first die and/or in a second die (e.g., connected via an interconnect to the first die). A local and/or dedicated clock circuit (e.g., clock circuit 410) (e.g., in an I/O PHY) (e.g., phase-locked loop (PLL) circuit) may be employed to enable higher I/O bandwidths by filtering the (e.g., mesh) barrier clock jitter components.

[0086] In the depicted embodiment, multiplexer 428 outputs a received clock signal (e.g., the square waveform clock signal in FIGS. 5 and 6) as a control signal to multiplexer 424. Multiplexer 424 may also take a second input from valid signal circuit 418, e.g., such that multiplexer 424 provides no output when the valid signal circuit 418 indicates invalid (e.g., a logical zero). Multiplexer 424 may then output data (e.g., payload data) from its output to data lane 416B, e.g., via transmitter 412B.

[0087] Multiplexer 430 may be included such that the clock signal output from multiplexer 428 passes through both multiplexer 424 and multiplexer 430, e.g., to replicate the delay through multiplexer 424. Multiplexer 430 may have a first input that is ground and a second input that is a power source. In the depicted embodiment, multiplexer 430 outputs its signal to clock lane 416C (e.g., via transmitter 412C) and clock inverse lane 416D (e.g., via transmitter 412D).

[0088] Although two data sources (e.g., D0 and D1) (for example, two wires or two signals, e.g., that are to cross a die boundary to another die) are depicted in certain figures herein as sharing a single data lane, it is understood that a single data source (e.g., wire or signal) may utilize a single data lane, e.g., data lane 412.

[0089] One or more components of circuit 400 may be switchable from a first clocking rate to a second, different clocking rate, e.g., for each different operating frequency.

[0090] By enabling a (e.g., data) valid signal (for example, active only when data is on the connection (for example, a data link, e.g., the one or more lanes of the link) is active (e.g., is to be utilized for data transfer), clock gating may be employed to save power. A valid signal controller 418 may generate a valid signal, e.g., when a first die is to transmit data to a second die. A data signal (e.g., data payload) is separate from a control signal in certain embodiments. Valid signal circuit 418 (e.g., valid signal controller) may be a part of a power management circuit (e.g., power management controller). Power management circuit may be a component of a die. Each die may have its own power management

controller. Valid signal circuit **418** may assert a valid signal or invalid signal, e.g. to start or stop (respectively) the receipt and/or passage of data from a first die (e.g., from transmitter circuit **402**) to a second die (e.g., to receiver circuit **404**) and/or out of second die (e.g., out of receiver circuit **404**), e.g., by turning off receivers **414B** and/or **414C**. Retimer circuit **425** may retime the data valid signal (e.g., out of receiver **414A**) based on the clock phase placement.

[0091] Receiver circuit **404** may receive a valid signal on the valid lane **416A** of interconnect **406**, a data signal on data lane **416B** of interconnect **406**, and/or a clock signal (or inverse signal, or combination of those as a strobe signal) on clock lane **416C** and/or clock lane **416D** of interconnect **406**. Retimer circuit **425** may retime the valid signal such that it is synchronized with the data and/or clock signal(s) that it was sent with. For example, a valid data signal may be sent for one or more streams of data and that signal may be output to AND gate **422**. AND gate **422** may receive a clock signal from clock circuit **408** of receiver circuit **404**, e.g., such that the output of AND gate **422** is used to turn on one of the plurality of receivers **414B** and **414C** (e.g., where a NOT gate (an inverter) is included before the control signal input into receiver **414B**). As shown in FIG. 5, this allows the serial transmittal of data from source **D0**, then source **D1**, then source **D0** again, and repeating that so that the data signals alternate between **D0** and **D1** (e.g., subject to whatever data signal is being output, e.g., logical high (e.g., a one) or logical low (e.g., a zero)). Multiplexer **426** may thus alternate between outputting data from receiver **414B** and from receiver **414C**. Control signal (e.g., output of AND gate **422**) is used to switch multiplexer **426** inputs between sourcing an output from receiver **414B** and from receiver **414C**.

[0092] Depicted clock circuit **408** receives an input clock signal or signals from the transmitter circuit **402** and is to align one or more of the clock edges and the received data signals (e.g., payload data on data lane **416B**, which may be more than one data lane) such that the received data is correctly received (e.g., such that the data sent from transmitter circuit **402** matches the data received at receiver circuit **404**). In one embodiment, the clock circuit **408** is to shift the phase (and not the frequency) of the received clock signal to align it as desired with the received data signal (e.g., payload data on data lane **416B**).

[0093] In one embodiment, clock circuit **408** of receiver circuit **404** includes circuitry to align (e.g., shift) the (e.g., source-synchronous) clock edges of a received clock signal (e.g., waveform) from the transmitter circuit **402** with the corresponding received data signal (e.g., different than a clock signal) for high-performance timing, e.g., such that the data in the data signal is not altered, lost, destroyed, or any combination thereof. Clock circuit **408** may include a clock phase delay generator **408A** (e.g., DLL circuit) and/or phase interpolator circuit **408B**. In one embodiment, clock phase placement is performed by a phase interpolator e.g. phase interpolator circuit **408B**. In one embodiment, a phase interpolator is a circuit that adjusts (e.g., shifts) the phase of a clock signal. In one embodiment, a phase interpolator has a level (e.g., 2, 4, 6, 8, 10, 12, etc.) of granularity of steps per each clock phase e.g., that are equally spaced apart and it may set a rising clock edge and/or falling clock edge at any of those steps, for example, as discussed further in reference to FIG. 13 below.

[0094] Clock circuit **408**, e.g., including a delay-locked loop (DLL) circuit, may be employed at the receiver circuit **404** of the receiver die to appropriately align the source-synchronous clocking edge for high-performance timing (e.g., to enable effective high-speed signaling). A DLL circuit may be a negative-delay gate placed in the clock path of a digital circuit. In one embodiment, clock circuit **408** is a component of receiver circuit **404**. A local and/or dedicated clock circuit (e.g., clock circuit **410**) (e.g., in an I/O PHY) (e.g., phase-locked loop (PLL) circuit) may be employed to enable higher I/O bandwidths by filtering the (e.g., mesh) barrier clock jitter components. PLL circuit may be a control circuit that generates an output signal whose phase is related to the phase of an input signal. Although there are different types of PLL circuits, one example is a circuit with a variable frequency oscillator and a phase detector in a feedback loop, e.g., where the oscillator generates a periodic signal, the phase detector compares the phase of that signal with the phase of the input periodic signal, and adjusts the oscillator to keep the phases matched. A PLL may be an all digital PLL (ADPLL). In one embodiment, a DLL circuit uses a variable phase (e.g., delay) block and a PLL circuit uses a variable frequency block. Clock circuit **408** may include a control register **409**, for example, to store the clock phase placement settings, e.g., to cause clock circuit **408** to apply those settings.

[0095] To maintain high power efficiency for the transmitter circuit and/or receiver circuit (e.g., I/O PHY), techniques such as low swing signaling, clock-gating, and aggregating the source-synchronous clocking power between a plurality (e.g., a large number) of serviced data lanes may be employed. For example, one forwarded source-synchronous clock may be utilized for each of 2, 3, 4, 5, 6, 7, 8, 16, 32, 64, 128, 256, etc. data lanes or any subset thereof. Data lane **416B** is merely an example and a plurality of lanes may be utilized.

[0096] In certain embodiments, clock phase delay generator **408A** (e.g., DLL circuit) generates lock (e.g., not clock) timing (e.g., as in FIG. 16) for a clock rate of an operating frequency (for example, clock phase locking of 90 degrees or 180 degrees, e.g., as in FIGS. 6 and 7, respectively). In certain embodiments, phase interpolator circuit **408B** subdivides those clock signals into a finer granularity. In certain embodiments, clock circuit **408** utilizes predetermined (e.g., before the current data transmittal) clock phase placement data, e.g., both clock phase delay generator **408A** (e.g., DLL circuit) and/or phase interpolator circuit **408B** utilize predetermined clock phase placement data. In one embodiment, clock phase delay generator **408A** is a clock phase controller or clock phase adjuster. In one embodiment, clock phase delay generator **408A** maintains a certain phase relationship of the clock arriving at the receivers (e.g., samplers) (e.g., of a second die) with respect to the input clock or clocks coming in from the transmitter (e.g., of a first die). In certain embodiments, the clock phase delay generator **408A** generates the clock phase delay and the phase interpolator circuit **408B** is to further subdivide those clock signals into the finer granularity. In one embodiment, clock phase delay generator **408A** looks up and utilizes a lock code for a particular clocking rate and/or operating frequency, and/or phase interpolator circuit **408B** looks up and utilizes the buffer settings for the phase interpolator for the particular clocking rate and/or operating frequency. For example, a lock code (e.g., of a DLL) may change for each frequency and/or each

process, voltage, and/or temperature point (e.g., of a plurality of points) and a phase interpolator circuit may perform the (e.g., finer granularity) clock (e.g., edge) placement within that (e.g., DLL) lock code. Once the (e.g., predetermined) clock phase placement for the operating frequency and clocking rate are looked-up and updated into the circuitry (e.g., clock circuit **408**), data may be received by receiver circuit, for example, output to data buffers **434** (e.g., as in FIG. **21**).

[0097] FIG. **5** illustrates a data timing diagram **501** and a clock timing diagram **502** for a first clocking rate according to embodiments of the disclosure. In the depicted embodiment, clock timing diagram **501** illustrates a 180 degree offset of the clock signal (e.g., clock₁₈₀ in FIG. **16**) used to clock in data relative to the clock signal received at the receiver for a 1× clocking rate. Data timing diagram **501** illustrates that the data (e.g., alternating D0 and D1 data transmitted with the circuit **400** of FIG. **4**) in the 1× clocking rate may be read in at each falling edge of the clock. As discussed herein, predetermined clock phase placement (e.g., relative to the data timing) may be utilized to place the clock edges.

[0098] FIG. **6** illustrates a data timing diagram **601** and a clock timing diagram **602** for a second clocking rate according to embodiments of the disclosure. In the depicted embodiment, clock timing diagram **601** illustrates a 90 degree offset of the clock signal (e.g., clock₉₀ in FIG. **16**) used to clock in data relative to the clock signal received at the receiver for a 2× clocking rate. Data timing diagram **601** illustrates that the data (e.g., alternating D0 and D1 data transmitted with the circuit **400** of FIG. **4**) in the 2× clocking rate may be read in at each of the rising and falling edge of the clock. As discussed herein, predetermined clock phase placement (e.g., relative to the data timing) may be utilized to place the clock edges.

[0099] FIG. **7** illustrates a transmitter circuit **702** of a first die coupled to a receiver circuit **704** of a second die through an interconnect **706** according to embodiments of the disclosure. FIG. **7** shows a high-level (e.g., source-synchronous clocking) circuit diagram for a transceiver (e.g., PHY I/O) connecting two dies together, e.g., for a data transfer therebetween. Transmitter circuit **702** includes a plurality of transmitters (**712A**, **712B**, **712C**, **712D**) that produce (e.g., amplify) signals. Receiver circuit **704** includes a plurality of receivers (**714A**, **714B**, **714C**, **714D**, **714E**, **714F**) that receives transmitted signals. Interconnect **706** includes a plurality of lanes (**716A**, **716B**, **716C**, **716D**). An interconnect may have any one or more of these lanes in certain embodiments. An interconnect may include a plurality of each of these lanes in certain embodiments. In one embodiment, each of these lanes is a discrete wire of the interconnect. Although two data lanes (i.e., data lanes **716B** and **716D**) are depicted, a single data or three or more data lanes (e.g., including one or more respective instances of one or more of the components of the transceiver circuit **702** and/or the receiver circuit **704**) may be utilized, e.g., with a single clock lane associated with those multiple data lanes. For example, a single data source (e.g., DO) may be utilized, e.g., by removing the control signal line from clock circuit **710** to multiplexer **724** (and/or removing multiplexer **724** and/or outputting data from data lane **716B** directly to a single receiver (e.g., receiver **714E**) without using multiplexer **726**.

[0100] In certain embodiments, transceiver circuit **702**, interconnect **706**, and/or receiver circuit **704** (e.g., any one of those or any combination thereof) include a circuit (e.g., clock circuit) to change in operating frequency and/or a clock rate for that operating frequency. In certain embodiments, a clock phase placement (e.g., as discussed herein) is determined (e.g., predetermined) for the operating frequency or frequencies and/or the clocking rate for those operating frequency or frequencies. As an example, data (e.g., payload data) to be transmitted from a first die to a second die may be received by transmitter circuit **702** and then sent to a second die via receiver circuit **704** through interconnect **706**. The first die may be operating at an operating frequency and the second die may be operating (e.g., switched to) at an (e.g., the same) operating frequency, but a clock circuit (e.g., clock circuit **708**) may adjust the clock phase placement for the operating frequency (e.g., and a clocking rate for the operating frequency) from a plurality of clock phase placements (e.g., for the same clock cycle). For example, the clock phase placement for the operating frequency may be selected such that no or a minimal amount of data is lost during transmittal.

[0101] As one example, transmitter circuit **702** may receive data from data generator **720** and/or data generator **730** (e.g., which may be combined into a single data generator) of a first die that is to be transmitted to receiver circuit **704** (e.g., second die including receiver circuit **704**). Data generator **720** and/or data generator **730** of first die may be a processor or processors (e.g., each processor including a decoder to decode an instruction to decode instruction in an execution unit to execute the decoded instruction to generate the data) of the first die. Data to be transmitted may include any of first data (e.g., data stream) (e.g., data D0), (e.g., separate) second data (e.g., data stream) (e.g., data D1), (e.g., separate) third data (e.g., data stream) (e.g., data D2), (e.g., separate) fourth data (e.g., data stream) (e.g., data D3), or any combination thereof.

[0102] A clock signal (e.g., from or based on the clock signal in first die) from the transmitter circuit **702** (e.g., transmitter side) may be sent (e.g., forwarded) along with (e.g., concurrently with) the data (e.g., payload data) being sent to the receiver circuit **704**. Clock circuit **710** may be the internal (e.g., main) clock of the first die (e.g., of the mesh in the first die), a separate clock generator, e.g., separate from the internal (e.g., main) clock of the first die, and/or a dedicated clock circuit of the transmitter circuit **702**.

[0103] As a component of or separate from interconnect **706**, circuit **700** (or other circuits herein) may include a control lane to send a control signal from a first die (e.g., via transmitter circuit **702**) to second die (e.g., via receiver circuit **704**). Control signal may be sent by power management circuit **740** (e.g., a power management controller), e.g., sent to receiver circuit **704** (e.g., clock circuit **708** of receiver circuit **704** and/or second die). Control signal may switch a circuit (e.g., a clock circuit) between a closed-loop mode and an open-loop mode. Power management circuit may control the switching of an operating frequency and/or a clocking rate, for example, the operating frequency and/or a clocking rate in a first die and/or in a second die (e.g., connected via an interconnect to the first die). A local and/or dedicated clock circuit (e.g., clock circuit **710**) (e.g., in an I/O PHY) (e.g., phase-locked loop (PLL) circuit) may be employed to enable higher I/O bandwidths by filtering the (e.g., mesh) barrier clock jitter components. In one embodiment, a first

die is to request a second die (e.g., both dies) to operate at a different frequency and/or clocking rate based on usage, for example, operating at a (e.g., single) frequency and increasing the clocking rate when data is backing up (e.g., in a buffer in the first die) and/or at a (e.g., single) frequency and decreasing the clocking rate when data is not backing up (e.g., an empty or not filled buffer in the first die).

[0104] In the depicted embodiment, clock circuit 710 outputs a clock signal (e.g., the square waveform clock signal in FIGS. 8 and 9) as a control signal to multiplexer 724 and/or multiplexer 734. Multiplexer 724 may then output data (e.g., payload data) from its output to data lane 716B, e.g., via transmitter 712B and/or multiplexer 734 may then output data (e.g., payload data) from its output to data lane 716D, e.g., via transmitter 712D. Clock signal may be transmitted from transmitter circuit 702 to transmitter 712C, through clock (e.g., strobe) lane 716C (e.g., of interconnect 706) to receiver 714C of receiver circuit 704, e.g., and then to clock circuit 708.

[0105] Although two pairs of data sources (e.g., D0/D1 and D2/D3) (for example, four wires or four signals, e.g., that are to cross a die boundary to another die) are depicted in certain figures herein as sharing a single data lane, it is understood that a single data source (e.g., wire or signal) may utilize a single data lane, e.g., data lane 716B or data lane 716D.

[0106] One or more components of circuit 700 may be switchable from a first clocking rate to a second, different clocking rate, e.g., for each different operating frequency.

[0107] By enabling a (e.g., data) control signal (for example, active only when data is on the connection (for example, a data link, e.g., the one or more lanes of the link) is active (e.g., is to be utilized for data transfer), clock gating may be employed to save power. A power management circuit 740 (e.g., power management controller) may generate a valid data and/or frequency change and/or clocking rate change signal, e.g., when a first die is to transmit data to a second die. A data signal (e.g., data payload) is separate from a control signal in certain embodiments. Power management circuit may be a component of a die. Each die may have its own power management controller. Power management circuit may assert a valid signal or invalid signal, e.g., to start or stop (respectively) the receipt and/or passage of data from a first die (e.g., from transmitter circuit 702) to a second die (e.g., to receiver circuit 704) and/or out of second die (e.g., out of receiver circuit 704), e.g., by turning off transmitter(s) and/or receiver(s).

[0108] Receiver circuit 704 may receive a control signal (e.g., to change the frequency and/or clocking rate) on the control lane 716A of interconnect 706, a data signal on data lane 716B of interconnect 706, a data signal on data lane 716D of interconnect 706, and/or a clock signal (or inverse signal, or combination of those as a strobe signal) on clock lane 716C of interconnect 706. For example, power management circuit 740 may send a signal to receiver circuit 704 (e.g., clock circuit 708 thereof) to enable a certain frequency and/or clocking rate for the receiver circuit 704 (e.g., clock circuit 708 thereof), e.g., the same frequency and/or clocking rate of the transmitter circuit 702.

[0109] Receiver 722 may receive a clock signal from clock circuit 708 of receiver circuit 704, e.g., such that the output of receiver 722 is used to turn on one of the plurality of receivers 714B and 714E (e.g., where a NOT gate (an inverter) is included before the control signal input into

receiver 714B) (e.g., and turn off the other receiver of the pair) and/or turn on one of the plurality of receivers 714D and 714F (e.g., where a NOT gate (an inverter) is included before the control signal input into receiver 714D) (e.g., and turn off the other receiver of the pair). As shown in FIG. 8, this allows the serial transmittal of data from source D0, then source D1, then source D0 again, and repeating that so that the data signals alternate between D0 and D1 (e.g., subject to whatever data signal is being output, e.g., logical high (e.g., a one) or logical low (e.g., a zero)) and/or (e.g., in parallel with the serial sending of D0 and D1) the serial transmittal of data from source D2, then source D3, then source D2 again, and repeating that so that the data signals alternate between D2 and D3 (e.g., subject to whatever data signal is being output, e.g., logical high (e.g., a one) or logical low (e.g., a zero)). Multiplexer 726 may thus alternate between outputting data from receiver 714B and from receiver 714E. Control signal (e.g., output of receiver 722) (e.g., the received source synchronous clock after it has gone through the DLL/PI/clock distribution circuitry) is used to switch multiplexer 726 inputs between sourcing an output from receiver 714B and from receiver 714E. Multiplexer 728 may thus alternate between outputting data from receiver 714D and from receiver 714F. Control signal (e.g., output of receiver 722) (e.g., the received source synchronous clock after it has gone through the DLL/PI/clock distribution circuitry) is used to switch multiplexer 728 inputs between sourcing an output from receiver 714D and from receiver 714F.

[0110] Depicted clock circuit 708 receives an input clock signal or signals from the transmitter circuit 702 and is to align one or more of the clock edges and the received data signals (e.g., payload data on data lane 716B and/or data lane 716D, and which may be more than two data lanes) such that the received data is correctly received (e.g., such that the data sent from transmitter circuit 702 matches the data received at receiver circuit 704. In one embodiment, the clock circuit 708 is to shift the phase (and not the frequency) of the received clock signal to align it as desired with the received data signal (e.g., payload data on data lane 716B and/or data lane 716D)).

[0111] In one embodiment, clock circuit 708 of receiver circuit 704 includes circuitry to align (e.g., shift) the (e.g., source-synchronous) clock edges of a received clock signal (e.g., waveform) from the transmitter circuit 702 with the corresponding received data signal (e.g., different than a clock signal) for high-performance timing, e.g., such that the data in the data signal is not altered, lost, destroyed, or any combination thereof. Clock circuit 708 may include a clock phase delay generator 708A (e.g., DLL circuit) and/or phase interpolator circuit 708B. In one embodiment, clock phase placement is performed by a phase interpolator e.g. phase interpolator circuit 708B. In one embodiment, a phase interpolator is a circuit that adjusts (e.g., shifts) the phase of a clock signal. In one embodiment, a phase interpolator has a level (e.g., 2, 4, 6, 8, 10, 12, etc.) of granularity of steps per each clock phase e.g., that are equally spaced apart and it may set a rising clock edge and/or falling clock edge at any of those steps, for example, as discussed further in reference to FIG. 13 below.

[0112] Clock circuit 708, e.g., including a delay-locked loop (DLL) circuit, may be employed at the receiver circuit 704 of the receiver die to appropriately align the source-synchronous clocking edge for high-performance timing

(e.g., to enable effective high-speed signaling). A DLL circuit may be a negative-delay gate placed in the clock path of a digital circuit. In one embodiment, clock circuit 708 is a component of receiver circuit 704. A local and/or dedicated clock circuit (e.g., clock circuit 710) (e.g., in an I/O PHY) (e.g., phase-locked loop (PLL) circuit) may be employed to enable higher I/O bandwidths by filtering the (e.g., mesh) barrier clock jitter components. PLL circuit may be a control circuit that generates an output signal whose phase is related to the phase of an input signal. Although there are different types of PLL circuits, one example is a circuit with a variable frequency oscillator and a phase detector in a feedback loop, e.g., where the oscillator generates a periodic signal, the phase detector compares the phase of that signal with the phase of the input periodic signal, and adjusts the oscillator to keep the phases matched. A PLL may be an all digital PLL (ADPLL). In one embodiment, a DLL circuit uses a variable phase (e.g., delay) block and a PLL circuit uses a variable frequency block. Clock circuit 708 may include a control register 709, for example, to store the clock phase placement settings, e.g., to cause clock circuit 708 to apply those settings.

[0113] To maintain high power efficiency for the transmitter circuit and/or receiver circuit (e.g., I/O PHY), techniques such as low swing signaling, clock-gating, and aggregating the source-synchronous clocking power between a plurality (e.g., a large number) of serviced data lanes may be employed. For example, one forwarded source-synchronous clock may be utilized for each of 2, 3, 4, 5, 6, 7, 8, 16, 32, 64, 128, 256, etc. data lanes or any subset thereof. Data lane 716B is merely an example and a plurality of lanes may be utilized.

[0114] In certain embodiments, clock phase delay generator 708A (e.g., DLL circuit) generates lock (e.g., not clock) timing (e.g., as in FIG. 16) for a clock rate of an operating frequency (for example, clock phase locking of 90 degrees or 180 degrees, e.g., as in FIGS. 8 and 9, respectively). In certain embodiments, phase interpolator circuit 708B subdivides those clock signals into a finer granularity. In certain embodiments, clock circuit 708 utilizes predetermined (e.g., before the current data transmittal) clock phase placement data, e.g., both clock phase delay generator 708A (e.g., DLL circuit) and/or phase interpolator circuit 708B utilize predetermined clock phase placement data. In one embodiment, clock phase delay generator 708A is a clock phase controller or clock phase adjuster. In one embodiment, clock phase delay generator 708A maintains a certain phase relationship of the clock arriving at the receivers (e.g., samplers) (e.g., of a second die) with respect to the input clock or clocks coming in from the transmitter (e.g., of a first die). In certain embodiments, the clock phase delay generator 708A generates the clock phase delay and the phase interpolator circuit 708B is to further subdivide those clock signals into the finer granularity. In one embodiment, clock phase delay generator 708A looks up and utilizes a lock code for a particular clocking rate and/or operating frequency, and/or phase interpolator circuit 708B looks up and utilizes the buffer settings for the phase interpolator for the particular clocking rate and/or operating frequency. For example, a lock code (e.g., of a DLL) may change for each frequency and/or each process, voltage, and/or temperature point (e.g., of a plurality of points) and a phase interpolator circuit may perform the (e.g., finer granularity) clock (e.g., edge) placement within that (e.g., DLL) lock code. Once the (e.g., predeter-

mined) clock phase placement for the operating frequency and clocking rate are looked-up and updated into the circuitry (e.g., clock circuit 708), data may be received by receiver circuit, for example, output to data buffers 735 and/or data buffers 736 (e.g., as in FIG. 21). In one embodiment, a first die includes one or more transmitter circuits (e.g., transmitter circuit 402 of FIG. 4 or transmitter circuit 702 of FIG. 7) and a second die includes one or more receiver circuits (e.g., receiver circuit 404 of FIG. 4 or receiver circuit 704 of FIG. 7). Additionally or alternatively, that second die may include one or more transmitter circuits (e.g., transmitter circuit 402 of FIG. 4 or transmitter circuit 702 of FIG. 7) and that first die may include one or more receiver circuits (e.g., receiver circuit 404 of FIG. 4 or receiver circuit 704 of FIG. 7), e.g., to allow two-way communication between the dies.

[0115] FIG. 8 illustrates a data timing diagram 801 and a clock timing diagram 802 for a first clocking rate according to embodiments of the disclosure. In the depicted embodiment, clock timing diagram 801 illustrates a 180 degree offset of the clock signal (e.g., clock_180 in FIG. 16) used to clock in data relative to the clock signal received at the receiver for a 1× clocking rate. Data timing diagram 801 illustrates that the data (e.g., alternating D0 and D1 data and/or alternating D2 and D3 data transmitted with the circuit 700 of FIG. 7) in the 1× clocking rate may be read in at each falling edge of the clock. As discussed herein, predetermined clock phase placement (e.g., relative to the data timing) may be utilized to place the clock edges.

[0116] FIG. 9 illustrates a data timing diagram 901 and a clock timing diagram 902 for a second clocking rate according to embodiments of the disclosure. In the depicted embodiment, clock timing diagram 901 illustrates a 90 degree offset of the clock signal (e.g., clock_90 in FIG. 16) used to clock in data relative to the clock signal received at the receiver for a 2× clocking rate. Data timing diagram 901 illustrates that the data (e.g., alternating D0 and D1 data and/or alternating D2 and D3 data transmitted with the circuit 700 of FIG. 7) in the 2× clocking rate may be read in at each of the rising and falling edge of the clock. As discussed herein, predetermined clock phase placement (e.g., relative to the data timing) may be utilized to place the clock edges.

[0117] In one embodiment, an I/O PHY circuit (e.g., the transmitter circuit of one die and receiver circuit of another die or dies) is capable of (e.g., quickly) changing between different clocking rates (e.g., data rates) (e.g., 1×, 2×, 4×, etc.) and/or clock frequency rate changes, e.g., to support an interconnect employed in the mesh of a die. In certain embodiments, the clock circuit or circuits (e.g., Delay Locked Loop (DLL) and Phase Interpolator (PI)) used for (e.g., receiver) clocking edge alignment are calibrated for a plurality of (e.g., all) possible clocking rates (e.g., data rates) and/or frequencies, e.g., at initial boot time. In an embodiment where a digital-control DLL+PI is employed, the calibration information for each of the clocking rates (e.g., data rates) and operating frequency configurations is stored (for example, in a memory array, e.g., in clock circuit) and recalled when a circuit (e.g., a die) initiates a clocking rates (e.g., data rates) and/or frequency change (e.g., of the interconnect connecting two or more dies). This may also be accomplished for analog-controlled DLL+PI circuits, for example, by converting analog bias points to digital information using analog to digital (A/D) converters for storage

in a memory array and then a digital to analog (D/A) converter to convert back to analog bias points when updating operating points. These recalled clock (e.g., DLL+PI) calibration settings may be used to override the current clock (e.g., DLL+PI) calibration settings to allow for quick clock (e.g., DLL+PI) lock and/or calibration to the new settings and/or operating point. Certain embodiments herein thus allow rapid transitions between different clocking rates (e.g., data rates) and/or frequencies.

[0118] Certain embodiments herein provide for novel circuitry and algorithm to allow fast and dynamic I/O clocking rates (e.g., data rates) and/or frequency changes on the fly. In one embodiment, I/O timing (e.g., clocking rate and/or operating frequency) between dies is facilitated by tuned clock phases (e.g., by a combination of DLL auto-tracking circuitry and training PI sweeps). In one embodiment, the training occurs all at one time (e.g., one training session) (e.g., at manufacturing time, before end users utilize the processor). The I/O clocking architecture may be source-synchronous, e.g., forwarded clock which is tuned to a specific phase relationship with respect to the data lane or lanes of to maximize I/O timing margin. FIG. 4 and FIG. 7 illustrate examples of the high-level clocking architecture. FIGS. 5, 6, 8, and 9 illustrate example timing diagrams depicting 1× (single clocking rate) and 2× (double clocking rate) clocking relationships with respect to data eyes (e.g., data eyes D0 and D1 in the upper portions of each of FIGS. 5, 6, 8, and 9). In certain embodiments, fine-grain control of clock strobe placement allows for maximum performance. Certain embodiments achieve this by a combination of DLL+PI for small phase step granularity (e.g., 1 or about 1 picosecond (ps) increments). FIG. 13 (discussed further below) shows example circuit architecture specifics of the digital delay line within a DLL as well as a digital-style PI. The output of that DLL+PI may be either one clock (e.g., use both clock edges to time), or two outputs (e.g., use one clock edge of each to time) or four outputs (e.g., in the case of 4× clocking rate) (e.g., use one clock edge of each clock or alternatively, send out 2 clocks and use both clock edges of each clock to time all 4 data bits per cycle). Note that FIGS. 5, 6, 8, and 9 show a single clock output (e.g., use one clock edge for 1× clocking rate or both edges to time for 2× clocking rate), but FIG. 13 shows two outputs to show that this circuit and method may also be used for 2× clocking, e.g., by using only one clock edge per clock cycle for timing. In certain embodiments, the tuned clock phase will be unique for each frequency point and clocking rate at that frequency point (e.g., as well as unique per instantiation of hardware within a die and/or as well as die to die).

[0119] FIG. 10 illustrates a flow diagram 1000 for interconnect (e.g., I/O) programming according to embodiments of the disclosure. Flow diagram may be included in circuitry (e.g., finite state machine (FSM)) within a die (e.g., within a transmitter circuit and/or receiver circuit). FIG. 11 illustrates clock phase placement 1100 according to embodiments of the disclosure. Referring to both FIGS. 10 and 11, a clock circuit (e.g., of a first die) (e.g., clock circuit 410 or clock circuit 420 in FIG. 4 or clock circuit 710 in FIG. 7) (e.g., PLL of a mesh of a die) (e.g., of a transmitter circuit) may be set to a (e.g., new) desired operating frequency 1002 (e.g., 400, 500, 600, 700, 800, 900 megahertz (MHz), 1, 2, 3, 4, 5 gigahertz (GHz), etc.). A clock circuit (e.g., of a second die) (e.g., of a receiver circuit) may be locked to that desired (e.g., mesh) operating frequency 1004 in flow dia-

gram 1000. Clock circuit may then sweep its settings (e.g., DLL and PI settings) 1006 in flow diagram 1000 to find the clock phase placement (e.g., values) (e.g., codes) (e.g., L1 and R1 codes) (e.g., as discussed in reference to FIG. 11), e.g., that allows the data to be transmitted (e.g., a “pass” and not a “fail”). In one embodiment, a plurality of (e.g., each of) the clock phase placements (e.g., the clock edge placement for a same frequency) are swept (e.g., enabled and tested) to find the fail-to-pass and pass-to-fail codes, e.g., to determine the clock phase placement (e.g., DLL+PI) settings. For each clock phase placement (e.g., DLL+PI phase) setting, data along with a clock signal (e.g., whose phase is determined by the DLL+PI code setting) may be transmitted from a first die and received by a second die through an I/O link (e.g., interconnect). Some clock phase placements may be too early with respect to the data to be captured correctly by the second die (e.g., “fail”) and some clock phase placements may allow the data to be captured correctly by the second die (e.g., “pass”). In one embodiment, a plurality (e.g., all) of the clock phase placements (e.g., settings to achieve those placements) that pass and a plurality (e.g., all) of the clock phase placements that fail are found, e.g., so as to determine the optimal setting for the best (e.g., maximum timing margin) reliable timing.

[0120] FIG. 11 demonstrates an example of these phase relationships. For example, each signal to be transmitted (e.g., D0, D1, D2, or D3) may be turned off and on (e.g., from high to low and then low to high) multiple times to generate the eye diagram 1102 in FIG. 11. A fail-to-pass code (e.g., corresponding to the settings of value “3”) is the left edge of the eye opening of the eye diagram 1102 that corresponds to a specific clock (e.g., DLL+PI) phase placement (for example, the (e.g., receiver) clock circuit settings to achieve that placement), e.g., “passing” leading edge placement indicated by L1 in eye diagram 1102. A pass-to-fail code (e.g., corresponding to the settings of value “7”) is the right edge of the eye opening of the eye diagram 1102 that corresponds to a specific clock (e.g., DLL+PI) phase placement (for example, the (e.g., receiver) clock circuit settings to achieve that placement), e.g., indicated by “passing” trailing edge placement R1 in eye diagram 1102. These codes may be the settings (e.g., for DLL and/or phase interpolator circuits) to achieve that placement, e.g., the codes may be an index into a storage array storing the circuit setting values that achieve that placement. In one embodiment, clock phase placement data (e.g., left (L1) and right (R1) passing clock phase (e.g., edge) placement data may be used to determine an optimal clock-phase placement code (OCP). The optimal clock phase placement (e.g., OCP1 for L1 and R1) may be the clock phase placement (e.g., DLL+PI settings) corresponding to the middle phase between L1 and R, e.g., $OCP1 = L1 + (R1 - L1)/2$. In FIG. 11, this corresponds to the settings of value “5” for OCP1. The clock phase placement (e.g., DLL+PI settings) (e.g., clock phase placement code or codes) may be stored in memory. For example, FIG. 12 illustrates a table 1200 including clock phase placements (e.g., placement data) according to embodiments of the disclosure. Table 1200 is merely an example of a data structure format and other formats are possible. A table may include one more entries for a first die (e.g., die D1) transmitting data to a second die (e.g., die D2) via a coupling (e.g., interconnect) and/or a second die (e.g., die D2) transmitting data to a first die (e.g., die D1) via a coupling (e.g., interconnect). Row 1201 of table 1200 includes data (e.g.,

predetermined clock phase placement) for a plurality of clocking rates of data sent from die D_1 (e.g., transmitter circuit thereof) to die D_2 (e.g., receiver circuit thereof) at a single frequency (e.g., a first frequency (f_1)). Row **1203** of table **1200** includes data (e.g., predetermined clock phase placement) for a plurality of clocking rates of data sent from die D_2 (e.g., transmitter circuit thereof) to die D_1 (e.g., receiver circuit thereof) at a single frequency (e.g., a first frequency (f_1)). As discussed herein, table **1200** may be populated with this data beforehand, e.g., before run-time of the processor and/or before the data to be transmitted is generated. Using the above example from FIG. **11**, entry **1202** of row **1201** of table **1200** may include predetermined clock phase placement data (e.g., codes for left clock edge placement, the right clock edge placement, and/or the center of the optimal clock phase placement) for a plurality of clocking rates of data sent between D_1 to D_2 at a single frequency (e.g., a first frequency (f_1)). In this example, entry **1202** include a value of “3” for the left clock edge (L1), a value of “5” for the center of the optimal clock phase placement (OSP1), and a value of “7” for the right clock edge (R1) for a first clocking rate (e.g., $1\times$) at a first operating frequency f_1 . The value 1, value 2, value 3, etc. may refer to a particular value, but the numbers 1, 2, 3, etc. are not necessarily the code values or other settings for clock phase placement. The sweeping for clock phase placement (e.g., code) may be performed for each clocking rate for a frequency (e.g., and die transmitter circuit and die receiver circuit combination).

[0121] In one embodiment, optimal clock phase placement (e.g., $OCPL = L + (R - L)/2$) may result in a fraction. One option for a fraction result is to round up or down the OCP value (e.g., always performing the same rounding type). A second option is to employ a fraction (e.g., half-step) PI setting, for example, if a standard PI step is an integer value (e.g., 1 ps), then the fraction (e.g., half-step) is used to generate a fraction (e.g., 0.5) of that integer step (e.g., 0.5 ps). As an example, at the end of an OCP calculation if a setting of X.5 (e.g., 6.5) is needed, then the circuitry may go to PI setting X (e.g., 6) and then turn on the half-step setting to get to X.5 (e.g., 6.5). The half-step hardware circuit (e.g., in clock circuit) may be turned on (e.g., at the end of the calculation) to add a half step. One advantage of this is to avoid adding more (e.g., twice the) area of the PI circuitry to reduce the step sizes. For example, if a PI is to interpolate between 100 ps and the PI step is chosen to be 1 ps, then 100 transistor (e.g., variable invertors discussed in reference to FIG. **13**) legs may be turned on one at a time to achieve the 1 ps, 2 ps, 3 ps, 4 ps, . . . 100 ps settings. So to achieve a 45.5 ps setting, one embodiment would be to change the entire interpolator to 200 steps of 0.5 ps each to generate 45.5 ps. This may, in this example, utilize 200 transistor legs now. However, another embodiment uses the original 100 legs and also adds just one half-transistor leg, so to achieve a 45.5 ps setting, 45 full transistor legs and the single half-leg are turned on to get to 45.5 ps setting.

[0122] Returning to FIG. **10**, the clock phase placement(s) (e.g., code or codes) may be stored **1010** in flow diagram **1000**, e.g., for that particular operating frequency (e.g., and die transmitter and die receiver combination). The sweeping **1006** and/or calculating **1008** (if performed) may be repeated (and stored **1010**) for each operating frequency until complete **1012** (e.g., frequency point of operation) and/or for each clock circuit (e.g., for each DLL+PI instan-

tiation within a die) as well as for all die connected. Once completed, the interconnect programming (e.g., table **1200**) here is complete **1014**. The inter-dies connection may then be utilized, e.g., as discussing in reference to FIG. **14** below.

[0123] As an example of a clock circuit (e.g., in a receiver circuit), FIG. **13** illustrates a digital delay-locked loop (DLL) delay line and digital phase interpolator circuit **1300** according to embodiments of the disclosure. In one embodiment, a circuit (e.g., clock circuit **408** of FIG. **4** and/or clock circuit **708** of FIG. **7**) includes an instance of circuit **1300**. In certain embodiments, e.g., to counteract die to die and within-die process, voltage, and/or temperature variations, the ratio of data lanes per clock lane (e.g., forwarded source-synchronous clocks) may be optimized for best performance. For example, a single forwarded source-synchronous clock (e.g., single clock lane) per a plurality (e.g., 32, 64, 128, 256, 512, etc.) data lanes may be used, e.g., to achieve the desired granularity (e.g., a plurality of equally spaced steps for each single clock phase) (e.g., a plurality of steps between adjacent, received clock edges). In certain embodiments, the clock circuit (e.g., DLL and PI) tuning information for each operating point for the clock circuit controlling these data lanes, for example, will be unique on each die due to physical (e.g., process, voltage, and/or temperature) variations and/or on-die unique power delivery conditions. In one embodiment, a die to die connection (e.g., interconnect) includes 2048 total data lanes connecting multiple die together through these I/O lanes, then using the example of 128 data lanes per clock lane (e.g., clock signal), one would calibrate and store unique clock setting (e.g., clock phase placement) (e.g., DLL+PI) information for a total of 32 unique die crossings (2048/32=64 unique I/O block instances to comprise 32 crossings). Circuit **1300** is a schematic of phase-generation hardware that includes both DLL+PI functionality. Buffers **1302** in the center of the schematic are the digital DLL delay line and each generate a delay value (e.g., X number) (e.g., of picoseconds) of the delay. Each gate (e.g., gate **1308**) may include an interpolator circuit **1304** and/or interpolator circuit **1306**, although only the interpolator (e.g., muxing) circuits connected to node **4** and node **5** are depicted for clarity. Interpolator circuits thus allow for any two buffers that are in sequence (for example, nodes ck2 and ck3, or ck4 and ck5 as shown in the diagram) to be routed to the digital phase interpolators shown at the top and the bottom of the schematic. Phase interpolation works by varying the strengths of the two “fighting” variable invertors. For example, if one wanted the phase of ck4 to come out of the (rising edge) interpolator circuit **1304** at the top of the schematic, one would enable all 31 legs of mix_r_en[30:0] circuit **1310** and disable all of the mix_r_enb[30:0] circuit **1312** legs, e.g., to achieve the desired granularity. If one wanted the phase of ck5 to come out of the same (rising edge) interpolator circuit **1304**, then the exact opposite would be done: disable all mix_r_en[30:0] circuit **1310** legs and enable all mix_r_enb[30:0] circuit **1312** legs, e.g., to achieve the desired granularity. If one wanted a phase exactly in the middle of ck4 and ck5 to come out of the same interpolator circuit **1304** at output **1314**, then one would enable exactly half of the mix_r_en[30:0] circuit **1310** legs and also exactly half of the mix_r_enb[30:0] circuit **1312** legs, e.g., to achieve the desired granularity. If one wanted a phase that was a quarter of the way between ck4 and ck5, then one would enable three quarters of the mix_r_en[30:0] circuit **1310** legs and enable one quarter of

the mix_r_enb[30:0] circuit 1312 legs, etc., e.g., to achieve the desired granularity. In the specific case of the schematic shown, one can interpolate 31 steps between any sequential clock (ck) phases of the DLL delay line, although any number may be achieved, e.g., by adding further buffers/mix circuits to achieve the desired granularity. For example, if one wanted the phase of ck4 to come out of the (falling edge) interpolator circuit 1306 at the bottom of the schematic, one would enable all 31 legs of mix_f_en[30:0] circuit 1316 and disable all of the mix_f_enb[30:0] circuit 1318 legs, e.g., to achieve the desired granularity. If one wanted the phase of ck5 to come out of the same (falling edge) interpolator circuit 1306, then the exact opposite would be done: disable all mix_f_en[30:0] circuit 1316 legs and enable all mix_f_enb[30:0] circuit 1318 legs, e.g., to achieve the desired granularity. If one wanted a phase exactly in the middle of ck4 and ck5 to come out of the same interpolator circuit 1306 at output 1320 then one would enable exactly half of the mix_f_en[30:0] circuit 1316 legs and also exactly half of the mix_f_enb[30:0] circuit 1318 legs, e.g., to achieve the desired granularity. If one wanted a phase that was a quarter of the way between ck4 and ck5, then one would enable three quarters of the mix_f_en[30:0] circuit 1316 legs and enable one quarter of the mix_f_enb[30:0] circuit 1318 legs, etc., e.g., to achieve the desired granularity. In the specific case of the schematic shown, one can interpolate 31 steps (e.g., of clock phase granularity) between any sequential clock (ck) phases of the DLL delay line, although any number may be achieved, e.g., by adding further buffers/mix circuits. Table 1200 or other data structure may store the settings to place a clock phase as desired (for example, the settings for the mix circuits, e.g., circuits 1310, 1312, 1314, 1316). Output 1314 and output 1320 may be sent (e.g., by clock circuit 408 of FIG. 4 and/or clock circuit 708 of FIG. 7) to a receiver (e.g., one or more of receivers 414B, 414C of FIG. 4 and/or one or more of receivers 714B, 714D, 714E, 714F of FIG. 7) to clock data into a receiver (e.g., trigger when to latch data into a latch circuit).

[0124] FIG. 14 illustrates a flow diagram 1400 for a frequency transition through an interconnect according to embodiments of the disclosure. Circuitry (e.g., FSM) may utilize flow diagram 1400. In one embodiment, first die to send data to a second die and/or second die to receive data from the first die utilize flow 1400. In one embodiment, flow 1400 occurs when a first die is cause a transition of operating frequency and/or clocking rate, e.g., in a second die and/or interconnect therebetween. Flow 1400 includes halting interconnect (e.g., only from first die to second die) between the dies (and/or mesh interconnect on the first and/or second dies), e.g., halting via a ring stop of the interconnect and/or a back pressure signal circuit 1402; locking clock circuit (e.g., clock circuit 410 and/or clock circuit 420 in FIG. 4 or clock circuit 710 in FIG. 7) to a new desired operating frequency and/or clocking rate 1404; retrieving stored clock phase placement(s), for example OCP value(s) and/or other DLL+PI settings, for each clock circuit (e.g., receiver clock circuit 408 in FIG. 4 or receiver clock circuit 708 in FIG. 7) for (e.g., all) instantiations on (e.g., all) die(s) 1406; placing all clock circuit(s) (e.g., DLLs) in open-loop mode 1408; updating clock phase placement with clock phase placement data retrieved in retrieval 1406 (e.g., overriding existing settings) 1410; putting clock circuits (e.g., receiver clock circuit 408 in FIG. 4 or receiver clock circuit 708 in FIG. 7) back in closed-loop mode 1412 (e.g., such that the clock

circuit functions according to the updated clock phase placement); and resume interconnect traffic (e.g., only from first die to second die) (e.g., release back pressure and/or release the stop by ring stop) (e.g., and/or resume intra-mesh traffic in first die and/or second die) 1414.

[0125] In certain embodiments, once normal operation post-boot has started, circuitry is to pick clock phase placement (e.g., DLL+PI) information stored in the memory (e.g., array) for the starting frequency and/or clocking rate desired and update the clock circuit (e.g., DLL+PI) with this data (e.g., the codes). This may be done for each clock circuit (e.g., of a coupled receiver circuit) (e.g., DLL+PI) instantiation. In one embodiment, first, circuitry is to halt data transfer in the interconnect (e.g., by use of a back-pressure mechanism), second, place clock circuit (e.g., DLL) in open-loop mode and update clock circuit (e.g., DLL+PI) on each instantiation and each die with their respective clock circuit (e.g., trained DLL+PI) codes from for the desired frequency of operation and/or clocking rate, third, once codes are updated, place the clock circuits (e.g., DLLs) back in closed-loop mode (for example, to allow the clock circuits to perform auto-tracking to compensate for temperature and voltage drift, e.g., different than changing the operating frequency and/or clocking rate), and fourth, resume data transfer on the interconnect (e.g., by releasing the back pressure, data traffic halting mechanism). In one embodiment, a summary of the flow from the circuitry is to tune I/O (e.g., clock circuit(s)) clock phase for each frequency operating point and/or clocking rate for those operating points, store values in a storage array (e.g., upon first bootup sequence of processor), retrieve clock phase information (e.g., from register/memory) each time a frequency and/or clocking rate change is desired and update clock circuits (e.g., DLL+PI) with these values for a rapid update as opposed to much longer auto calibration/training that would be required (e.g., certain embodiments herein make the transition to a different frequency and/or clocking rate seamless or on the fly, e.g., within 1, 2, 3, 4, 5, 6, 7, 8, 9, or 10 clock cycles, e.g., to allow for trained codes to be retrieved from memory arrays and updated into the clocking circuit (e.g., DLL+PI offset) control register(s) (e.g., control register 409 in FIG. 4 or control register 709 in FIG. 7). In one embodiment, when initiating a frequency and/or clocking rate transition during normal operation (e.g., a receiver circuit receiving a request from a transmitter circuit to change the clocking rate and/or operating frequency), the data flow between the dies through the interconnect is temporarily halted (e.g., for 1, 2, 3, 4, 5, 6, 7, 8, 9, or 10 clock cycles) to allow for the (e.g., DLL+PI) clock tuning update for the new operating point. In one embodiment, power management circuit (e.g., a power management controller) causes (e.g., controls) the frequency and/or clocking rate transition.

[0126] FIG. 15 illustrates clocking architecture of a receiver circuit 1500 according to embodiments of the disclosure. Receiver circuit 1500 may be utilized as DLL circuit (for example, clock phase delay generator (e.g., DLL circuit) 408A in FIG. 4 or clock phase delay generator (e.g., DLL circuit) 708A in FIG. 7). Receiver circuit 1500 includes a local clock buffer 1502 to clean up the edges of the received clock, e.g. received clock signal(s) (e.g., clock positive (clkp) and/or clock negative (clkn)) and may remove clock jitter. QLS is a quadrature lock sensor. Finite state machine (FSM) 1504 may include a first state where

the circuit is in a closed-loop mode (e.g., where the settings therein may not be changed) and a second state where the circuit is in an open-loop mode (e.g., where the settings therein may be changed).

[0127] FIG. 16 illustrates clock timing diagrams (1604, 1608) for 1× and 2× clocking rate modes according to embodiments of the disclosure. Clock timing diagram (e.g., where the horizontal axis is the passage of time and the vertical axis is the data signal) 1602 and 1606 illustrates a reference clock, clock timing diagram 1604 illustrates a 180 degree offset (e.g., 1× clocking rate mode) relative to the reference clock 1602, and clock timing diagram 1608 illustrates a 90 degree offset (e.g., 2× clocking rate mode) relative to the reference clock 1606. Clock circuitry herein may perform an (e.g., further) offset, for example, according to the (e.g., trained) clock phase placement settings for particular circuitry (e.g., table 1200 in FIG. 12). In one embodiment, clock circuit (e.g., clock phase delay generator 408A in FIG. 4 or clock phase delay generator 708A in FIG. 7) is to look up the settings to set the clock timing diagrams (e.g., in 1× or 2× mode) when a frequency and/or clocking rate change is to occur and utilize those looked-up values (e.g., for the points A and B on the diagrams), for example, in a data structure, e.g., from Table 12. For example, in 2× mode, line A is depicted as a longer passage of time than line B (e.g., to indicate the circuitry has purposely added the trained offset to optimize the clock phase setting), although the 210° setting (e.g., adding 30° of trained offset to 180° offset) is an example.

[0128] FIG. 17 illustrates clock timing diagrams 1700 for 1× and 2× clocking rate modes according to embodiments of the disclosure. Diagrams 1700 illustrate a mesh (e.g., interconnect) dataA (e.g., DO) and dataB (e.g., D1), as well as the multiple die (e.g., fabric) interconnect (e.g., MDFI) receiver and transmitters signals, e.g., for a 1× mode and 2× mode. FIG. 17 illustrates a clock signal in comparison to the data signals in 1× mode and 2× mode for a same frequency. MDFI or other circuitry herein may be used in a server.

[0129] FIG. 18 illustrates a transmission datapath 1800 of a transmitter circuit 1803 that includes lane repair circuitry according to embodiments of the disclosure. Transmission datapath 1800 includes a die portion 1801 (e.g., a first die) and a transmitter circuit 1803, e.g., with an interconnect therebetween. In one embodiment, transmitter circuit is used as transmitter circuit 402 of FIG. 4 or transmitter circuit 702 of FIG. 7. Lane repair multiplexer (mux) may switch from a lane (e.g., wire) that is not functioning (e.g., that needs repair) and a lane (e.g., wire) that is functioning. Example delays caused are listed by certain components herein.

[0130] As one example, transmitter circuit 1802 may receive data from a data generator 1820A and/or data generator 1820B of a first die that is to be transmitted to a receiver circuit (e.g., second die including receiver circuit). Data generator 1820A and/or data generator 1820B of first die may be a processor (e.g., a processor including a decoder to decode an instruction into a decoded instruction and an execution unit to execute the decoded instruction to generate the data) of the first die. Data to be transmitted may include first data (e.g., data stream) (e.g., data DO) and (e.g., separate) second data (e.g., data stream) (e.g., data D1).

[0131] A clock signal (e.g., from or based on the clock signal in first die) from the transmitter circuit 1802 (e.g., transmitter side) may be sent (e.g., forwarded) along with (e.g., concurrently with) the data (e.g., payload data) being

sent to the receiver circuit 1804. Clock circuit 1820 may be the internal (e.g., main) clock of the first die (e.g., of the mesh in the first die). Clock circuit 1810 may be a separate clock generator, e.g., separate from the internal (e.g., main) clock of the first die, and/or a dedicated clock circuit of the transmitter circuit 1802. A multiplexer may select and output one of multiple inputs according to a control signal. Multiplexer (mux) 1828 may be set to provide a clock signal from clock circuit 1810 or clock circuit 1820, e.g., based on a control signal. Multiplexer 1828 (and/or other control signals) may be controlled by power management circuit 1832, e.g., based on a control signal received from power management circuit (e.g., a power management controller). A power management circuit may control the switching of an operating frequency and/or a clocking rate, for example, the operating frequency and/or a clocking rate in a first die and/or in a second die (e.g., connected via an interconnect to the first die). A local and/or dedicated clock circuit (e.g., clock circuit 1810) (e.g., in an I/O PHY) (e.g., phase-locked loop (PLL) circuit) may be employed to enable higher I/O bandwidths by filtering the (e.g., mesh) barrier clock jitter components.

[0132] Transmitter 1803 (e.g., amplifier) may receive a signal (for example, from a requestor, e.g., a first die to request that the interconnect and/or second die receive data at a faster or slower frequency and/or clocking rate) indicating which (e.g., clocking) mode the transmitter circuit 1802 is to be in, e.g., 1× or 2× clocking rate mode. Transmitter 1805 may receive a signal indicating the data is valid, e.g., as discussed above in reference to FIG. 4. Multiplexer 1828 is to send a clock signal (or a clock signal may be sent directly without use of multiplexer 1828). Circuit component 1817 and other such instances of that circuit component may be a rising edge triggered mux and a falling edge triggered mux, for example, to perform an action based on a rising edge of a signal (e.g., clock) and an action based on a falling edge of a signal (e.g., clock), e.g., a serializer circuit.

[0133] Transmitter (TX) select circuit block 1809 may receive a signal indicating if the transmitter circuit (and receiver circuit coupled to interconnect 1806) is to be in a first or second (or other) clocking mode. As discussed in reference to FIG. 4, a positive clock signal (TxCLKP) and negative clock signal (TxCLKN) may be utilized, or a single clock signal (e.g., TxCLKP) may be utilized (e.g., as discussed in reference to FIG. 7).

[0134] Transmitter (TX) valid circuit block 1811 may receive a signal indicating if the transmitter circuit (and receiver circuit coupled to interconnect 1806) is to transmit data, e.g., as discussed above in reference to FIG. 4. Transmitter (TX) clock circuit block 1813 may receive a clock signal for the data that is to be sent. Transmitter (TX) data circuit block 1815 may receive the data signal or signals of the data to be transmitted, for example, in a first or second (or other) clocking mode (e.g., transmitted to a receiver circuit coupled to interconnect 1806). LCB may generally refer to a local clock buffer 1502 to clean up the edges of the received clock, e.g. received clock signal(s), and may remove clock jitter. In certain embodiments, debug circuit 1807 is used to send the patterns that are used to sweep (e.g., train) the circuitry. For example, debug circuit 1807 may send signals (e.g., D0, D1, D2, or D3) (e.g., turned off and on) (e.g., from high to low and then low to high) multiple times to generate the eye diagram 1102 in FIG. 11, e.g., to

train the circuitry according to the flow diagram **1000** in FIG. **10** (e.g., to generate the table in FIG. **12**). Clocking rate signal (e.g., received by transmitter **1803**) (e.g., from a requestor, e.g., a first die to request that the interconnect and/or second die receive data at a faster or slower frequency and/or clocking rate) indicating which (e.g., clocking) mode the transmitter circuit **1802** is to be in, e.g., 1× or 2× clocking rate mode, may further switch the transmitter data circuit block **1815** between modes for each clocking rate. TD[*] may refer to a transmission data path, and the asterisk may be replaced by a number for that lane, e.g., data DO may be transmitted on TD[0]. In one embodiment, transmitter circuit **1802** may output (e.g., to interconnect **1806**) a clock signal (e.g., either of or both of TxCLKP or TxCLKN), and one or more data signals (e.g., TD[*] where the * is the lane number), a valid signal (e.g., either of or both of TValidP or TValidN), a clocking rate signal (for example, TSelect, e.g., being one or multiple bits), or any combination thereof. A circuit outputting a positive and a negative signal may use an inverter on the input to that circuit to invert the positive signal to produce the negative signal.

[0135] In certain embodiments, e.g., given the I/O (e.g., PHY) (e.g., high) lane count possible to implement multiple-die processors, redundant lanes may be included inside the I/O (e.g., PHY), for example, to allow for post silicon processing and post-package assembly defect repairs. One repair scheme, at a high-level, includes muxed paths between adjacent I/O lanes inside both the TX and RX lanes that may be programmed appropriately to fix any defective lanes, e.g., due to silicon processing defects and/or package (e.g., interconnect) assembly defects.

[0136] FIG. **19** illustrates clock timing diagrams **1900** for a 1× clocking rate mode of a transmitter circuit according to embodiments of the disclosure. In one embodiment, clock timing diagrams **1900** are utilized for the circuitry in FIG. **18**, e.g., in 1× clocking rate mode.

[0137] FIG. **20** illustrates clock timing diagrams **2000** for a 2× clocking rate mode of a transmitter circuit according to embodiments of the disclosure. In one embodiment, clock timing diagrams **2000** are utilized for the circuitry in FIG. **18**, e.g., in 2× clocking rate mode.

[0138] FIG. **21** illustrates a receiver datapath **2100** of a receiver circuit **2104** that includes clock-crossing buffers according to embodiments of the disclosure. RD[*] may refer to a receiver data path, and the asterisk may be replaced by a number for that lane, e.g., data DO may be received on RD[0]. In one embodiment, receiver circuit **2104** is coupled (e.g., via interconnect **2106**, e.g., in one embodiment, interconnect **2106** is coupled to or the same as interconnect **1806** in FIG. **18**) to a transmitter circuit. Receiver circuit **2104** includes one or more inputs to receive signals, e.g., from interconnect **2106**. Depicted receiver circuit **2104** includes clock receiver **2113** to receive one or more clock signals (e.g., for signals RxCLKP (where P stands for positive) and/or RxCLKN (where N stands for negative)), valid signal receiver **2111** (e.g., for signals RValidP and/or RValid N), clocking rate receiver **2109** (e.g., for signal RSelect), data receiver or receivers (**2115A**, **2115B**) (e.g., for RD[0] and RD[1], with 0 and 1 being examples of two different lanes (e.g., signals)), although any combination thereof may be utilized. In one embodiment, receiver circuit **2104** is coupled to transmitter circuit **1802** of FIG. **18**, such that each TD[*] is coupled to a respective RD[*] (e.g., to alternative sending data DO and data D1), TxCLKP is coupled to RxCLKP,

TxCLKN is coupled to RxCLKN, TValidP is coupled to RValidP, TValidN is coupled to RValidN, TSelect is coupled to RSelect, or any combination thereof.

[0139] Receiver circuit **2104** includes a clock circuit **2108** (e.g., DLL or DLL+PI). In one embodiment, clock circuit **2108** receives clock signal from a transmitter circuit (e.g., transmitter circuit **1802** of FIG. **18**) to align (e.g., shift) the (e.g., source-synchronous) clock edges of a received clock signal (e.g., waveform) from the transmitter circuit with the corresponding received data signal(s) (e.g., different than a clock signal) for high-performance timing, e.g., such that the data in the data signal is not altered, lost, destroyed, or any combination thereof. Clock circuit **2108** may include a clock phase delay generator (e.g., DLL circuit) and/or phase interpolator circuit, e.g., as discussed herein. In one embodiment, clock phase placement is performed by a phase interpolator e.g. phase interpolator circuit. In one embodiment, a phase interpolator is a circuit that adjusts (e.g., shifts) the phase of a clock signal. In one embodiment, a phase interpolator has a level (e.g., 2, 4, 6, 8, 10, 12, etc.) of granularity of steps per each clock phase e.g., that are equally spaced apart and it may set a rising clock edge and/or falling clock edge at any of those steps, for example, as discussed further in reference to FIG. **13**.

[0140] Clock circuit **2108**, e.g., including a delay-locked loop (DLL) circuit, may be employed at the receiver circuit **2104** of the receiver die to appropriately align the source-synchronous clocking edge for high-performance timing (e.g., to enable effective high-speed signaling). A DLL circuit may be a negative-delay gate placed in the clock path of a digital circuit. In one embodiment, clock circuit **2108** is a component of receiver circuit **2104**. A local and/or dedicated clock circuit (e.g., clock circuit **410** in FIG. **4**) (e.g., in an I/O PHY) (e.g., phase-locked loop (PLL) circuit) may be employed to enable higher I/O bandwidths by filtering the (e.g., mesh) barrier clock jitter components. PLL circuit may be a control circuit that generates an output signal whose phase is related to the phase of an input signal. Although there are different types of PLL circuits, one example is a circuit with a variable frequency oscillator and a phase detector in a feedback loop, e.g., where the oscillator generates a periodic signal, the phase detector compares the phase of that signal with the phase of the input periodic signal, and adjusts the oscillator to keep the phases matched. A PLL may be an all digital PLL (ADPLL). In one embodiment, a DLL circuit uses a variable phase (e.g., delay) block and a PLL circuit uses a variable frequency block. Clock circuit **2108** may include a control register **2107**, for example, to store the clock phase placement settings, e.g., to cause clock circuit **2108** to apply those settings.

[0141] Receiver buffer synchronizer **2152** may utilize the clock signal (e.g., a modified clock signal based on the clock phase placement settings) to clock in the data (e.g., with receiver **2115A**, receiver **2115B**, latch (e.g., flop) **2154C**, and/or latch (e.g., flop) **2154D**), the valid signal (e.g., with receiver **2111** and/or latch (e.g., flop) **2154A**), the clocking rate signal (e.g., with receiver **2109** and/or latch (e.g., flop) **2154B**), or any combination thereof. In certain embodiments, one or more of those data items may be sent to a respective buffer (e.g., buffers **2150A**, **2150B**, **2150C**, and **2150D**). Receiver buffer synchronizer **2152** may receive one or more of these signals (e.g., modified clock signal based on the clock phase placement settings) to buffer data and send corresponding data signals to die **2103**, for example, send a

corresponding (e.g., matching or substantially matching the signals that were sent from the receiver) set of signals for valid data (e.g., Valid), clocking rate (e.g., Select[]), and/or the data (e.g., payload) (e.g., DataA[*] and/or DataB[*]), for example, a set of signals for a forwarded clock signal.

[0142] FIG. 22 illustrates clock timing diagrams 2200 for a 1× clocking rate mode of a receiver circuit according to embodiments of the disclosure. In one embodiment, clock timing diagrams 2200 are utilized for the circuitry in FIG. 21, e.g., in 1× clocking rate mode.

[0143] FIG. 23 illustrates clock timing diagrams 2300 for a 2× clocking rate mode of a receiver circuit according to embodiments of the disclosure. In one embodiment, clock timing diagrams 2300 are utilized for the circuitry in FIG. 21, e.g., in 2× clocking rate mode.

[0144] A processor, e.g., as discussed herein, may include one or more of the features or circuits discussed herein. A processor may be formed on a single fabrication of integrated circuits (e.g., as a single die). In one embodiment, a single die may have manufacturing process defects that impede or remove certain functionality of the die. This liability to process defect may increase with the die area. The fabrication investment at risk of loss in construction may increase with the die area (e.g., of large processors). A processor may be formed on a single fabrication having all hardware functionality at one design release and not have hardware supported features added, enhanced, or optimized where those new capabilities were not in the original design release. Certain embodiments herein may provide solutions to the above.

[0145] Certain embodiments herein provide sharing processor primary resources over a high bandwidth and low-latency electrical interconnect such that the performance in accessing remote die resources is better, the same, or substantially the same (e.g., very near) the performance of a monolithically fabricated integrated die. Certain embodiments herein provide sharing processor infrastructure resources to enable intimate management of power, thermal, clocking, reset, configuration, error handling, etc., or combinations thereof, with an electrical interconnect such that the performance in accessing die resources (e.g., between a first die and a second die) is better, the same, or substantially the same (e.g., very near) the performance of a monolithically fabricated integrated die. Certain embodiments herein reduce the fabrication yield risk associated with a single large die size. Certain embodiments herein allow scaling to larger numbers of functional logic components to offer redundancy for yield recovery and/or special uses such as die testability. Certain embodiments herein allow a late decision on design cycle whether to manufacture a monolithic design of a die or multiple dies (e.g., a 2 way or 4 way split of the single die design).

[0146] Certain embodiments herein allow combinations of dissimilar dies to enable staging over time design completion for some dies or for some dies to be manufactured in more matured or special fabrication process, as well as better monetizing some older dies from previous products. Certain embodiments herein allow combinations of dissimilar dies and/or quantities of dies to enable a wide variety of unique processors products (e.g., stock keeping units (SKUs)) with minimal or without re-design efforts.

[0147] Certain embodiments herein provide for a larger (e.g., area) die to connect to a smaller (e.g., area) die or multiple dies having a different number of physical connec-

tions on their die. Certain embodiments herein allow for the forming of a processor from the same and/or a mirrored version(s) of a die duplicated multiple times to create a larger monolithic domain. Certain embodiments herein allow a scale up in two dimensions (e.g., X-Y) and/or three dimensions (e.g., X-Y-Z).

[0148] FIG. 24 illustrates a hardware processor 2400 having two dies (2402, 2404) that share resources via an interconnect 2406 according to embodiments of the disclosure. Although not depicted, certain circuitry (e.g., decode unit(s), execution unit(s), core(s), cache coherency circuitry, cache(s), or other components) may be utilized, for example, as discussed below. In one embodiment, the processor components on a single die 2402 may be coupled together via an electrical interconnect, such as a high bandwidth and low-latency interconnects illustrated in FIG. 24. For example, die 2402 may include one or more of components 2408 (e.g., that communicate with each other) and die 2404 may include one or more of components 2410 (e.g., that communicate with each other), for example, where the components of first die 2402 communicate with the components of second die 2404 through electrical interconnect 2406. In one embodiment, components include a memory (for example, a cache, e.g., in coherent die memory). In one embodiment, coherent die memory is circuitry that includes a cache coherency circuit, for example, to manage cache coherency, e.g., in one or more dies. In one embodiment, physically separate die 2402 is to communicate with physically separate die 2404 through interconnect 2406. In one embodiment, the processor components on a single die 2402 may be coupled together via an electrical interconnect, such as the (e.g., intra-die) mesh interconnects (2420, 2422) depicted in each die illustrated in FIG. 2. For example, die 2402 may include one or more of components 2408, e.g., that may communicate via interconnect 2420 with other of components 2408. For example, die 2404 may include one or more of components 2410, e.g., that may communicate via interconnect 2422 with other of components 2410. Die and/or interconnect may include a transceiver (e.g., one or more instances of receiver circuit(s) and/or one or more instances of transmitter circuit(s) disclosed herein) to transmit data between die 2402 and die 2404. Note that a single headed arrow herein may not require one-way communication, for example, it may indicate two-way communication to and from that component. Any or all combinations of communications paths may be utilized in certain embodiments herein. In one embodiment, each of die 2402 and die 2404 are identical. In another embodiment, die 2404 is a mirror image (e.g., reversed image) of die 2402. In one embodiment, die 2402 and die 2404 are different, for example, each representing a portion of a single die design that has been cleaved into multiple physical dies that are then joined together (e.g., electrically coupled) via interconnect 2406.

[0149] Certain embodiments herein provide for merged infrastructure across coupled (e.g., adjacent or stacked) dies. Certain embodiments herein provide infrastructure messaging electrical interconnect that supports one or more of multi-die cohesive and/or unified management and as well die independent management. Infrastructure management may include management of power supply, thermal, clock, boot/reset, power-down/throttle/turbo modes, debug, testing, reliability/serviceability, security, performance monitoring and analytics, configuration/control, and/or any combi-

nation thereof. In certain embodiments, an electrical interconnect between dies is capable of early wire signaling and as well more complex messaging enables multi-die cohesive and/or unified management in a monolithic master-slave hierarchical mode to provide a low-latency and responsive dominion over a wide area of processor, with significant added capabilities to a central management. Certain embodiments herein designate a management circuit in one of the infrastructure circuits in each of the plurality of physically separate dies as master and the rest as slave to the master.

[0150] In certain embodiments an electrical interconnect between dies and separately connected to each die enables die independent mode to provide a separately addressable die access, means to isolate dies, and die functionality to test each die independently within a package or for conditionally disabling some dies in packaged product in case early parts suffer from low fabrication yields. Infrastructure circuitry in each of the plurality of physically separate dies may be switchable between a master mode and a slave mode. Cache coherency circuitry in each of the plurality of physically separate dies may be switchable between a master mode and a slave mode. Cache coherency circuitry may be provided in each of the plurality of physically separate dies that is switchable between a master mode and a slave mode. Cache coherence circuitry, for example, as part of a cache, may be utilized according to a cache coherence protocol, e.g., the four state modified (M), exclusive (E), shared(S), and invalid (I) (MESI) protocol or the five state modified (M), exclusive (E), shared(S), invalid (I), and forward (F) (MESIF) protocol. Cache coherence circuitry may provide, for multiple copies of a data item (e.g., stored in an memory), an update to other copies of the data item when one copy of that data item is changed, e.g., to ensure the data values of shared operands are propagated throughout the system in a timely fashion

[0151] In certain embodiments, each die has the ability to boot independently for support of die fabrication defect testing and characterization, e.g., with the same die independent testing apparatus also effective in the case the die is packaged with the merged die connected. In certain embodiments, each die has the ability to negotiate security status and processing error status coherently to enable primary communications to pass unencumbered by encryption and fault containment overhead. A first die and a second die of the plurality of physically separate dies may extend in a single plane and a third die of the plurality of physically separate dies may be laterally spaced from that single plane.

[0152] In certain embodiments, master-slave hierarchical boot/reset/power management supports modularity and extensibility of tiling several modular dies and/or heterogeneous modular dies, while enabling extensible access to product specific breadth of the controllable infrastructure. In certain embodiments, high volume manufacturing (HVM) and test innovation provides a cohesive flow of individual dies in wafers into packaged modular die products. This may include support for HVM testing for wafer-die-sort and package-class flows and fuse programming that supports fuse settings that result from remote die attributes. In certain embodiments, security innovation enables allowing dies to transact without non-native proposal overhead and with (e.g., unlimited) resource access despite die exposure of private sideband messaging between them.

[0153] FIG. 25 illustrates infrastructure management controllers (2508, 2518) for a hardware processor 2500 having two dies (2502, 2504) that share resources via an interconnect 2506 according to embodiments of the disclosure. FIG. 25 illustrates a hardware processor 2500 according to embodiments of the disclosure. Although not depicted, certain circuitry (e.g., power controller(s), thermal sensors(s), voltage sensor(s), PLL(s), fuse array(s), or other components) may be utilized, for example, as discussed herein. In one embodiment, the processor components on a single die 2502 may be coupled together via an electrical interconnect, such as the (e.g., intra-die) mesh interconnects (2520, 2522) depicted in each die illustrated in FIG. 2. For example, die 2502 may include one or more of components 2528, e.g., that may communicate via interconnect 2520 with other of components 2528. For example, die 2504 may include one or more of components 2538, e.g., that may communicate via interconnect 2522 with other of components 2538. Any of components 2538 of die 2504 and any of components 2528 of die 2502 may communicate with each other through the electrical interconnect 2506. In one embodiment, physically separate die 2502 is to communicate with physically separate die 2504 through interconnect 2506. Die and/or interconnect may include a transceiver (e.g., one or more instances of receiver circuit(s) and/or one or more instances of transmitter circuit(s) disclosed herein) to transmit data between die 2502 and die 2504. Note that a single headed arrow herein may not require one-way communication, for example, it may indicate two-way communication (e.g., to and from that component). Any or all combinations of communications paths may be utilized in certain embodiments herein. In one embodiment, each of die 2502 and die 2504 are identical. In another embodiment, die 2504 is a mirror image of die 2502. In one embodiment, die 2502 and die 2504 are different, for example, each representing a portion of a single die design that has been cleaved into multiple physical dies that are then joined together (e.g., electrically coupled) via an interconnect. In one embodiment, an electrical interconnect of a die does not depend on a connection to another die to function, for example, the data signals (e.g., requests and/or answers) may loop back into that die, e.g., if interconnect 2506 is not functioning or present. In one embodiment, such data signals are not blocking signals (e.g., not fences).

[0154] FIG. 26 illustrates an infrastructure management controller 2620 for a hardware processor 2600 having four dies (2602, 2604, 2606, 2608) that share resources via interconnect 2601 therebetween according to embodiments of the disclosure. A mesh interconnect is not shown in each die for clarity, but it may be utilized, e.g., as in FIG. 24 or 25. FIG. 26 illustrates a three-dimensional stacked architecture. A plurality of dies may extend in any single direction with an electrical interconnect(s) between each die. In the depicted embodiment, die 2602 and die 2604 extend in a first, single plane and die 2606 and die 2608 extend in a second, different single plane that is laterally spaced from the first single plane. A die may be affixed to another substrate, e.g., a mounting substrate (not depicted). Controller 2620 may control functionality in only die 2606. Additionally or alternatively, controller 2620 may control functionality in one or more of dies (2602, 2604, 2608). Controller 2620 may control a transceiver of one or more of the dies (e.g., one or more instances of receiver circuit(s) and/or one or more instances of transmitter circuit(s) dis-

closed herein). In one embodiment, controller **2620** controls the transceivers in its die **2606**. In one embodiment, controller **2620** controls the transceivers in each (e.g., all) of the dies.

[0155] Certain embodiments herein provide for a merged infrastructure interconnect. Certain interconnects herein support bidirectional boot handshake signals and/or bidirectional messaging that allow designation of the master die, e.g., after die design, at package assembly, and/or platform assembly. Certain interconnects herein support indication of die status, e.g., to enable both holding messages in back-pressure (e.g. credit passing) and/or in long-term lack of readiness to allow auto-responding a message (e.g. not Power OK). Certain interconnects herein support stage-by-stage message delivery resource crediting, e.g., even for the stage passing between dies. Certain interconnects herein support die to die unbounded clock uncertainty and/or full bandwidth matching for cases the dies operate at the same clock frequency.

[0156] Certain interconnects herein support being brought to full functionality up (e.g., very early) in the boot sequence to allow the master die to manage the slave die(s) boot flows (e.g., for the majority of the boot flow), for example, allowing a system power management unit and a single boot service providing core to run BIOS on the entire multi-die processor. Certain interconnects herein support passage of a security status and/or functional/environmental error status to enable a monolithic domain of resolved status that allows full die-to-die communication without additional performance reducing solutions (e.g. encryption) or allows not having missed fault containment due to unseen errors. Certain interconnects herein support a separate physical channel for general purpose sideband messaging (e.g., control data and/or clock data) interconnect without shared resource with a second dedicated power management sideband messaging interconnect. This may support an unencumbered dedicated channel for power/clock/reset management that is not liable of a deadlock. Certain interconnects herein support a programmable message address translation known as a sideband address bridge to enable addressing through far die routers and designation decoding that were not known to the transmitting die at the time the die was constructed.

[0157] Certain embodiments herein provide master and slave designations, e.g., via one or more controllers. In certain embodiments, master-slave resource management across dies is supported by a die bump(s) that permanently designate the master die at the package construction. For example, during boot a read of that value will instruct a (e.g., infrastructure management) controller to continue as master or hold internal progress until the master takes over. In certain embodiments, operation in testing while in the wafer sort command the unpackaged die under test to behave as a master with no slave dies. In this case of each die as master and operating independent and without other dies, the die-to-die interconnect may be isolated, e.g., taken to safe signal values and loop-back paths provided (e.g., for the ports that would couple to the interconnect if they were utilized).

[0158] FIG. 27 illustrates infrastructure management controllers (**2720**, **2722**, **2724**, **2726**, **2728**, **2730**) for a hardware processor **2700** having six dies (**2702**, **2704**, **2706**, **2708**, **2710**, **2712**) that share resources via an interconnect **2701** according to embodiments of the disclosure. In the depicted embodiment, die **2710** and **2712** are smaller (e.g., in area)

than die **2702**, die **2704**, die **2706**, and die **2708**. FIG. 27 illustrates that certain of a plurality of dies may be different in certain embodiments (e.g., in one embodiment, they are not symmetric). FIG. 27 illustrates that an infrastructure on-die interconnect on a die may be different than another infrastructure on-die interconnect on a different die in certain embodiments (e.g., in one embodiment, they are not copies of the same die or symmetries of the same die). In one embodiment, controller **2720** is the master controller and the other controllers are slaves to that master (e.g., under the control of that master).

[0159] FIG. 28 illustrates infrastructure management controllers (**2820**, **2822**, **2824**, **2826**, **2828**, **2830**) for a hardware processor **2800** having six dies (**2802**, **2804**, **2806**, **2808**, **2810**, **2812**) coupled via an interconnect **2801** according to embodiments of the disclosure. In the depicted embodiment, die **2810** and **2812** are smaller than die **2802**, die **2804**, die **2806**, and die **2808**. FIG. 28 illustrates that certain of a plurality of dies may be different in certain embodiments (e.g., in one embodiment, they are not symmetric). FIG. 28 illustrates that an infrastructure on-die interconnect on a die may be different than another infrastructure on-die interconnect on a different die in certain embodiments (e.g., in one embodiment, they are not copies of the same die or symmetries of the same die). In one embodiment, each of the controllers (**2820**, **2822**, **2824**, **2826**, **2828**, **2830**) is a master, e.g., none are slaves to another master.

[0160] In certain embodiments (for example, where each die is individually manufactured and/or tested, e.g., even when to-be-assembled in a multiple-die package with an interconnect according to this disclosure), loop-back capability is provided, e.g., for any traffic that is addressed to cross a die boundary of a first die (e.g., but another die is not connected to that first die boundary or communication across that die boundary is not desired or enabled (e.g., yet)). In one embodiment, the loop-back capability is provided by a controller. If the request (e.g., to cross a die boundary) is a non-posted request (e.g., where the requested transaction causes a response to indicate success or failure of the requested transaction), a controller may return an “unsupported request” message and/or legally retire/terminate/block a message that is trying to cross to the other die. In one embodiment, messages (e.g., traffic) to cross a die boundary is prevented at the sending component of a die unless specifically authorized, but in certain cases (e.g., a broadcast message to send data to multiple dies) precluding the messages may not be desired so the controller (e.g., of the receiver die(s)) may retire/terminate/block those messages. Due to the bounce or loop-back nature, the retirement, termination, and/or blocking of these messages is illustrated as a returning arrow (e.g., returning arrow **2840**). Certain embodiments thus may provide isolation between dies.

[0161] Certain embodiments herein provide for a merged infrastructure boot flow. Certain embodiments herein provide for multiple physically separate discrete dies to be electrically interconnect coupled to the platform infrastructure status signaling or to receive to the platform infrastructure status though a master die (e.g., a die that has the mastership). In one embodiment, both cases occur in the same platform at separate times of the boot sequence. Certain embodiments herein provide reuse of the die-independent boot flows for some sequences, e.g., even in the case that master-slave monolithic merged die mode will ultimately manage portions of the flow from the master die.

[0162] FIG. 29 illustrates a flat communication topology 2902 for data exchanges in a multiple die processor 2900 according to embodiments of the disclosure. In the depicted embodiment, topology 2902 represents a flat communication structure that resembles multiple independent processors, as seen in a platform with multiple processor sockets/packages.

[0163] FIG. 30 illustrates a hierarchical master and slave communication topology 3004 for data exchanges in a multiple die processor 3000 according to embodiments of the disclosure. In the depicted embodiment, topology 3004 represents a hierarchical master-slave communication structure that resembles a single processor as seen by the platform, as seen in a platform with a single processor socket/package. FIGS. 26-30 illustrate that a combination of the two structures may be used through the various phases of boot start-up, e.g., with flat topology often predominate at early stages and the hierarchical topology taking over as the processor becomes more enabled.

[0164] FIGS. 31A-31B illustrate a flow diagram 3100 for a master and slave boot and a die-independent boot according to embodiments of the disclosure. The crossed-out portions of the flow diagram indicate steps that may be removed during a boot according to embodiments herein. In another embodiment, those crossed-out portions may be utilized. Flow 3100 includes providing a plurality of physically separate dies in the left column (e.g., for die 1), center column (e.g., for die 2), and right column (e.g., for die 3) of flow 3100. Three dies are used as an example, and any number or dies may be utilized. Dies are electrically coupled, e.g., the plurality of physically separate dies are coupled together with an electrical interconnect to create a hardware processor. Flow stage 3102 initiates the sequence with a broadcast signal to indicate that platform power and clock are ready. Each die is treated as an independent processor at this stage. Flow stage 3104 depicts the actions taken by the controller (e.g., hardware controller) (e.g., controller(s) in FIGS. 25-28) for infrastructure startup. The die-to-die electrical interconnect used for master-slave infrastructure management is enabled at the end of this phase in the depicted embodiments. Flow stage 3106 depicts the innovation to aggregate slave processor readiness indications, e.g., and only initiate the master processor for the higher-level controller functions. Flow stage 3108 actions are the setup of the master controller (e.g., infrastructure controller) and related infrastructure startup. Capabilities from the die-to-die infrastructure electrical interconnect discussed herein enable the master die to communicate to slave dies startup commands and receive acknowledgements. Flow stage 3110 identifies a synchronization (synch) point that all the dies have reached a readiness for reset to be released. Flow stage 3112 include large stage of actions by the master controller (e.g., infrastructure controller) to enable the broad sets of processor functionality. This may include the processor cores and microcode therein. Flow stage 3114 has the action that has the highest level of management as BIOS configures and enables functionality. Flow stage 3116 is the completion of the flow as there is a handoff to the Operating System (OS) and software. A die may include programmed or programmable fuses, e.g., data storage to store information (e.g., sensitive information, such as, but not limited to, encryption keys or manufacturer codes). The underlined portions may be additional functionality and messages added to support forming a processor from multiple dies as discussed herein. In certain embodi-

ments, a modular die infrastructure interconnect is the between die interconnect (e.g., inter-die interconnect) discussed herein. In one embodiment, enabling the interconnect is turning on (e.g., and establishing communication between) a transmitter circuit (e.g., in a first die) and a receiver circuit (e.g., in a second die), for example, one or more instances of receiver circuit(s) and/or one or more instances of transmitter circuit(s) disclosed herein. In certain embodiments herein, an infrastructure controller includes a power management circuit, e.g., as discussed herein. In certain embodiments, a mesh interconnect is the interconnect inside (e.g., intra-die interconnect) of a single die, e.g., connecting the components of that die.

[0165] Certain embodiments herein provide for a merged mesh across dies. Certain embodiments herein provide for multiple physically separate (e.g., discrete) dies to be electrically connected together by an electrical interconnect to form a larger (e.g., and having more capabilities) processor. Certain embodiments herein provide for a single shared cache coherency domain across multiple dies to form a monolithic cache domain over the entire processor. A first die and a second die of the plurality of physically separate dies may be affixed in a single plane and affixing a third die of the plurality of physically separate dies in a laterally spaced orientation from that single plane. Certain embodiments herein provide an electrical interconnect for delivering a low-latency high-bandwidth die-to-die coherent interconnect connection, e.g., the same or substantially the same as a monolithic experience. Bandwidth performance equivalency with a single die is achievable, e.g., while clock uncertainty compensation and an interlocked queued clock crossing are the same or approach nearly as low route path crossing latency and idle power saving capabilities may minimize the power consumption growth over the single die (e.g., monolithic) case. Certain embodiments herein provide for support for end-to-end destination resource crediting even across dies. Separate dies may present significant uncertainties in transaction resource status for source to destination crediting and for transaction merger (e.g., mesh “clock polarity” used in routing). Certain embodiments herein solve the resource/routing uncertainties when crossing into another die fabric with queueing and dispatching performed in the transceiver circuitry (e.g., system fabric-to-fabric crossover circuit). Certain embodiments herein provide extremely low die crossover latencies and/or solve the clock alignment uncertainties with a high performance clock crossing (e.g., a buffer or buffers, which may be referred to as a transparent queue (TQ), e.g., as in Figure cluster buffers in FIG. 21.

[0166] Although not depicted in certain Figures throughout, certain circuitry (e.g., decode unit(s), execution unit(s), core(s), cache coherency circuitry, cache(s), or other components) may be utilized, for example, as discussed herein.

[0167] FIG. 32 illustrates a hardware processor 3200 according to embodiments of the disclosure. A mesh interconnect is not shown in each die for clarity, but it may be utilized, e.g., as in FIG. 1, 2A, 2B, 33, or 34. FIG. 32 illustrates a three dimensional stacked architecture. A plurality of dies may extend in any single direction with an electrical interconnect(s) between each die. In the depicted embodiment, die 3202 and die 3204 extend in a first, single plane and die 3206 and die 3208 extend in a second, different single plane that is laterally spaced from the first single

plane. A die may be affixed to another substrate, e.g., a mounting substrate (not depicted).

[0168] In one embodiment, multiple die architecture is implemented using silicon interposer (si-interposer) as a physical manufacturing technology. In this realization, the metal wires to implement the bridging between the two or more dies may be implemented in a different die (e.g., silicon) that forms the base of all the other dies. The base die may have through silicon vias (TSVs) to deliver power to the dies and/or route the I/O signals out on to the board/external connectors. Alternately, the base die may not have TSVs and the power delivery and I/O break outs may be provided by some form of peripheral wire-bonding.

[0169] Certain embodiments herein provide for multiple physically separate discrete dies to be electrically connected together by an electrical interconnect to form a larger and more capable processor. Certain embodiments herein provide for a single shared cache coherency domain over that interconnect to form a monolithic cache domain over the entire processor. Certain embodiments herein include communication with the native protocol of each die internal data transport and does not require the overhead of packetizing nor serializing the data transmitted or received over an electrical interconnect between dies. Certain embodiments herein allow transportation according to a single or to multiple simultaneous transaction protocols between dies.

[0170] Certain embodiments herein allow for multiple dies to have relative clock alignment uncertainty, different power sources, different die fabrication process skew, and different die temperature. Certain embodiments herein allow for one die to run at a different frequency than another die or dies of that hardware processor. Certain embodiments herein allow for the interconnect to have divisible independent power, clock, and/or reset domains to help yield recovery, e.g., by disabling row and/or column of a mesh interconnect. In certain embodiments, an electrical interconnect allows (e.g., very large) cross bandwidth but also having minimal latency and power impact. Certain embodiments herein provide for a mesh loopback design, e.g., to tolerate die to die differences.

[0171] Certain embodiments herein add an entry into a look-up table (LUT) (e.g., within a transceiver) to indicate if data (e.g., a cache line) is to cross a physical die boundary to pass through an interconnect between two die. Certain transport protocols herein enable a (e.g., high speed) interconnect between multiple dies and/or seamless crossing of the die boundaries. Alternatively to using those protocols as die to die connection, certain embodiments herein may use other solutions, e.g., utilizing an interposer. Certain interconnects herein include a fabric arbitration block circuit (e.g., in a transceiver) to accommodate uncertainties in transaction destination resource status without forcing the source to delay for a latent indication, as well as accommodating transaction merger into open transaction routing slots in the remote die fabric. In certain embodiments, an electrical interconnect fabric arbitration block circuit (e.g., controller) is located at only one of a receiver circuit or a transmitter circuit. Certain interconnects herein include a post silicon tunable buffer (e.g., a transparent queue (TQ)), e.g., for supporting high bandwidth and low latencies to accomplish the die crossover amid clock alignment uncertainty, different power sources, different die fabrication process skew, and/or different die temperature. In certain embodiments, an electrical interconnect buffer may have no

latency impact if both domain are running at the same frequency and managed clock uncertainties despite dies on different power sources, different die fabrication process skew, and different die temperature. In certain embodiments, an electrical interconnect buffer is located at only one of a receiver circuit or a transmitter circuit. In certain embodiments, an interconnect buffer is located at both transmitter and receiver circuits.

[0172] FIG. 33 illustrates a hardware processor 3300 according to embodiments of the disclosure. In the depicted embodiment, die 3302 and 3304 are smaller than die 3306, die 3308, die 3310, and die 3312. Each of the depicted dies is coupled to an adjacent die via an (e.g., inter die) interconnect (INT). Die 3302 is depicted as having two discrete interconnects with die 3306, e.g., interconnects that include one or more instances of receiver circuit(s) and/or one or more instances of transmitter circuit(s) disclosed herein. Die 3304 is depicted as having a different number of (e.g., three) discrete interconnects with die 3308. Die 3306 is depicted as having four discrete interconnects with die 3308. Die 3310 is depicted as having a different number of (e.g., three) discrete interconnects with die 3312. The intersection of mesh interconnect of a die (e.g., intersection 3314 or intersection 3316 of die 3306) may be the access point into the mesh interconnect by a circuit component. In one embodiment, multiple (e.g., any) mesh configurations with different sizes on their respective die are coupled together by certain embodiments herein. In one embodiment, a die with a mesh interconnect is coupled to a die without a mesh interconnect, for example, die 3318 is depicted in FIG. 33 as coupled to mesh interconnect of die 3306 through single interconnect (INT). Although a mesh interconnect is discussed in certain embodiments, other interconnect topologies may be utilized (e.g., ring, star, tree, fully connected mesh, partially connected mesh, etc.).

[0173] FIG. 34 illustrates a hardware processor 3400 according to embodiments of the disclosure. In the depicted embodiment, dies 3402 and 3404 (e.g., of the same size) are smaller than die 3406, die 3408, die 3410, and die 3412. Die 3406 is depicted as including a different mesh interconnect than die 3408, e.g., having a different number of intersections (e.g., intersection 3414) and/or transceivers (e.g., transceiver 3416). FIG. 34 illustrates that certain of a plurality of dies may be different in certain embodiments (e.g., in one embodiment, they are not symmetric). FIG. 34 illustrates that a mesh interconnect on a die may be different than another mesh interconnect on a different die in certain embodiments (e.g., in one embodiment, they are not symmetric).

[0174] Certain embodiments herein provision coherency resources and mesh transactions. Certain embodiments here provide for a master die controller to discover resources conditions across all dies to build resource capability, resource address table, and/or routing performance bias tables. Certain embodiments of a master controller walk through anticipated possible resources and subtract, e.g., by reading remote fuses or registers and based on successful handshakes. Certain embodiments of a master controller have preprogrammed set of maps to configure the resource tables (e.g., credits), mesh look-up-tables (LUTs), address translations services (e.g., system address map), etc. to allow mesh traversal cross dies. The chosen preprogrammed map may be based on resource identified.

[0175] Certain embodiments of an electrical interconnect (e.g., and/or transceiver circuit(s)) between multiple dies provides very high bandwidth matching the bandwidth of on-die integrated (e.g., mesh) interconnect. Certain embodiments of an electrical interconnect (e.g., and/or transceiver circuit(s)) between multiple dies provides (e.g., very) low latency, e.g., which matches or substantially matches the latency of an on-die integrated interconnect. Certain interconnects (e.g., and/or transceiver circuit(s)) herein include communication with the native protocol of each die internal data transport and/or does not require the overhead of packetizing nor serializing the data transmitted or received over an electrical interconnect between dies (e.g., minimizing latency impact for the interconnect). Certain interconnects (e.g., and/or transceiver circuit(s)) herein include bandwidth reduction for communication without error protection as a way to increase data transfer efficiency and reduced latency. Certain interconnects (e.g., and/or transceiver circuit(s)) herein include dynamic transfer rate transitions (e.g., matching on-die communication bus frequency changes) on-the-fly with minimal (e.g., single-digit) clock cycles to update and transition the timing synchronization of an electrical interconnect.

[0176] Certain interconnects (e.g., and/or transceiver circuit(s)) herein provide reduced pin count but allow full cross sectional bandwidth (BW) (e.g., clocking rate), such as $\frac{1}{4}$ pins used with $4\times$ data rate as compared to data frequency within a die, or $\frac{1}{2}$ pins used with $2\times$ data rate as compared to data frequency within a die. Certain interconnects (e.g., and/or transceiver circuit(s)) herein provide reduced pin count but allow selectable bandwidth (BW), such as $2\times$ bandwidth with $4\times$ data rate as compared to data frequency within a die, or $1\times$ bandwidth with $2\times$ data rate as compared to data frequency within a die. Certain interconnects (e.g., and/or transceiver circuit(s)) herein include dynamic and rapid transitions between a first (e.g., $1\times$) bandwidth and second, different (e.g., $2\times$) bandwidth as two modes that conditionally provide the optimal choice of benefits in bandwidth performance versus benefits in power savings, reduced penalty in latency caused by additional clock crossings into low jitter clocking domain, and/or reducing the error rate that high performance transfers may have. Certain interconnects (e.g., and/or transceiver circuit(s)) herein provide for dynamic and rapid transitions between a first (e.g., $1\times$) bandwidth and a second, different (e.g., higher or lower) (e.g., $2\times$) bandwidth modes. Certain interconnects (e.g., and/or transceiver circuit(s)) herein include traffic flow control circuitry to halt traffic temporarily when transitioning, for example, when transitioning between clocking rates (e.g., $1\times$, $2\times$, $4\times$, etc.) and/or when transitioning between different operating frequencies (e.g., frequency rates).

[0177] Certain interconnects (e.g., and/or transceiver circuit(s)) herein provision for separate and independent tuning of receiver, transmitter, and/or clocking circuits for each bandwidth (e.g., clocking rate) and frequency mode on each instantiation and on each die, for example, so as to compensate for within-die and die-to-die process variations as well as temporal temperature and voltage supply variations. Certain interconnects (e.g., and/or transceiver circuit(s)) herein include a communication error detection mechanism (e.g., parity or similar) that allows for proper handling at the processor level (e.g., re-booting, etc.).

[0178] Certain embodiments herein provide for an electrical interconnect (e.g., and/or transceiver circuit(s)) that

has facilities for boot-time multi-point characterization sweeping across multiple variables for transmitter and receiver circuit parameters with storage for rapid parameter look-up during runtime changes, e.g., changes in clock frequency, voltage level, or clocking rates (e.g., $1\times$, $2\times$, $4\times$, etc.). Certain embodiments herein provide for an electrical interconnect (e.g., and/or transceiver circuit(s)) that provides for periodic refresh of stored transmitter and receiver circuit parameters re-characterization to recapture changed environment and circuit conditions. Certain embodiments of an electrical interconnect (e.g., and/or transceiver circuit(s)) herein provide for rapid processor clock, power, and/or data-rate transitions during critical runtime operations and apply the low running multi-point sweeping characterization and parameter recording, e.g., only during Boot time or periods of runtime that are not processor performance sensitive. Certain embodiments of an electrical interconnect (e.g., and/or transceiver circuit(s)) herein provide for die-to-die exchange that optimizes explicit state update (e.g., Rx DLL is locked, Tx PLL is locked, Tx duty cycle corrector (DCC) is locked, etc.) and/or reduces latency from assumption timers. Certain embodiments of an electrical interconnect (e.g., and/or transceiver circuit(s)) herein provide for after the multi-point sweeping characterization to be autonomous management within the interconnect circuitry, e.g., that does not need management from firmware, BIOS, and/or drivers.

[0179] In various embodiments, an integrated circuit such as a system on chip (SoC) or other multicore processor may be formed with an interconnection fabric that interconnects together processor cores and/or other intellectual property (IP) agents. Any IP blocks which couple directly to the interconnect fabric are generally referred to as “agents” or “IP agents”, or “fabric agents.” While different forms of this interconnection fabric are possible, in representative embodiments described herein a mesh interconnect is used to couple together the IP agents. Further, to ensure that agents located at a periphery of the design are accommodated with sufficient bandwidth for communication of messages, embodiments provide so-called “turn agents”, which may be implemented as buffer structures used to store and re-route messages intended for communication on a given direction of the mesh interconnect via another direction of the mesh interconnect. More particularly, a representative embodiment described herein provides such turn agents associated with mesh stops that couple one or more IP agents to the mesh interconnect.

[0180] In general, an integrated circuit may be configured such that all IP agents inject messages only via a single direction on the mesh interconnect (e.g., horizontally or vertically). In a particular implementation described herein, this configured direction is in the vertical direction. With turn agents associated with peripheral IP agents, these IP agents may inject messages in multiple directions, namely both vertical and horizontal directions, to enable improved bandwidth for these devices, which otherwise would suffer from limited bandwidth, as they would only be able to inject messages in a single way of this one (e.g., vertical) direction.

[0181] Referring now to FIG. 35, shown is a block diagram of an integrated circuit in accordance with an embodiment of the present invention. As shown in FIG. 35, integrated circuit 3500 is a given SoC that includes a plurality of intellectual property agents 3510_A-3510_F (generically “agents 3510” or “IP agents 3510”). Note that only a subset

of representative IP agents are shown, and in a given actual implementation, more agents may be present. In an embodiment, agents 3510 are coupled together via a mesh interconnect 3520. Agents 3510 and mesh interconnect 3520 may be formed on a single semiconductor die. However, in other cases such agents may span across multiple die implemented within a given IC package such as a multichip module. Nevertheless for purposes of discussion of representative embodiments, assume that agents 3510 shown in FIG. 35 are implemented, along with mesh interconnect 3520, on a single semiconductor die.

[0182] With further reference to FIG. 35, note that mesh interconnect 3520 itself is implemented of individual interconnects running in the horizontal and vertical directions. More specifically, interconnects 3520_{H1-H3} are provided in the horizontal direction and interconnects 3520_{V1-V4} are provided in the vertical direction. With this mesh interconnect arrangement, IP agents 3510 may communicate with each other. Understand that while a limited and representative number of horizontal and vertical interconnects are shown in FIG. 35, in different implementations a much larger number of such interconnects may form a mesh interconnect, particularly in embodiments of SoCs that may include a large number of cores or other IP agents, e.g., 32 or 64-core implementations.

[0183] As described above, in a conventional configuration of such a mesh interconnect, IP agents 3510 are typically configured to source messages onto mesh interconnect 3520 in a single one of the vertical and horizontal directions. This is so, as even though agents 3510 are provided connectivity to both the horizontal and vertical interconnects 3520_{H,V} of mesh interconnect 3520, in order to reduce design complexity, the injection of traffic onto mesh interconnect 3520 by IP agents 3510 may be limited to a particular direction in typical implementations. Without an embodiment and with a typical configuration, IP agents 3510 would be configured to only inject traffic along the vertical direction. This helps simplify injection logic routing tables associated with the traffic. Note that with this conventional arrangement, IP agents that are at a periphery of mesh interconnect 3520 (which in the implementation of FIG. 35 include IP agents 3510_A-3510_D) would have half the bandwidth capability of IP agents at an interior of mesh interconnect 3500, such as IP agents 3510_{E,F}. Thus as shown in FIG. 35, IP agents 3510_{E,F} can communicate packets vertically in both ways (i.e., north and south in the vertical direction), realizing, in a conventional arrangement, twice the bandwidth that could be realized by IP agents 3510_{A,D}.

[0184] Such limited bandwidth of at least peripheral IP agents 3510 could be significant when these peripheral or edge IP agents are high bandwidth agents. In typical SoC designs, IP agents on the edges tend to be agents that connect to external buses such as memory buses, cache coherent buses or IO buses. In addition, as technology advances, there is a continuous push to increase connectivity bandwidth due to multiple factors. These factors include increased network speeds. For example, the industry is enabling 200 Gb Ethernet cards today and is expected to transition to 400 Gb Ethernet cards in 2020. This will double the per IO agent bandwidth from 50 GB/s to 100 GB/s. In addition, many communication protocols seek to introduce higher speeds. For example, it is anticipated that Peripheral Component Interconnect Express (PCIe) Gen5 will run at 32 GT/s, and

result in bandwidths at 100+GB/s, also in the 2020 time-frame. In addition, as memory bandwidth of a platform continues to increase, coherent interconnect bandwidth may scale to keep the inter-socket bandwidth scaling proportionally.

[0185] As such, edge devices may be configured in accordance with embodiments to enable injection in multiple mesh interconnect directions to realize more injection bandwidth than is available due to their location. Although the scope of the present invention is not limited in this regard, in an embodiment with dual-direction injection from peripheral IP agents, bandwidths as high as 128 Gigabytes per second (GB/s) may be realized. Still further, techniques herein enable this higher bandwidth without increasing the operation frequency of the mesh interconnect (reducing power consumption and/or die area), and without providing additional stops to the agent, which could constrain design requirements.

[0186] As such in embodiments, IP agents 3510 coupled at a periphery of mesh interconnect 3520 may be configured to source messages in both horizontal and vertical directions. In different implementations, all such peripheral IP agents may be provided with this capability to source messages in both horizontal and vertical directions. In other cases, only one or some subset of peripheral IP agents may be configured for this dual-direction message sourcing.

[0187] To effect this ability to communicate messages in both horizontal and vertical directions on mesh interconnect 3520, turn agents may be provided in association with peripheral IP agents that are to be configured for dual direction sourcing. More particularly in embodiments herein, such turn agents may be included in or otherwise associated with mesh stops that are formed as connection points between horizontal and vertical interconnects of the mesh interconnect. In the high level illustrated in FIG. 35, a plurality of mesh stops 3525₀-3525_x are provided, each located in association with an intersection between a corresponding horizontal interconnect 3520_H and a corresponding vertical interconnect 3520_V. Understand while shown at this high level in the embodiment of FIG. 35, many variations and alternatives are possible.

[0188] Referring now to FIG. 36, shown is a block diagram of a portion of a SoC in accordance with another embodiment of the present invention. As shown in FIG. 36, a portion of an integrated circuit 3600 includes multiple agents 3610_A-3610_E that couple together via a mesh interconnect 3620 including multiple horizontal interconnects 3620_{H1-H2} and multiple vertical interconnects 3620_{V1-V5}. Note that in this limited view in FIG. 36, focus is on agent 3610_B, which is enabled, via inclusion of a turn agent in an associated mesh stop 3625_S, to inject messages in a horizontal direction. Thus as illustrated, via inclusion of a turn agent in mesh stop 3625_S, IP agent 3610_B injects packets or other messages into mesh interconnect 3620 at mesh stop 3625_S in the horizontal direction, and mesh stop 3625_T is configured to re-route this traffic to mesh stop 225_D, which in turn may couple to a destination IP agent (not shown for ease of illustration in FIG. 36).

[0189] Depending upon a desired configuration, note that mesh stops associated with all of agents 3610_A-3610_E may be configured with turn agents to enable these IP agents to source packets horizontally as well as vertically. It is also possible for a given SoC instantiation to independently and individually include turn agents for only a single one or

some subset of mesh stops associated with peripheral agents and not for others. In this way, some peripheral IP agents may be enabled to source messages in both horizontal and vertical directions of a mesh interconnect, while other peripheral agents may be configured to source messages in only a single one of vertical and horizontal directions. Understand while shown at this high level in the embodiment of FIG. 36, many variations and alternatives are possible.

[0190] Referring now to FIG. 37A, shown is a more detailed block diagram of a representative mesh stop including a turn agent in accordance with an embodiment of the present invention. As shown in FIG. 37A, a mesh stop 3700 couples between a horizontal mesh interconnect 3760_H and a vertical mesh interconnect 3760_V. While the details of a single mesh stop 3700 are shown in FIG. 37A, note that a portion of another mesh stop 3780 also is illustrated. Mesh stop 3780 is a conventional mesh stop not including a turn agent.

[0191] With reference to mesh stop 3700, incoming packets sourced by IP agents are received via input lines 3705_{0,1} and into a set of egress buffers 3708, via an age order matrix (AOT) 3709, a queue structure that records age information per entry and enforces first-in first-out order per a quality mask. From there, such messages are provided to a ring stop 3710, more specifically a vertical ring stop, which according to typical convention of the SoC design injects packets via vertical mesh interconnect 3760_V. In addition, to allow certain messages received within mesh stop 3700 from another mesh stop (and not a true source packet from an IP agent directly coupled to mesh stop 3700) to change direction at ring stop 3710, these messages instead proceed from ring stop 3710 to a transgress buffer 3715 and thereafter to another ring stop 3720, namely a horizontal ring stop, so that messages may be communicated via horizontal mesh interconnect 3760_H. In an embodiment, transgress buffer 3715 may include a plurality of entries, each to store messages on a path from ring stop 3710 to ring stop 3720. In embodiments, transgress buffer 3715 may be implemented as a first-in-first-out (FIFO) buffer including multiple entries to store such messages.

[0192] Furthermore, messages that are to be sunk to IP agents directly coupled to ring stop 3700 may proceed from ring stop 3720 via communication line 3725 to a selection circuit 3730, e.g., implemented as a multiplexer. When selection circuit 3730 is to direct messages to directly coupled agents, it is controlled to output such messages via a given one of output lines 3735₀-3735₁ to a given sink IP agent.

[0193] Still further with embodiments herein, to enable a turn to occur such that incoming source messages from a directly coupled IP agent can be re-routed to horizontal mesh interconnect 3760_H, selection circuit 3730 may be controlled to direct such messages to a turn agent 3740. In an embodiment, turn agent 3740 may include buffer circuitry, such as a FIFO buffer including a plurality of entries to buffer such messages and re-route them via communication through egress buffers 3708. In a particular embodiment, turn agent 3740 may include, e.g., 24 entries and can be implemented with multiple read and write ports. In addition, turn agent 3740 may include control circuitry to control operation of the buffer so that messages can be provided with appropriate information and sent along to an appropriate destination.

[0194] As further illustrated in FIG. 37A, control of where to direct given messages may proceed based on information stored in at least one lookup table 3750. Such lookup table may be implemented as a routing table that includes entries each associated with a given destination IP agent and which stores routing information. More specifically, using a destination identifier of a given message, lookup table 3750 may be accessed to determine a next destination for the message in its communication from a given source IP agent to a given destination IP agent. In a particular embodiment, lookup table 3750 may have a plurality of entries each including multiple fields including a next destination field to identify a next destination for the message, a turn agent field to indicate whether a packet is to be re-routed via an associated turn agent of the mesh stop, and a valid field to indicate whether, at the present time, the given entry is valid or not.

[0195] In an embodiment, routing tables as implemented within one or more lookup tables (per mesh stop) may be adapted to indicate that a static route for traffic between a given source IP agent and a destination IP agent is to be routed via a turn agent. With this routing information, traffic from this source IP agent is injected onto an interconnect mesh towards the turn agent. In an embodiment, certain design constraints may simplify implementation. In the example of FIG. 37A, an injecting IP agent injects data typically on the vertical direction and sinks incoming traffic on the horizontal direction. Transgress buffer 3715 enables traffic to be directed from the vertical mesh to horizontal mesh. In the case where a sender and receiver are on the same mesh stop, transgress buffer 3715 can be used to hold the packets that are for the co-located agent without performing a mesh injection in the vertical direction. To use turn agent 3740, the injecting IP agent can reuse transgress buffer 3715 and inject outgoing traffic into transgress buffer 3715, which then injects it onto horizontal mesh interconnect 3760_H towards turn agent 3740. In this way, a source agent (not shown in FIG. 37A) at mesh stop 3780 may send a packet on horizontal mesh interconnect 3760_H to ring stop 3700 and then proxy through turn agent 3740 to send on vertical mesh interconnect 3760_V. Note that turn agents may be incorporated into various mesh scheduling agents including those that arbitrate for credits. Understand while shown at this high level in the embodiment of FIG. 37A, many variations and alternatives are possible.

[0196] Referring now to FIG. 37B, shown is a block diagram of a method in accordance with another embodiment of the present invention. As shown in FIG. 37B, SoC 3790 includes a plurality of agents 3750_A-3750_E, located at a periphery of the SoC. While only these four agents are shown for ease of illustration, understand that SoC 3790 may include a plurality of other agents located throughout a mesh interconnect 3750 formed of multiple vertical and horizontal interconnects. Illustrated in the high level of FIG. 4 are a plurality of mesh stops 3755_A-3755_X, which as shown are located in a familiar row and column matrix. With embodiments herein there is no limitation as to having the same number of agents per row or the same number of agents per column. Thus as illustrated in FIG. 37B, agent 3750_E is the only agent present in its column. Or stated another way, a first row of SoC 400 (having IP agents 3750_{A-E}) includes at least one more agent than other rows of SoC 400. With an embodiment, a design limitation of having the same number of columns on the entire die equaling the maximum number of agents in a row can be removed. In this

way, relatively lower bandwidth agents may be located without a dedicated column for the entire die, which may save significant die costs. As shown in FIG. 37B, assume that IP agent 3750_E is a relatively lower bandwidth agent. In the arrangement of FIG. 37B, all traffic sourced from it may use turn agents on different columns (e.g., one of mesh stops 3755_{B-D}) to make its way to a given destination (e.g., an IP agent coupled to mesh stop 3755_D in FIG. 37B). Understand while shown at this high level in the embodiment of FIG. 37B, many variations and alternatives are possible.

Exemplary Core Architectures, Processors, and Computer Architectures

[0197] Processor cores may be implemented in different ways, for different purposes, and in different processors. For instance, implementations of such cores may include: 1) a general purpose in-order core intended for general-purpose computing; 2) a high performance general purpose out-of-order core intended for general-purpose computing; 3) a special purpose core intended primarily for graphics and/or scientific (throughput) computing. Implementations of different processors may include: 1) a CPU including one or more general purpose in-order cores intended for general-purpose computing and/or one or more general purpose out-of-order cores intended for general-purpose computing; and 2) a coprocessor including one or more special purpose cores intended primarily for graphics and/or scientific (throughput). Such different processors lead to different computer system architectures, which may include: 1) the coprocessor on a separate chip from the CPU; 2) the coprocessor on a separate die in the same package as a CPU; 3) the coprocessor on the same die as a CPU (in which case, such a coprocessor is sometimes referred to as special purpose logic, such as integrated graphics and/or scientific (throughput) logic, or as special purpose cores); and 4) a system on a chip that may include on the same die the described CPU (sometimes referred to as the application core(s) or application processor(s)), the above described coprocessor, and additional functionality. Exemplary core architectures are described next, followed by descriptions of exemplary processors and computer architectures.

Exemplary Core Architectures

In-Order and Out-of-Order Core Block Diagram

[0198] FIG. 38A is a block diagram illustrating both an exemplary in-order pipeline and an exemplary register renaming, out-of-order issue/execution pipeline according to embodiments of the disclosure. FIG. 38B is a block diagram illustrating both an exemplary embodiment of an in-order architecture core and an exemplary register renaming, out-of-order issue/execution architecture core to be included in a processor according to embodiments of the disclosure. The solid lined boxes in FIGS. 38A-B illustrate the in-order pipeline and in-order core, while the optional addition of the dashed lined boxes illustrates the register renaming, out-of-order issue/execution pipeline and core. Given that the in-order aspect is a subset of the out-of-order aspect, the out-of-order aspect will be described.

[0199] In FIG. 38A, a processor pipeline 3800 includes a fetch stage 3802, a length decode stage 3804, a decode stage 3806, an allocation stage 3808, a renaming stage 3810, a

scheduling (also known as a dispatch or issue) stage 3812, a register read/memory read stage 3814, an execute stage 3816, a write back/memory write stage 3818, an exception handling stage 3822, and a commit stage 3824.

[0200] FIG. 38B shows processor core 3890 including a front end unit 3830 coupled to an execution engine unit 3850, and both are coupled to a memory unit 3870. The core 3890 may be a reduced instruction set computing (RISC) core, a complex instruction set computing (CISC) core, a very long instruction word (VLIW) core, or a hybrid or alternative core type. As yet another option, the core 3890 may be a special-purpose core, such as, for example, a network or communication core, compression engine, coprocessor core, general purpose computing graphics processing unit (GPGPU) core, graphics core, or the like.

[0201] The front end unit 3830 includes a branch prediction unit 3832 coupled to an instruction cache unit 3834, which is coupled to an instruction translation lookaside buffer (TLB) 3836, which is coupled to an instruction fetch unit 3838, which is coupled to a decode unit 3840. The decode unit 3840 (or decoder or decoder unit) may decode instructions (e.g., macro-instructions), and generate as an output one or more micro-operations, micro-code entry points, micro-instructions, other instructions, or other control signals, which are decoded from, or which otherwise reflect, or are derived from, the original instructions. The decode unit 3840 may be implemented using various different mechanisms. Examples of suitable mechanisms include, but are not limited to, look-up tables, hardware implementations, programmable logic arrays (PLAs), microcode read only memories (ROMs), etc. In one embodiment, the core 3890 includes a microcode ROM or other medium that stores microcode for certain macroinstructions (e.g., in decode unit 3840 or otherwise within the front end unit 3830). The decode unit 3840 is coupled to a rename/allocator unit 3852 in the execution engine unit 3850.

[0202] The execution engine unit 3850 includes the rename/allocator unit 3852 coupled to a retirement unit 3854 and a set of one or more scheduler unit(s) 3856. The scheduler unit(s) 3856 represents any number of different schedulers, including reservations stations, central instruction window, etc. The scheduler unit(s) 3856 is coupled to the physical register file(s) unit(s) 3858. Each of the physical register file(s) units 3858 represents one or more physical register files, different ones of which store one or more different data types, such as scalar integer, scalar floating point, packed integer, packed floating point, vector integer, vector floating point, status (e.g., an instruction pointer that is the address of the next instruction to be executed), etc. In one embodiment, the physical register file(s) unit 3858 comprises a vector registers unit, a write mask registers unit, and a scalar registers unit. These register units may provide architectural vector registers, vector mask registers, and general purpose registers. The physical register file(s) unit(s) 3858 is overlapped by the retirement unit 3854 to illustrate various ways in which register renaming and out-of-order execution may be implemented (e.g., using a reorder buffer(s) and a retirement register file(s); using a future file(s), a history buffer(s), and a retirement register file(s); using a register maps and a pool of registers; etc.). The retirement unit 3854 and the physical register file(s) unit(s) 3858 are coupled to the execution cluster(s) 3860. The execution cluster(s) 3860 includes a set of one or more execution units 3862 and a set of one or more memory access units 3864.

The execution units **3862** may perform various operations (e.g., shifts, addition, subtraction, multiplication) and on various types of data (e.g., scalar floating point, packed integer, packed floating point, vector integer, vector floating point). While some embodiments may include a number of execution units dedicated to specific functions or sets of functions, other embodiments may include only one execution unit or multiple execution units that all perform all functions. The scheduler unit(s) **3856**, physical register file(s) unit(s) **3858**, and execution cluster(s) **3860** are shown as being possibly plural because certain embodiments create separate pipelines for certain types of data/operations (e.g., a scalar integer pipeline, a scalar floating point/packed integer/packed floating point/vector integer/vector floating point pipeline, and/or a memory access pipeline that each have their own scheduler unit, physical register file(s) unit, and/or execution cluster—and in the case of a separate memory access pipeline, certain embodiments are implemented in which only the execution cluster of this pipeline has the memory access unit(s) **3864**). It should also be understood that where separate pipelines are used, one or more of these pipelines may be out-of-order issue/execution and the rest in-order.

[0203] The set of memory access units **3864** is coupled to the memory unit **3870**, which includes a data TLB unit **3872** coupled to a data cache unit **3874** coupled to a level 2 (L2) cache unit **3876**. In one exemplary embodiment, the memory access units **3864** may include a load unit, a store address unit, and a store data unit, each of which is coupled to the data TLB unit **3872** in the memory unit **3870**. The instruction cache unit **3834** is further coupled to a level 2 (L2) cache unit **3876** in the memory unit **3870**. The L2 cache unit **3876** is coupled to one or more other levels of cache and eventually to a main memory.

[0204] By way of example, the exemplary register renaming, out-of-order issue/execution core architecture may implement the pipeline **3800** as follows: 1) the instruction fetch **3838** performs the fetch and length decoding stages **3802** and **3804**; 2) the decode unit **3840** performs the decode stage **3806**; 3) the rename/allocator unit **3852** performs the allocation stage **3808** and renaming stage **3810**; 4) the scheduler unit(s) **3856** performs the schedule stage **3812**; 5) the physical register file(s) unit(s) **3858** and the memory unit **3870** perform the register read/memory read stage **3814**; the execution cluster **3860** perform the execute stage **3816**; 6) the memory unit **3870** and the physical register file(s) unit(s) **3858** perform the write back/memory write stage **3818**; 7) various units may be involved in the exception handling stage **3822**; and 8) the retirement unit **3854** and the physical register file(s) unit(s) **3858** perform the commit stage **3824**.

[0205] The core **3890** may support one or more instructions sets (e.g., the x86 instruction set (with some extensions that have been added with newer versions); the MIPS instruction set of MIPS Technologies of Sunnyvale, CA; the ARM instruction set (with optional additional extensions such as NEON) of ARM Holdings of Sunnyvale, CA), including the instruction(s) described herein. In one embodiment, the core **3890** includes logic to support a packed data instruction set extension (e.g., AVX1, AVX2), thereby allowing the operations used by many multimedia applications to be performed using packed data.

[0206] It should be understood that the core may support multithreading (executing two or more parallel sets of operations or threads), and may do so in a variety of ways

including time sliced multithreading, simultaneous multithreading (where a single physical core provides a logical core for each of the threads that physical core is simultaneously multithreading), or a combination thereof (e.g., time sliced fetching and decoding and simultaneous multithreading thereafter such as in the Intel® Hyperthreading technology).

[0207] While register renaming is described in the context of out-of-order execution, it should be understood that register renaming may be used in an in-order architecture. While the illustrated embodiment of the processor also includes separate instruction and data cache units **3834/3874** and a shared L2 cache unit **3876**, alternative embodiments may have a single internal cache for both instructions and data, such as, for example, a Level 1 (L1) internal cache, or multiple levels of internal cache. In some embodiments, the system may include a combination of an internal cache and an external cache that is external to the core and/or the processor. Alternatively, all of the cache may be external to the core and/or the processor.

Specific Exemplary in-Order Core Architecture

[0208] FIGS. 39A-B illustrate a block diagram of a more specific exemplary in-order core architecture, which core would be one of several logic blocks (including other cores of the same type and/or different types) in a chip. The logic blocks communicate through a high-bandwidth interconnect network (e.g., a ring network) with some fixed function logic, memory I/O interfaces, and other necessary I/O logic, depending on the application.

[0209] FIG. 39A is a block diagram of a single processor core, along with its connection to the on-die interconnect network **3902** and with its local subset of the Level 2 (L2) cache **3904**, according to embodiments of the disclosure. In one embodiment, an instruction decode unit **3900** supports the x86 instruction set with a packed data instruction set extension. An L1 cache **3906** allows low-latency accesses to cache memory into the scalar and vector units. While in one embodiment (to simplify the design), a scalar unit **3908** and a vector unit **3910** use separate register sets (respectively, scalar registers **3912** and vector registers **3914**) and data transferred between them is written to memory and then read back in from a level 1 (L1) cache **3906**, alternative embodiments of the disclosure may use a different approach (e.g., use a single register set or include a communication path that allow data to be transferred between the two register files without being written and read back).

[0210] The local subset of the L2 cache **3904** is part of a global L2 cache that is divided into separate local subsets, one per processor core. Each processor core has a direct access path to its own local subset of the L2 cache **3904**. Data read by a processor core is stored in its L2 cache subset **3904** and can be accessed quickly, in parallel with other processor cores accessing their own local L2 cache subsets. Data written by a processor core is stored in its own L2 cache subset **3904** and is flushed from other subsets, if necessary. The ring network ensures coherency for shared data. The ring network is bi-directional to allow agents such as processor cores, L2 caches and other logic blocks to communicate with each other within the chip. Each ring data-path is 1012-bits wide per direction.

[0211] FIG. 39B is an expanded view of part of the processor core in FIG. 39A according to embodiments of the disclosure. FIG. 39B includes an L1 data cache **3906A** part of the L1 cache **3904**, as well as more detail regarding the

vector unit **3910** and the vector registers **3914**. Specifically, the vector unit **3910** is a 16-wide vector processing unit (VPU) (see the 16-wide ALU **3928**), which executes one or more of integer, single-precision float, and double-precision float instructions. The VPU supports swizzling the register inputs with swizzle unit **3920**, numeric conversion with numeric convert units **3922A-B**, and replication with replication unit **3924** on the memory input. Write mask registers **3926** allow predicating resulting vector writes.

[0212] FIG. 40 is a block diagram of a processor **4000** that may have more than one core, may have an integrated memory controller, and may have integrated graphics according to embodiments of the disclosure. The solid lined boxes in FIG. 40 illustrate a processor **4000** with a single core **4002A**, a system agent **4010**, a set of one or more bus controller units **4016**, while the optional addition of the dashed lined boxes illustrates an alternative processor **4000** with multiple cores **4002A-N**, a set of one or more integrated memory controller unit(s) **4014** in the system agent unit **4010**, and special purpose logic **4008**.

[0213] Thus, different implementations of the processor **4000** may include: 1) a CPU with the special purpose logic **4008** being integrated graphics and/or scientific (throughput) logic (which may include one or more cores), and the cores **4002A-N** being one or more general purpose cores (e.g., general purpose in-order cores, general purpose out-of-order cores, a combination of the two); 2) a coprocessor with the cores **4002A-N** being a large number of special purpose cores intended primarily for graphics and/or scientific (throughput); and 3) a coprocessor with the cores **4002A-N** being a large number of general purpose in-order cores. Thus, the processor **4000** may be a general-purpose processor, coprocessor or special-purpose processor, such as, for example, a network or communication processor, compression engine, graphics processor, GPGPU (general purpose graphics processing unit), a high-throughput many integrated core (MIC) coprocessor (including 30 or more cores), embedded processor, or the like. The processor may be implemented on one or more chips. The processor **4000** may be a part of and/or may be implemented on one or more substrates using any of a number of process technologies, such as, for example, BiCMOS, CMOS, or NMOS.

[0214] The memory hierarchy includes one or more levels of cache within the cores, a set or one or more shared cache units **4006**, and external memory (not shown) coupled to the set of integrated memory controller units **4014**. The set of shared cache units **4006** may include one or more mid-level caches, such as level 2 (L2), level 3 (L3), level 4 (L4), or other levels of cache, a last level cache (LLC), and/or combinations thereof. While in one embodiment a ring based interconnect unit **4012** interconnects the integrated graphics logic **4008**, the set of shared cache units **4006**, and the system agent unit **4010**/integrated memory controller unit(s) **4014**, alternative embodiments may use any number of well-known techniques for interconnecting such units. In one embodiment, coherency is maintained between one or more cache units **4006** and cores **4002A-N**.

[0215] In some embodiments, one or more of the cores **4002A-N** are capable of multi-threading. The system agent **4010** includes those components coordinating and operating cores **4002A-N**. The system agent unit **4010** may include for example a power control unit (PCU) and a display unit. The PCU may be or include logic and components needed for regulating the power state of the cores **4002A-N** and the

integrated graphics logic **4008**. The display unit is for driving one or more externally connected displays.

[0216] The cores **4002A-N** may be homogenous or heterogeneous in terms of architecture instruction set; that is, two or more of the cores **4002A-N** may be capable of execution the same instruction set, while others may be capable of executing only a subset of that instruction set or a different instruction set.

Exemplary Computer Architectures

[0217] FIGS. 41-44 are block diagrams of exemplary computer architectures. Other system designs and configurations known in the arts for laptops, desktops, handheld PCs, personal digital assistants, engineering workstations, servers, network devices, network hubs, switches, embedded processors, digital signal processors (DSPs), graphics devices, video game devices, set-top boxes, micro controllers, cell phones, portable media players, hand held devices, and various other electronic devices, are also suitable. In general, a huge variety of systems or electronic devices capable of incorporating a processor and/or other execution logic as disclosed herein are generally suitable.

[0218] Referring now to FIG. 41, shown is a block diagram of a system **4100** in accordance with one embodiment of the present disclosure. The system **4100** may include one or more processors **4110**, **4115**, which are coupled to a controller hub **4120**. In one embodiment the controller hub **4120** includes a graphics memory controller hub (GMCH) **4190** and an Input/Output Hub (IOH) **4150** (which may be on separate chips); the GMCH **4190** includes memory and graphics controllers to which are coupled memory **4140** and a coprocessor **4145**; the IOH **4150** is couples input/output (I/O) devices **4160** to the GMCH **4190**. Alternatively, one or both of the memory and graphics controllers are integrated within the processor (as described herein), the memory **4140** and the coprocessor **4145** are coupled directly to the processor **4110**, and the controller hub **4120** in a single chip with the IOH **4150**. Memory **4140** may include a cache coherency and/or interconnect management module **4140A**, for example, to store code that when executed causes a processor to perform any method of this disclosure.

[0219] The optional nature of additional processors **4115** is denoted in FIG. 41 with broken lines. Each processor **4110**, **4115** may include one or more of the processing cores described herein and may be some version of the processor **4000**.

[0220] The memory **4140** may be, for example, dynamic random access memory (DRAM), phase change memory (PCM), or a combination of the two. For at least one embodiment, the controller hub **4120** communicates with the processor(s) **4110**, **4115** via a multi-drop bus, such as a frontside bus (FSB), point-to-point interface such as Quick-Path Interconnect (QPI), or similar connection **4195**.

[0221] In one embodiment, the coprocessor **4145** is a special-purpose processor, such as, for example, a high-throughput MIC processor, a network or communication processor, compression engine, graphics processor, GPGPU, embedded processor, or the like. In one embodiment, controller hub **4120** may include an integrated graphics accelerator.

[0222] There can be a variety of differences between the physical resources **4110**, **4115** in terms of a spectrum of metrics of merit including architectural, microarchitectural, thermal, power consumption characteristics, and the like.

[0223] In one embodiment, the processor 4110 executes instructions that control data processing operations of a general type. Embedded within the instructions may be coprocessor instructions. The processor 4110 recognizes these coprocessor instructions as being of a type that should be executed by the attached coprocessor 4145. Accordingly, the processor 4110 issues these coprocessor instructions (or control signals representing coprocessor instructions) on a coprocessor bus or other interconnect, to coprocessor 4145. Coprocessor(s) 4145 accept and execute the received coprocessor instructions.

[0224] Referring now to FIG. 42, shown is a block diagram of a first more specific exemplary system 4200 in accordance with an embodiment of the present disclosure. As shown in FIG. 42, multiprocessor system 4200 is a point-to-point interconnect system, and includes a first processor 4270 and a second processor 4280 coupled via a point-to-point interconnect 4250. Each of processors 4270 and 4280 may be some version of the processor 4000. In one embodiment of the disclosure, processors 4270 and 4280 are respectively processors 4010 and 4015, while coprocessor 4238 is coprocessor 4045. In another embodiment, processors 4270 and 4280 are respectively processor 4010 coprocessor 4045.

[0225] Processors 4270 and 4280 are shown including integrated memory controller (IMC) units 4272 and 4282, respectively. Processor 4270 also includes as part of its bus controller units point-to-point (P-P) interfaces 4276 and 4278; similarly, second processor 4280 includes P-P interfaces 4286 and 4288. Processors 4270, 4280 may exchange information via a point-to-point (P-P) interface 4250 using P-P interface circuits 4278, 4288. As shown in FIG. 42, IMCs 4272 and 4282 couple the processors to respective memories, namely a memory 4232 and a memory 4234, which may be portions of main memory locally attached to the respective processors.

[0226] Processors 4270, 4280 may each exchange information with a chipset 4290 via individual P-P interfaces 4252, 4254 using point to point interface circuits 4276, 4294, 4286, 4298. Chipset 4290 may optionally exchange information with the coprocessor 4238 via a high-performance interface 4239. In one embodiment, the coprocessor 4238 is a special-purpose processor, such as, for example, a high-throughput MIC processor, a network or communication processor, compression engine, graphics processor, GPGPU, embedded processor, or the like.

[0227] A shared cache (not shown) may be included in either processor or outside of both processors, yet connected with the processors via P-P interconnect, such that either or both processors' local cache information may be stored in the shared cache if a processor is placed into a low power mode.

[0228] Chipset 4290 may be coupled to a first bus 4216 via an interface 4296. In one embodiment, first bus 4216 may be a Peripheral Component Interconnect (PCI) bus, or a bus such as a PCI Express bus or another third generation I/O interconnect bus, although the scope of the present disclosure is not so limited.

[0229] As shown in FIG. 42, various I/O devices 4214 may be coupled to first bus 4216, along with a bus bridge 4218 which couples first bus 4216 to a second bus 4220. In one embodiment, one or more additional processor(s) 4215, such as coprocessors, high-throughput MIC processors, GPGPU's, accelerators (such as, e.g., graphics accelerators

or digital signal processing (DSP) units), field programmable gate arrays, or any other processor, are coupled to first bus 4216. In one embodiment, second bus 4220 may be a low pin count (LPC) bus. Various devices may be coupled to a second bus 4220 including, for example, a keyboard and/or mouse 4222, communication devices 4227 and a storage unit 4228 such as a disk drive or other mass storage device which may include instructions/code and data 4230, in one embodiment. Further, an audio I/O 4224 may be coupled to the second bus 4220. Note that other architectures are possible. For example, instead of the point-to-point architecture of FIG. 42, a system may implement a multi-drop bus or other such architecture.

[0230] Referring now to FIG. 43, shown is a block diagram of a second more specific exemplary system 4300 in accordance with an embodiment of the present disclosure. Like elements in FIGS. 42 and 43 bear like reference numerals, and certain aspects of FIG. 42 have been omitted from FIG. 43 in order to avoid obscuring other aspects of FIG. 43.

[0231] FIG. 43 illustrates that the processors 4270, 4280 may include integrated memory and I/O control logic ("CL") 4272 and 4282, respectively. Thus, the CL 4272, 4282 include integrated memory controller units and include I/O control logic. FIG. 43 illustrates that not only are the memories 4232, 4234 coupled to the CL 4272, 4282, but also that I/O devices 4314 are also coupled to the control logic 4272, 4282. Legacy I/O devices 4315 are coupled to the chipset 4290.

[0232] Referring now to FIG. 44, shown is a block diagram of a SoC 4400 in accordance with an embodiment of the present disclosure. Similar elements in FIG. 40 bear like reference numerals. Also, dashed lined boxes are optional features on more advanced SoCs. In FIG. 44, an interconnect unit(s) 4402 is coupled to: an application processor 4410 which includes a set of one or more cores 202A-N and shared cache unit(s) 4006; a system agent unit 4010; a bus controller unit(s) 4016; an integrated memory controller unit(s) 4014; a set of one or more coprocessors 4420 which may include integrated graphics logic, an image processor, an audio processor, and a video processor; an static random access memory (SRAM) unit 4430; a direct memory access (DMA) unit 4432; and a display unit 4440 for coupling to one or more external displays. In one embodiment, the coprocessor(s) 4420 include a special-purpose processor, such as, for example, a network or communication processor, compression engine, GPGPU, a high-throughput MIC processor, embedded processor, or the like.

[0233] In one embodiment, the processor 4110 executes instructions that control data processing operations of a general type. Embedded within the instructions may be coprocessor instructions. The processor 4110 recognizes these coprocessor instructions as being of a type that should be executed by the attached coprocessor 4145. Accordingly, the processor 4110 issues these coprocessor instructions (or control signals representing coprocessor instructions) on a coprocessor bus or other interconnect, to coprocessor 4145. Coprocessor(s) 4145 accept and execute the received coprocessor instructions.

[0234] Referring now to FIG. 42, shown is a block diagram of a first more specific exemplary system 4200 in accordance with an embodiment of the present disclosure. As shown in FIG. 42, multiprocessor system 4200 is a point-to-point interconnect system, and includes a first processor

cessor **4270** and a second processor **4280** coupled via a point-to-point interconnect **4250**. Each of processors **4270** and **4280** may be some version of the processor **4000**. In one embodiment of the disclosure, processors **4270** and **4280** are respectively processors **4010** and **4015**, while coprocessor **4238** is coprocessor **4045**. In another embodiment, processors **4270** and **4280** are respectively processor **4010** coprocessor **4045**.

[0235] Processors **4270** and **4280** are shown including integrated memory controller (IMC) units **4272** and **4282**, respectively. Processor **4270** also includes as part of its bus controller units point-to-point (P-P) interfaces **4276** and **4278**; similarly, second processor **4280** includes P-P interfaces **4286** and **4288**. Processors **4270**, **4280** may exchange information via a point-to-point (P-P) interface **4250** using P-P interface circuits **4278**, **4288**. As shown in FIG. **42**, IMCs **4272** and **4282** couple the processors to respective memories, namely a memory **4232** and a memory **4234**, which may be portions of main memory locally attached to the respective processors.

[0236] Processors **4270**, **4280** may each exchange information with a chipset **4290** via individual P-P interfaces **4252**, **4254** using point to point interface circuits **4276**, **4294**, **4286**, **4298**. Chipset **4290** may optionally exchange information with the coprocessor **4238** via a high-performance interface **4239**. In one embodiment, the coprocessor **4238** is a special-purpose processor, such as, for example, a high-throughput MIC processor, a network or communication processor, compression engine, graphics processor, GPGPU, embedded processor, or the like.

[0237] A shared cache (not shown) may be included in either processor or outside of both processors, yet connected with the processors via P-P interconnect, such that either or both processors' local cache information may be stored in the shared cache if a processor is placed into a low power mode.

[0238] Chipset **4290** may be coupled to a first bus **4216** via an interface **4296**. In one embodiment, first bus **4216** may be a Peripheral Component Interconnect (PCI) bus, or a bus such as a PCI Express bus or another third generation I/O interconnect bus, although the scope of the present disclosure is not so limited.

[0239] As shown in FIG. **42**, various I/O devices **4214** may be coupled to first bus **4216**, along with a bus bridge **4218** which couples first bus **4216** to a second bus **4220**. In one embodiment, one or more additional processor(s) **4215**, such as coprocessors, high-throughput MIC processors, GPGPU's, accelerators (such as, e.g., graphics accelerators or digital signal processing (DSP) units), field programmable gate arrays, or any other processor, are coupled to first bus **4216**. In one embodiment, second bus **4220** may be a low pin count (LPC) bus. Various devices may be coupled to a second bus **4220** including, for example, a keyboard and/or mouse **4222**, communication devices **4227** and a storage unit **4228** such as a disk drive or other mass storage device which may include instructions/code and data **4230**, in one embodiment. Further, an audio I/O **4224** may be coupled to the second bus **4220**. Note that other architectures are possible. For example, instead of the point-to-point architecture of FIG. **42**, a system may implement a multi-drop bus or other such architecture.

[0240] Referring now to FIG. **43**, shown is a block diagram of a second more specific exemplary system **4300** in accordance with an embodiment of the present disclosure.

Like elements in FIGS. **42** and **43** bear like reference numerals, and certain aspects of FIG. **42** have been omitted from FIG. **43** in order to avoid obscuring other aspects of FIG. **43**.

[0241] FIG. **43** illustrates that the processors **4270**, **4280** may include integrated memory and I/O control logic ("CL") **4272** and **4282**, respectively. Thus, the CL **4272**, **4282** include integrated memory controller units and include I/O control logic. FIG. **43** illustrates that not only are the memories **4232**, **4234** coupled to the CL **4272**, **4282**, but also that I/O devices **4314** are also coupled to the control logic **4272**, **4282**. Legacy I/O devices **4315** are coupled to the chipset **4290**.

[0242] Referring now to FIG. **44**, shown is a block diagram of a SoC **4400** in accordance with an embodiment of the present disclosure. Similar elements in FIG. **40** bear like reference numerals. Also, dashed lined boxes are optional features on more advanced SoCs. In FIG. **44**, an interconnect unit(s) **4402** is coupled to: an application processor **4410** which includes a set of one or more cores **202A-N** and shared cache unit(s) **4006**; a system agent unit **4010**; a bus controller unit(s) **4016**; an integrated memory controller unit(s) **4014**; a set or one or more coprocessors **4420** which may include integrated graphics logic, an image processor, an audio processor, and a video processor; an static random access memory (SRAM) unit **4430**; a direct memory access (DMA) unit **4432**; and a display unit **4440** for coupling to one or more external displays. In one embodiment, the coprocessor(s) **4420** include a special-purpose processor, such as, for example, a network or communication processor, compression engine, GPGPU, a high-throughput MIC processor, embedded processor, or the like.

[0243] Embodiments (e.g., of the mechanisms) disclosed herein may be implemented in hardware, software, firmware, or a combination of such implementation approaches. Embodiments of the disclosure may be implemented as computer programs or program code executing on programmable systems comprising at least one processor, a storage system (including volatile and non-volatile memory and/or storage elements), at least one input device, and at least one output device.

[0244] Program code, such as code **4230** illustrated in FIG. **42**, may be applied to input instructions to perform the functions described herein and generate output information. The output information may be applied to one or more output devices, in known fashion. For purposes of this application, a processing system includes any system that has a processor, such as, for example; a digital signal processor (DSP), a microcontroller, an application specific integrated circuit (ASIC), or a microprocessor.

[0245] The program code may be implemented in a high level procedural or object oriented programming language to communicate with a processing system. The program code may also be implemented in assembly or machine language, if desired. In fact, the mechanisms described herein are not limited in scope to any particular programming language. In any case, the language may be a compiled or interpreted language.

[0246] One or more aspects of at least one embodiment may be implemented by representative instructions stored on a machine-readable medium which represents various logic within the processor, which when read by a machine causes the machine to fabricate logic to perform the techniques described herein. Such representations, known as "IP cores"

may be stored on a tangible, machine readable medium and supplied to various customers or manufacturing facilities to load into the fabrication machines that actually make the logic or processor.

[0247] Such machine-readable storage media may include, without limitation, non-transitory, tangible arrangements of articles manufactured or formed by a machine or device, including storage media such as hard disks, any other type of disk including floppy disks, optical disks, compact disk read-only memories (CD-ROMs), compact disk rewritables (CD-RWs), and magneto-optical disks, semiconductor devices such as read-only memories (ROMs), random access memories (RAMs) such as dynamic random access memories (DRAMs), static random access memories (SRAMs), erasable programmable read-only memories (EPROMs), flash memories, electrically erasable programmable read-only memories (EEPROMs), phase change memory (PCM), magnetic or optical cards, or any other type of media suitable for storing electronic instructions.

[0248] Accordingly, embodiments of the disclosure also include non-transitory, tangible machine-readable media containing instructions or containing design data, such as Hardware Description Language (HDL), which defines structures, circuits, apparatuses, processors and/or system features described herein. Such embodiments may also be referred to as program products.

Emulation (Including Binary Translation, Code Morphing, Etc.)

[0249] In some cases, an instruction converter may be used to convert an instruction from a source instruction set to a target instruction set. For example, the instruction converter may translate (e.g., using static binary translation, dynamic binary translation including dynamic compilation), morph, emulate, or otherwise convert an instruction to one or more other instructions to be processed by the core. The instruction converter may be implemented in software, hardware, firmware, or a combination thereof. The instruction converter may be on processor, off processor, or part on and part off processor.

[0250] FIG. 45 is a block diagram contrasting the use of a software instruction converter to convert binary instructions in a source instruction set to binary instructions in a target instruction set according to embodiments of the disclosure. In the illustrated embodiment, the instruction converter is a software instruction converter, although alternatively the instruction converter may be implemented in software, firmware, hardware, or various combinations thereof. FIG. 45 shows a program in a high level language 4502 may be compiled using an x86 compiler 4504 to generate x86 binary code 4506 that may be natively executed by a processor with at least one x86 instruction set core 4516. The processor with at least one x86 instruction set core 4516 represents any processor that can perform substantially the same functions as an Intel® processor with at least one x86 instruction set core by compatibly executing or otherwise processing (1) a substantial portion of the instruction set of the Intel® x86 instruction set core or (2) object code versions of applications or other software targeted to run on an Intel® processor with at least one x86 instruction set core, in order to achieve substantially the same result as an Intel® processor with at least one x86 instruction set core. The x86 compiler 4504 represents a compiler that is operable to generate x86 binary

code 4506 (e.g., object code) that can, with or without additional linkage processing, be executed on the processor with at least one x86 instruction set core 4516. Similarly, FIG. 45 shows the program in the high level language 4502 may be compiled using an alternative instruction set compiler 4508 to generate alternative instruction set binary code 4510 that may be natively executed by a processor without at least one x86 instruction set core 4514 (e.g., a processor with cores that execute the MIPS instruction set of MIPS Technologies of Sunnyvale, CA and/or that execute the ARM instruction set of ARM Holdings of Sunnyvale, CA). The instruction converter 4512 is used to convert the x86 binary code 4506 into code that may be natively executed by the processor without an x86 instruction set core 4514. This converted code is not likely to be the same as the alternative instruction set binary code 4510 because an instruction converter capable of this is difficult to make; however, the converted code will accomplish the general operation and be made up of instructions from the alternative instruction set. Thus, the instruction converter 4512 represents software, firmware, hardware, or a combination thereof that, through emulation, simulation or any other process, allows a processor or other electronic device that does not have an x86 instruction set processor or core to execute the x86 binary code 4506.

[0251] Certain embodiments provide for the cohesive flow of individual dies in wafers into packaged modular die products. Additionally, these embodiments provide for modularity and extensibility of tiling several modular dies (e.g., heterogeneous modular dies) and further provide for interconnections between the dies over an interconnect mesh or fabric.

[0252] Different embodiments achieve connectivity between these dies in different ways. For example, in 2.5D packaging solutions, a silicon interposer and through-substrate vias (TSVs) connect dies at silicon interconnect speed in a minimal footprint. In another example, a bridge die may be used. For example, an Embedded Multi-Die Interconnect Bridge (EMIB) is a silicon bridge embedded under the edges of two interconnecting dies facilitates electrical coupling between them. In a three-dimensional (3D) architecture, the dies are stacked one above the other, creating a smaller footprint overall. Typically, the electrical connectivity and mechanical coupling in such 3D architecture is achieved using TSVs and high pitch solder-based bumps (e.g., C4 interconnections). The EMIB and the 3D stacked architecture may also be combined using an omni-directional interconnect (ODI), which allows for top-packaged chips to communicate with other chips horizontally using EMIB and vertically, using Through-Mold Vias (TMVs) which are typically larger than TSVs.

[0253] However, as the number of individual IC dies integrated onto a single microprocessor or other such system-in-package increases, the footprint available on a fixed-size package substrate for interconnecting these IC dies becomes challenging. To help alleviate the footprint challenge, IC dies may be sized to be uniform and arranged in a grid pattern in a tiled compute architecture. This tiling allows adding more core complex IC dies or replacing the input/output (IO) dies to fit different products. As used herein, the terms “core complex,” and “core” are used interchangeably to refer to a circuit comprising a reusable unit of logic, cell, or IC layout design with a particular functionality and defined interface, which serves as a build-

ing block in an IC chip design. For example, cores may comprise a set of memory registers, arithmetic logic unit (ALU), power converters, high-speed I/O interfaces, peripherals, programmable microprocessors, micro-controllers, digital signal processors, analog-digital mixed-signal processing blocks, configurable computing architectures, etc. A smaller core (e.g., computing core) may be combined with other smaller cores (e.g., memory) to form a larger core. For example, a core may comprise a computing core coupled to IO circuits that bring data into and out of the computing core, a power delivery circuit to deliver power to the computing core and aggregated or disaggregated memory banks that function as cache for the computing core. A plurality of such cores may be referred to as a core complex, although they may also be simply called cores. As computing cores typically require additional components to create a fully functional chip or a SOC, these complementary components are assumed to be inherent, either coupled directly to the cores in question or by way of other cores or circuit blocks (e.g., portions, i.e., “blocks” of circuits), in the microelectronic assembly of the various embodiments disclosed herein.

[0254] On the electrical and logic protocol side, this is accommodated through standardizing the die-to-die interfaces to accommodate connecting different dies together. In some scenarios, e.g., when moving core or IO dies to different process nodes or different manufacturers, the die sizes end up becoming a bit larger or smaller. This can be accommodated in standard organic packages where the die-to-die routing density is not very high and having matching die edge sizes is not a major requirement. However, this is very challenging to accommodate in 3D ICs with fixed (e.g., silicon) interposer or EMIB sizes/widths and tight channel specifications.

Interconnect Architecture Enabling Path Diversity for Strongly Ordered Messages

[0255] Mesh interconnect topologies for modern system-on-chip (SoC) processors connect to different core agents, caching agents, memory controllers, input-output (IO) agents and Ultra Path Interconnect (UPI) agents. While these mesh interconnect implementations perform fixed routing (e.g., following Y-X routing schemes), more than one route can be enabled with internal mesh routers and using agent devices.

[0256] FIG. 46 illustrates an example arrangement of UPI agents 4601A-D and IO agents 4611-4618 coupled to a mesh interconnect fabric 4620 spanning multiple interconnected IO dies. For example, the agents 4601A-B, 4611-4614 shown at the top edge of the interconnect fabric 4620 may be integral to or coupled to a first IO die (comprising a first portion of the mesh interconnect fabric 4620) and the agents 4601A-B, 4611-4614 shown at the bottom edge are integral to a second IO die (comprising a second portion of the mesh interconnect fabric 4620). Note that the terms “mesh interconnect fabric”, “mesh fabric”, “interconnect fabric”, “mesh interconnect”, and “fabric” are used interchangeably herein.

[0257] The various agents 4601A-D, 4611-4618 perform ordered transactions over the mesh interconnect fabric 4620. The ordered transactions can include (but are not limited to) non-coherent peer-to-peer (P2P) transactions and coherent transactions. This communication can be between different agents in the same socket (“local transactions”) or across sockets (“remote transactions”) that require routing through

UPI links via UPI agents 4601A-D. While UPI is used here as one example of a high speed interconnect protocol, various other interconnect protocols may be used (e.g., PCIe, CXL, Infinity Fabric links, etc.). With disaggregated IO dies present on different edges of the interconnect fabric 4620, the local transactions can take place within one IO die (“in-sector transactions”) or across multiple IO dies (“cross-sector transactions”). For example, FIG. 46 highlights a local in-sector connection 4630 between IO agent 4611 and IO agent 4613 (i.e., across one IO die in the same socket); a local cross-sector connection 4633 between IO agent 4611 and IO agent 4617 (i.e., across multiple IO dies); a remote P2P connection 4631 between IO die 4614 and the UPI agent 4601B (which connects to another socket); and a remote cross-sector connection 4632 between IO die 4611 and UPI agent 4601C (which traverses multiple IO dies and connects via the UPI agent 4601C to another socket).

[0258] As PCIe bandwidths scale (e.g., 14 GB/s for Gen3, 30 GB/s for Gen4, 60 GB/s for Gen5, and 120 GB/s for Gen6), the bandwidth requirements for certain types of transactions, such as P2P transactions, are increasing faster than the silicon frequency of the interconnect. These increased bandwidth requirements and the ordered, point-to-point nature of P2P traffic will ultimately result in severe contention with coherent traffic. In-sector P2P communication between IO agents can consume the entire single mesh row bandwidth for P2P traffic, stealing row bandwidth from the main band traffic between caching/home agents. This problem becomes even more significant for cross-sector P2P traffic where, for example, a single P2P stream between a pair of IO agents on different mesh edges can consume the entire column bandwidth, leaving little headroom for the core traffic. Alternatively, routing a dedicated fabric for P2P streams leads to varying degrees of area, complexity, and cost trade-offs for the overall SoC.

[0259] Embodiments of the invention address this problem with a specialized scatter-gather agent for various types of traffic, including P2P traffic, which implements a scatter and gather approach for cross sector local/remote traffic, utilizing the existing mesh interconnect fabric and addressing the main band bandwidth concerns. In some embodiments, the scatter-gather agent also includes logic to fulfill traffic ordering requirements. These embodiments achieve efficient utilization of the existing mesh interconnect 4620 at relatively low area cost and can be used for in-sector communication as well as cross-sector communication if the mesh fabric 4620 is used. However, in some embodiments, a separate dedicated fabric is used for in-sector communication.

[0260] Referring to FIG. 47, scatter-gather agents 4701-4702 are implemented to route traffic across the mesh fabric 4720 using gather and scatter style transactions while ensuring correct packet ordering as described herein. For example, scatter-gather agent 4701 is illustrated performing a “scatter” which distributes packets received from in-sector agents 4601A-B, 4611-4614 across columns 4677 of the mesh interconnect fabric 4620. Conversely, scatter-gather agent 4702 is shown performing a “gather” operation which combines the P2P packets received from the various columns 4677 and routes the gathered P2P packets to its in-sector agents 4601C-D, 4615-4618. Although illustrated in one direction in FIG. 47, the scatter-gather agents 4701-4702 are bi-directional (i.e., scatter-gather agent 4702 may

perform a scatter operation and scatter-gather agent 4701 may perform corresponding gather operations to combine and route the packets).

[0261] In some embodiments, the scatter-gather agents 4701-4702 are statically configured/programmed with a traffic distribution algorithm (e.g., via firmware during a system boot) which distributes mesh traffic equally across the various columns 4677 of the mesh fabric 4620 (e.g., via round robin or similar traffic distribution protocol). The ability to distribute packets across columns 4677 of the mesh fabric 4620 allows the scatter-gather agents 4701-4702 to manage bandwidth utilization across the mesh fabric 4620 more precisely and at a finer granularity than in prior implementations. In other embodiments, the traffic distribution algorithm may operate dynamically based on detected conditions on the mesh fabric 4620 (e.g., based on feedback indicating current traffic conditions in the columns 4677). For example, if a particular set of columns 4677 are heavily loaded (e.g., the corresponding buffers are filled beyond a threshold), the scatter-gather agents may choose a different set of columns or a different route for transmitting the packets.

[0262] In some embodiments, the scatter and gather circuitry of each scatter-gather agent 4701-4702 implements logic to distribute traffic to fabric routers 4678A-B of the interconnect fabric 4620, each of which is configured to route traffic over a corresponding column of the mesh fabric 4720. In FIG. 47, for example, the fabric routers 4640A at the top of the mesh fabric 4720 change the direction of traffic from a horizontal direction to a vertical direction, through a particular column 4677 of the mesh fabric 4701. Thus, to route a packet down a particular column, the scatter-gather agent 4701 identifies a corresponding fabric router 4678A in the packet header. The physical scatter-gather agent operating as a destination on the mesh fabric 4720 (e.g., scatter-gather agent 4702) is associated with multiple logical destination IDs corresponding to the destination fabric routers 4678A-B through which packets can be routed. The fabric routers 4678A-B may include buffer structures to temporarily buffer packets in transmission and routing logic to route and re-route packets on the interconnect fabric as needed. In one example, fabric routers 4678A-B may be associated with the converged/common mesh stops (CMSs) routers 4870A-D (described below with respect to FIG. 48) that couple the scatter-gather agents 4701-4702 to the interconnect fabric 4620. In some implementations, the fabric routers are turn agents described above with respect to FIGS. 37A-B. Note, however, that the underlying principles of the invention are not limited to this particular configuration.

[0263] In this embodiment, the UPI agents 4601A-D and IO agents 4611-4618 are coupled to separate dedicated mesh fabrics, 4720 and 4721, via a new set of fabric interfaces, 4740A and 4740B, respectively. The separate dedicated mesh fabrics, 4720 and 4721, are coupled to the scatter-gather agents, 4701 and 4702, respectively, to route transactions over the mesh fabric 4620 using the techniques described herein. In some implementations, a separate scatter-gather agent is included in each separate IO sector in the multi-die topology to perform the scatter and gather operations for packets.

[0264] In some embodiments, each scatter-gather agent, 4701 and 4702, collects the P2P packets (targeted to destination agents in the other IO sector) from multiple IO agents within the source sector and transmits them over the mesh

fabric 4620 to the other scatter-gather agent, 4702 and 4701, respectively. For example, scatter-gather agent 4701 collects P2P packets for UPI agents 4601A-B and IO agents 4611-4614 and scatter-gather agent 4702 collects P2P packets for UPI agents 4601C-D and IO agents 4615-4618.

[0265] As described further below with respect to FIG. 50, each scatter-gather agent (e.g., scatter-gather agent 4701) includes decode logic 5011 to decode the packet address information to identify the destination fabric agent, ordering logic 5012 to perform packet ordering operations as described herein, scatter and gather circuitry 5014-5015 to distribute the P2P packets across different columns on the mesh fabric 4620 using agents to route the packets to a scatter-gather agent on the destination IO-die (e.g., scatter-gather agent 4702), and reorder logic 5022 with a re-order buffer for reordering packets received from other IO sectors across the mesh fabric 4620. While some of the illustrated embodiments describe scatter-gather agents performing cross-sector packet processing, multiple scatter-gather agents may also be used to support in-sector P2P communication.

[0266] FIG. 48 illustrates an example implementation with four IO sectors A-D (e.g., four separate IO dies) coupled with compute sector fabrics 4891-4892 via multiple Embedded Multi-Die Interconnect Bridges (EMIBs) 4880A-E with additional routing and flow control logic to support communication as described herein. An EMIB is a silicon bridge embedded under the edges of two interconnecting dies and facilitates the electrical coupling between them.

[0267] In the illustrated example, a first set of scatter-gather agents 4803-4804 are associated with IO sector A which includes UPI agent 4801A and IO agent 4811, a second set of scatter-gather agents 4807-4810 are associated with IO sector B which includes UPI agent 4801B and IO agent 4812, a third set of scatter-gather agents 4853-4856 are associated with IO sector C which includes UPI agent 4801C and IO agent 4813, and a fourth set of scatter-gather agents 4857-4860 are associated with IO sector D which includes UPI agent 4801D and IO agent 4814.

[0268] In some embodiments, the scatter-gather routers interface with the network to inject packets via a CMS router. For example, for IO sector D, a set of four scatter-gather agents 4803, 4810, 4853, and 4860 are used to transit packets over one or both of the compute sector fabrics 4891-4892 using the various techniques described herein (e.g., scattering transmissions across columns and managing packet ordering). These packets may first pass from a corresponding agent through another scatter-gather agent. For example, IO agent 4814 may transmit packets to IO agent 4812 using a path that includes scatter-gather agents 4857-4858, 4860, and (after traversal of the compute sector fabric 4892) scatter-gather agents 4810, 4808, and 4807.

[0269] Referring to FIG. 49, some embodiments are implemented in a plurality of IO dies 4901-4904 connected across die-to-die interconnects 4921-4924 and Ultra Path Interconnect (UPI) channels 4911-4918 (e.g., Ultra Path CXL Interconnect (UXI) protocols). This particular implementation includes a plurality of IO agents 4931-4938, with IO agent 4931 associated with scatter-gather agent 4942 and IO agent 4935 associated with scatter-gather agent 4944. In addition, UPI interface 4911 of IO die 4901 is associated with scatter-gather agent 4941 and UPI interface 4913 of IO die 4902 is associated with scatter-gather agent 4943. A plurality of potential paths are illustrated for routing of

packets across the four IO dies **4901-4904** using a scatter and gather technique in accordance with embodiments of the invention. Packet routing may be scattered across the various UPI interconnects **4911-4918** and dies **4901-4904** to balance bandwidth utilization in accordance with the techniques described herein.

[0270] FIG. 50 illustrates additional details for one embodiment of a scatter-gather agent **5050** for communication between fabric agents as described herein (e.g., including cross-socket or remote P2P and route-through P2P communication). In some implementations, the scatter-gather agent **5050** includes source IO sector circuitry **5002** for managing incoming packets from agents of the source IO sector and transmitting the packets over the interconnect fabric **4620**. Similarly, destination IO sector circuitry **5003** manages incoming packets from the interconnect fabric **4620** (e.g., sent from another sector) and routing them to the destination agent or other IP block. Note that the same scatter-gather agent **5050** may be both the source for outgoing packets (processed by source IO sector circuitry **5002**) and the destination for incoming packets from the interconnect fabric **4620** (processed by the destination IO sector circuitry **5003**). In some implementations, the source IO sector circuitry **5002** and destination IO sector circuitry **5003** are located on the same edges of the die/package topology which are connected through the interconnect fabric **4620**.

[0271] In some embodiments, the scatter-gather agent **5050** collects packets from multiple agents in the source IO sector (i.e., the sector corresponding to the scatter-gather agent **5050**), and routes them through the interconnect fabric **4620** to a destination scatter-gather agent on a destination IO die using specified interconnect fabric routing rules. A sink buffer **5001** of the scatter-gather agent **5050** receives the P2P packets from IO agents in the same die over a dedicated fabric, such as mesh fabric **4720** shown in FIG. 47. This embodiment of the scatter-gather agent **5050** includes an in-sector interface **5007** on one side to couple to the dedicated fabric **4720** and fabric interfaces **5013**, **5023** to the interconnect fabric **4620** on the other side. The dedicated fabric **4720** may be any type of fabric supported by the scatter-gather agent in-sector interface **5007** and the interconnect fabric **4620** may be any type of fabric supported by the fabric interfaces **5013**, **5023**.

[0272] Target decode circuitry **5011** in the source IO sector circuitry **5002** processes incoming P2P packets to determine the destinations to which the packets are to be transmitted via the mesh interconnect **4620** (e.g., the final IO or UPI destinations). For example, the target decode circuitry **5011** may extract the packet header information from each packet and use routing tables **5017** (or other routing data structure) to identify the destination fabric agents and/or the destination scatter-gather agent to which the packet will be transmitted.

[0273] In one embodiment, scatter routing circuitry **5014** of the source IO sector circuitry **5002** and gather routing circuitry **5015** of the scatter-gather agent logic for the destination IO sector **5003** perform the scatter and gather operations, respectively, when sending and receiving packets, respectively, over the interconnect fabric **4620**. As mentioned, a “scatter” operation distributes packets received from in-sector agents across columns **4677** of the mesh interconnect fabric **4620**. Conversely, a “gather” operation

combines the P2P packets from the various columns and transmits them to the specified destination agent.

[0274] As mentioned, the scatter circuitry **5014** and gather circuitry **5015** may interact with or include one or more fabric routers **4678A-B** which buffer and re-route messages intended for communication on a given path of the interconnect fabric **4620** over another path. In some embodiments, the scatter routing circuitry **5011** may use the routing tables **5017** (or a different set of routing tables) to generate addressing information related to the path to be taken through the interconnect fabric **4620** (e.g., potentially using fabric routers **4678A-B** to re-route packets as described herein). For example, a set of fabric routers **4678A-B** may re-route a packet from a particular column corresponding to a destination fabric agent to a different column to distribute bandwidth among the columns and/or based on current conditions on the fabric. The fabric routers **4678A-B** include buffer storage circuitry to temporarily buffer packets in transmission and also include switching/routing logic to route and re-route packets in accordance with the routing tables **5017** (or using a separate set of routing tables within the interconnect fabric **4620**). In some implementations, the fabric routers may be associated with or integrated in the CMS routers **4870A-D** that couple the scatter-gather agents to the interconnect fabric **4620** (which may also store or have access to the routing tables).

[0275] In an implementation such as that shown in FIG. 47, there is only one destination when transmitting the P2P packets from the source sector scatter-gather agent **4701** to destination sector scatter-gather agent **4702** (or from source sector scatter-gather agent **4702** to destination sector scatter-gather agent **4701**). Thus, the same physical scatter-gather agent destination (e.g., scatter-gather agent **4702**) is associated with multiple logical destination IDs corresponding to the destination agents within the same sector (e.g., as decoded by the target decode circuitry **5014**). In one embodiment, an arbitration protocol is implemented (e.g., a round-robin scheme and/or a priority-based scheme implemented by arbitration circuitry of the fabric routers **4678A-B** and/or scatter-gather agents **4701-4702**) to select between these destination IDs when sequencing and routing P2P packets through the interconnect fabric **4620**. For example, the different fabric routers **4678A-B** and/or scatter-gather agents **4701-4702** may implement the arbitration protocol to select packets for routing through different columns to the same physical destination (e.g., the destination scatter-gather agent **4702**).

[0276] The physical scatter-gather agent **4702** may be mapped to the logical destination IDs of the destination fabric agents within the routing tables of the interconnect fabric **4620**. In some embodiments, the routing tables **5017** are managed by the CMS routers **4870A-D** that couple the scatter-gather agents to the interconnect fabric **4620**. In some implementations, the routing tables **5017** used by the scatter routing circuitry **5014** and/or decode circuitry **5011** include information related to potential paths through the interconnect **4620**, including fabric router information, so that one or more fields of the packets are updated to identify fabric routers **4678A-B** to be used for routing of the packet (e.g., indicating that the packet is to traverse through one or more mesh stops via corresponding fabric routers).

[0277] With scatter-gather agents in multiple IO sectors as shown in FIG. 48, some embodiments optimize placement of the scatter-gather agents **4803-4810** and **4853-4860** with

respect to each sector row. For example, the different scatter-gather agents **4803-4810** and **4853-4860** may be linked/coupled or otherwise associated with different rows to enable concurrent usage of fabric routers for the corresponding sectors.

[0278] Some embodiments of the interconnect fabric **4620** may provide unordered packet delivery. However, P2P packets have ordering requirements within different virtual channels that must be obeyed when they reach the destination agent. Hence, in some embodiments, the scatter-gather agents at both the source sector and the destination sector include ordering logic to ensure proper packet ordering. For example, ordering ID assignment logic **5012** at the source IO sector circuitry **5002** may tag the P2P packets with ordering IDs that are used by reordering logic **5022** of the destination IO sector circuitry **5003** to reorder the packets. The reordering logic **5022** may include a reorder buffer structure for storing the received packets to perform reordering. In particular, the reordering logic **5022** uses ordering IDs assigned to the packets on the sender side along with the message class and/or destination IDs to reorder the packets before sending them in order to the destination agents (e.g., over the dedicated P2P fabric **4720-4721** coupled to the agents).

[0279] In some embodiments, if the interconnect fabric guarantees a maximum latency between transmission from the source and receipt at the destination, the scatter-gather agent can stagger the streams to guarantee packet ordering without using an explicit ordering ID.

[0280] In some embodiments, an output fabric interface **5013** with flow control logic **5093** and an input fabric interface **5023** with flow control logic **5094** perform packet traffic flow control operations over the interconnect fabric **4620**. For example, the flow control logic **5093** of a source scatter-gather agent and the flow control logic **5094** of a destination scatter-gather agent may operate to provide end-to-end flow control for P2P packets transmitted over the interconnect fabric. In some implementations, the flow control logic **5093-5094** implements different fabric crediting configurations for different virtual channels over the interconnect fabric. In these embodiments, flow control logic **5093** of the source scatter-gather agent is allocated a number of credits for each virtual channel, and can only transmit a packet on a virtual channel if sufficient credits are available. Mesh credit return circuitry **5024** of the destination IO sector circuitry **5003** may provide the credit allocations to the flow control logic **5093** of the source IO sector circuitry **5002**. Using virtual channel credit allocations, those virtual channels with relatively larger credit allocations are provided with relatively higher bandwidth on the interconnect fabric. In this way, a total available bandwidth value can be subdivided between virtual channels using credit allocations. Additionally, higher priority virtual channels may be allocated relatively more buffer storage in the source and/or destination scatter-gather agents.

[0281] In some implementations, flow control credit allocations and buffer allocations enable the transmission of larger packet sizes. For example, while the interconnect fabric may support a relatively small maximum packet size (e.g., **64B**) the flow control credit logic of the scatter-gather agents and variable buffer sizes (per virtual channel) enable these larger packet sizes to be transmitted over the fabric (e.g., by subdividing the larger packets into block sizes

based on the maximum packet size of the interconnect fabric and allocating sufficient buffer storage and credits to ensure low latency operation).

[0282] In some embodiments, the buffer sizes may scale with the number of scatter-gather agents used in a given implementation. This also implies that in case of larger topologies with multiple IO sectors, the end-to-end credit loops between source and destination sectors can be broken down to smaller loops between intermediate scatter-gather agent instances to amortize the end-to-end buffer costs.

[0283] Additionally, the scatter-gather agents have visibility into P2P streams targeting various destinations using different message classes to enable the scatter-gather agents to fulfill any QoS requirements. For implementations where credits are shared across different source scatter-gather agent instances as described above, source throttling mechanisms may be used to ensure fairness across different sources and to meet the QoS requirements for the different traffic classes.

[0284] The IO agents using high bandwidth ordered traffic, can be spread across various dies, packages, and sockets. As mentioned, each scatter-gather agent **5050** includes scatter circuitry **5014** and gather circuitry **5015** for performing scatter and gather operations over the interconnect fabric **4620**. Depending on the implementation, these scatter and gather operations may be performed over high speed fabric links including, but not limited to intra-die links, inter-die links, inter-package links, and inter-socket links (e.g., UPI/UXI links). In some specific implementations, the die-to-die and socket-to-socket links use the Ultra Path Interconnect (UPI) architecture, and the UPI routing layer (as defined in the UPI architecture) can provision the ordering IDs for spreading the high bandwidth IO traffic across the available die-to-die and/or socket-to-socket links. This ordering-ID may be defined between a scatter source and the gather recipient. The gather recipient uses the ordering-ID to merge the ordered traffic, it being the final destination IO agent. Alternatively, merging with the ordering-ID may be performed on an intermediate routing entity between the source and the IO agent (e.g., a scatter-gather agent **5002**). In implementations that utilize scatter and gather, the scatter and gather operations continue until the packets are received by the final receiver.

[0285] One advantage of the techniques described herein is that they provide multiple routes for routing to the same physically located destination in the topology. Additionally, the ordering support in the scatter-gather agent allows connection to an unordered fabric with the agents remaining agnostic of the properties of the fabric and without requiring the responsibility of ordering the transaction streams to be transferred to the agents.

[0286] One particular implementation of the sequence number assignment and processing described above is illustrated in FIG. **51**. In this implementation, the ordering ID assignment circuitry **5012** adds a sequence number **5190** to each packet, where each packet is represented by a particular row in the table. The packet fields in this example include a type field indicating a type of operation to be performed, an address field indicating a destination address, a data field indicating the data to be transmitted in the packet, and a sequence number field to store the sequence number applied by the ordering ID assignment circuitry **5012**.

[0287] Packets are transmitted over the fabric interface **5013** as previously described. If the fabric requires strict ordering, determined at **5110**, then the packets are transmit-

ted in sequence as indicated at **5112**. If not, then the packets are not necessarily transmitted over the fabric in sequence as indicated at **5113**.

[0288] At the destination IO sector circuitry **5003**, sequence monitor and reordering circuitry **5100** processes and reorders the packets received over the fabric. In this embodiment, the sequence monitoring and reordering circuitry **5100** is a specific implementation of the reordering circuitry **5022** described with respect to FIG. **50**. The sequence monitoring and reordering circuitry **5100** performs sequence detection **5120** to detect the received packet sequence, and stores the corresponding packet sequence values to a memory **4628**. In one embodiment, the memory **4628** is a scratchpad memory, although various memory types may be used (e.g., a local cache memory, a local data store (LDS) memory, etc.). The sequence monitoring and reordering circuitry **5100** then performs sequence checking and reordering **5130** using the sequence values stored in the memory **4628**. For example, the sequence checking may involve identifying one or more packets received out of sequence and the reordering may include rearranging the packets so that they can be provided to the destination agent in order.

[0289] Some embodiments of the destination IO sector circuitry **5003** generate an interrupt **5135** in response to the sequence monitoring and reordering circuitry **5100** detecting that a packet sequence was received out of order. Additionally, or alternatively, the destination IO sector circuitry **5003** may generate an interrupt to signal an error condition if one of the packets is not received. This condition can be detected, for example, if the sequence monitoring and reordering circuitry **5100** does not detect the packet with a particular sequence number in a given packet sequence.

[0290] The triggered interrupts **5135** may be processed by interrupt handlers to perform appropriate actions. In the case of a sequence received out of order, for example, the interrupt handler may simply log the condition (e.g., as part of a debug or system testing process). In the case of a lost packet in the sequence, the interrupt handler and/or the destination IO sector circuitry **5003** may send a request to the source (e.g., via fabric interface **5013**) to retransmit the lost packet. In such a case, the sequence number may be used to uniquely identify the missing packet, which may be stored at the source for a period of time (e.g., within a transmit buffer/queue) so that it can be retransmitted if necessary.

[0291] Embodiments of the invention may be used to ensure ordered packet delivery in various scenarios. For example, the destination IO sector circuitry **5003** may be integral to or coupled to a memory controller and the packets being transmitted may be a sequence of memory operations, which need to be performed in the order in which they are transmitted. In this embodiment, the destination IO sector circuitry **5003** may reorder any packets received out of order before providing the packet sequence to the memory controller circuitry, which then implements the memory operations in the correct order.

[0292] These embodiments may also be used for PCIe ordering validation and for validation and debug testing, which currently has poor observability for these types of transactions. For example, these embodiments may be used in post-silicon validation where there is no existing mechanism to validate the ordering of transactions. During validation/test mode, software-visible registers can be updated

in response to the sequence checking performed by the sequence monitoring and reordering circuitry **5100**. These software-visible registers can then be polled/read to confirm correct (or incorrect) packet ordering at the target.

[0293] The embodiments described above may be implemented in any type of fabric or bus including, but not limited to, the IO system fabric (IOSF), advanced extensible interface (AXI) interconnects, and Ultra Path Interconnects.

[0294] In an environment where packets are encrypted before transmission over the fabric interconnect, some embodiments of the source ID sector circuitry **5002** do not encrypt the sequence numbers, even if the data in the packet is encrypted/compressed. Consequently, the destination IO sector circuitry can perform the techniques described herein without the need for accessing the encryption keys and other security features needed for decryption.

[0295] Embodiments disclosed herein may be implemented in hardware, software, firmware, or a combination of such implementation approaches. Embodiments of the disclosure may be implemented as computer programs or program code executing on programmable systems comprising at least one processor, a storage system (including volatile and non-volatile memory and/or storage elements), at least one input device, and at least one output device.

[0296] Program code may be applied to input instructions to perform the functions described herein and generate output information. The output information may be applied to one or more output devices, in known fashion. For purposes of this application, a processing system includes any system that has a processor, such as, for example; a digital signal processor (DSP), a microcontroller, an application specific integrated circuit (ASIC), or a microprocessor.

[0297] The program code may be implemented in a high level procedural or object oriented programming language to communicate with a processing system. The program code may also be implemented in assembly or machine language, if desired. In fact, the mechanisms described herein are not limited in scope to any particular programming language. In any case, the language may be a compiled or interpreted language.

[0298] One or more aspects of at least one embodiment may be implemented by representative instructions stored on a machine-readable medium which represents various logic within the processor, which when read by a machine causes the machine to fabricate logic to perform the techniques described herein. Such representations, known as “IP cores” may be stored on a tangible, machine readable medium and supplied to various customers or manufacturing facilities to load into the fabrication machines that actually make the logic or processor.

EXAMPLES

[0299] The following are example implementations of different embodiments of the invention.

[0300] Example 1. A processor, comprising: an interconnect fabric comprising a plurality of vertical interconnects coupled to a plurality of horizontal interconnects; and a plurality of bridge devices to route packets across the interconnect fabric on behalf of fabric agents; a first bridge device of the plurality of bridge devices comprising: target decode circuitry to decode a first plurality of the packets received from a plurality of source fabric agents to identify one or more destination

fabric agents associated with a second bridge device; first routing circuitry to route a first plurality of the packets across the interconnect fabric to the second bridge device, the first routing circuitry to distribute the first plurality of the packets across at least one of: multiple vertical interconnects of the plurality of vertical interconnects and multiple horizontal interconnects of the plurality of horizontal interconnects.

[0301] Example 2. The processor of example 1, further comprising a second bridge device of the plurality of bridge devices, the second bridge device comprising: a buffer to temporarily store the first plurality of packets received over the interconnect fabric; packet reordering circuitry to perform reordering of at least some of the first plurality of packets based on sequence identification fields in the first plurality of packets to produce an ordered sequence of the first plurality of packets; and an interface to couple the second bridge device to the one or more destination fabric agents, the second bridge device to transmit each packet of the first plurality of packets in accordance with the ordered sequence to a corresponding destination fabric agent of the one or more destination fabric agents.

[0302] Example 3. The processor of examples 1 or 2, wherein the first bridge device further comprises: ordering identification assignment circuitry to tag the first plurality of packets with the sequence identification fields based on an order in which the first plurality of packets are received by the first bridge device.

[0303] Example 4. The processor of any of examples 1-3, wherein the first bridge device further comprises: a buffer to temporarily store a second plurality of packets received over the interconnect fabric, the second plurality of packets addressed to one or more of the plurality of source fabric agents; packet reordering circuitry to perform reordering of at least some of the second plurality of packets based on sequence identification fields in the second plurality of packets to produce an ordered sequence of the second plurality of packets; and an interface to couple the second bridge device to the plurality of source fabric agents, the second bridge device to transmit each packet of the second plurality of packets in accordance with the ordered sequence to a corresponding source fabric agent of the plurality of source fabric agents.

[0304] Example 5. The processor of any of examples 1-4, wherein each bridge device of the plurality of bridge devices comprises: a fabric interface to couple the respective bridge device to the interconnect fabric; and credit-based flow control logic to implement credit-based flow control and bandwidth allocations, wherein each packet of the first plurality of packets is associated with a virtual channel or traffic class having a number of credits associated therewith, and wherein the interconnect fabric is to transmit each packet in accordance with a corresponding virtual channel or traffic class only if a sufficient number of corresponding credits are available.

[0305] Example 6. The processor of any of examples 1-5, wherein each bridge device of the plurality of bridge devices is associated with a different sector of a plurality of sectors of the processor, the plurality of sectors including a first sector associated with the first bridge device and the plurality of source fabric agents

and a second sector associated with the second bridge device and the one or more destination fabric agents.

[0306] Example 7. The processor of any of examples 1-6, wherein the first sector and the second sector are integral to at least one of: different dies in different processor packages, different dies of a single processor package, and different regions a processor die.

[0307] Example 8. The processor of any of examples 1-7, wherein the first sector is integral to a first die of the single processor package and the second sector is integral to a second die of the single processor package, wherein the interconnect fabric comprises one or more die-to-die links to couple the first die and the second die

[0308] Example 9. The processor of any of examples 1-8, wherein the first sector is integral to a first die of a first processor package and the second sector is integral to a second die of a second processor package, wherein the interconnect fabric comprises one or more socket-to-socket links to couple the first die and the second die.

[0309] Example 10. The processor of any of examples 1-9, further comprising a third sector integral to a third die of the first processor package, wherein the interconnect fabric comprises one or more die-to-die links to couple the first die and the third die.

[0310] Example 11. A method, comprising: decoding, by a first bridge device associated with a plurality of source fabric agents, a first plurality of packets received from the plurality of source fabric agents of an interconnect fabric comprising a plurality of vertical interconnects coupled to a plurality of horizontal interconnects, wherein decoding is to identify one or more destination fabric agents associated with a second bridge device; routing, by first routing circuitry, the first plurality of packets across the interconnect fabric to the second bridge device, the first routing circuitry to distribute the first plurality of packets across at least one of: multiple vertical interconnects of the plurality of vertical interconnects and multiple horizontal interconnects of the plurality of horizontal interconnects.

[0311] Example 12. The method of example 11, further comprising: temporarily buffering, at the second bridge device, the first plurality of packets received over the interconnect fabric; reordering, at the second bridge device, at least some of the first plurality of packets based on sequence identification fields in the first plurality of packets to produce an ordered sequence of the first plurality of packets; and transmitting, through an interface of the second bridge device, each packet of the first plurality of packets in accordance with the ordered sequence to a corresponding destination fabric agent of a plurality of destination fabric agents coupled to the second bridge device.

[0312] Example 13. The method of examples 11 or 12, further comprising: tagging, at the first bridge device, the first plurality of packets with the sequence identification fields based on an order in which the first plurality of packets are received by the first bridge device.

[0313] Example 14. The method of any of examples 11-13, further comprising: temporarily storing, at the first bridge device, a second plurality of packets received over the interconnect fabric, the second plurality of packets addressed to one or more of the

plurality of source fabric agents; reordering, at the first bridge device, at least some of the second plurality of packets based on sequence identification fields in the second plurality of packets to produce an ordered sequence of the second plurality of packets; and transmitting, through an interface of the first bridge device, each packet of the second plurality of packets in accordance with the ordered sequence to a corresponding source fabric agent of the plurality of source fabric agents.

[0314] Example 15. The method of any of examples 11-14, further comprising: performing, by the first bridge device and the second bridge device, credit-based flow control and bandwidth allocations, wherein each packet of the first plurality of packets is associated with a virtual channel or traffic class having a number of credits associated therewith, and wherein the interconnect fabric is to transmit each packet in accordance with a corresponding virtual channel or traffic class only if a sufficient number of corresponding credits are available.

[0315] Example 16. The method of any of examples 11-15, wherein each of the first bridge device and the second bridge device is associated with a different sector of a plurality of sectors of a processor, the plurality of sectors including a first sector associated with the first bridge device and the plurality of source fabric agents and a second sector associated with the second bridge device and the one or more destination fabric agents.

[0316] Example 17. The method of any of examples 11-16, wherein the first sector and the second sector are integral to at least one of: different dies in different processor packages, different dies of a single processor package, and different regions a processor die.

[0317] Example 18. The method of any of examples 11-17, wherein the first sector is integral to a first die and the second sector is integral to a second die of the single processor package, wherein the interconnect fabric comprises one or more die-to-die links to couple the first die and the second die.

[0318] Example 19. The method of any of examples 11-18, wherein the first sector is integral to a first die of a first processor package and the second sector is integral to a second die of a second processor package, wherein the interconnect fabric comprises one or more socket-to-socket links to couple the first die and the second die.

[0319] Example 20. A machine-readable medium having program code stored thereon which, when executed by a machine, causes the machine to perform operations, comprising: decoding, by a first bridge device associated with a plurality of source fabric agents, a first plurality of packets received from the plurality of source fabric agents of an interconnect fabric comprising a plurality of vertical interconnects coupled to a plurality of horizontal interconnects, wherein decoding is to identify one or more destination fabric agents associated with a second bridge device; routing, by first routing circuitry, the first plurality of packets across the interconnect fabric to the second bridge device, the first routing circuitry to distribute the first plurality of packets across at least one of: multiple vertical inter-

connects of the plurality of vertical interconnects and multiple horizontal interconnects of the plurality of horizontal interconnects.

[0320] Such machine-readable storage media may include, without limitation, non-transitory, tangible arrangements of articles manufactured or formed by a machine or device, including storage media such as hard disks, any other type of disk including floppy disks, optical disks, compact disk read-only memories (CD-ROMs), compact disk rewritables (CD-RWs), and magneto-optical disks, semiconductor devices such as read-only memories (ROMs), random access memories (RAMs) such as dynamic random access memories (DRAMs), static random access memories (SRAMs), erasable programmable read-only memories (EPROMs), flash memories, electrically erasable programmable read-only memories (EEPROMs), phase change memory (PCM), magnetic or optical cards, or any other type of media suitable for storing electronic instructions.

[0321] Accordingly, embodiments of the disclosure also include non-transitory, tangible machine-readable media containing instructions or containing design data, such as Hardware Description Language (HDL), which defines structures, circuits, apparatuses, processors and/or system features described herein. Such embodiments may also be referred to as program products.

What is claimed is:

1. A processor, comprising:

an interconnect fabric comprising a plurality of vertical interconnects coupled to a plurality of horizontal interconnects; and

a plurality of bridge devices to route packets across the interconnect fabric on behalf of fabric agents;

a first bridge device of the plurality of bridge devices comprising:

target decode circuitry to decode a first plurality of the packets received from a plurality of source fabric agents to identify one or more destination fabric agents associated with a second bridge device;

first routing circuitry to route a first plurality of the packets across the interconnect fabric to the second bridge device, the first routing circuitry to distribute the first plurality of the packets across at least one of: multiple vertical interconnects of the plurality of vertical interconnects and multiple horizontal interconnects of the plurality of horizontal interconnects.

2. The processor of claim 1, further comprising a second bridge device of the plurality of bridge devices, the second bridge device comprising:

a buffer to temporarily store the first plurality of packets received over the interconnect fabric;

packet reordering circuitry to perform reordering of at least some of the first plurality of packets based on sequence identification fields in the first plurality of packets to produce an ordered sequence of the first plurality of packets; and

an interface to couple the second bridge device to the one or more destination fabric agents, the second bridge device to transmit each packet of the first plurality of packets in accordance with the ordered sequence to a corresponding destination fabric agent of the one or more destination fabric agents.

3. The processor of claim 2, wherein the first bridge device further comprises:

ordering identification assignment circuitry to tag the first plurality of packets with the sequence identification fields based on an order in which the first plurality of packets are received by the first bridge device.

4. The processor of claim 1, wherein the first bridge device further comprises:

a buffer to temporarily store a second plurality of packets received over the interconnect fabric, the second plurality of packets addressed to one or more of the plurality of source fabric agents;

packet reordering circuitry to perform reordering of at least some of the second plurality of packets based on sequence identification fields in the second plurality of packets to produce an ordered sequence of the second plurality of packets; and

an interface to couple the second bridge device to the plurality of source fabric agents, the second bridge device to transmit each packet of the second plurality of packets in accordance with the ordered sequence to a corresponding source fabric agent of the plurality of source fabric agents.

5. The processor of claim 1, wherein each bridge device of the plurality of bridge devices comprises:

a fabric interface to couple the respective bridge device to the interconnect fabric; and

credit-based flow control logic to implement credit-based flow control and bandwidth allocations, wherein each packet of the first plurality of packets is associated with a virtual channel or traffic class having a number of credits associated therewith, and wherein the interconnect fabric is to transmit each packet in accordance with a corresponding virtual channel or traffic class only if a sufficient number of corresponding credits are available.

6. The processor of claim 1, wherein each bridge device of the plurality of bridge devices is associated with a different sector of a plurality of sectors of the processor, the plurality of sectors including a first sector associated with the first bridge device and the plurality of source fabric agents and a second sector associated with the second bridge device and the one or more destination fabric agents.

7. The processor of claim 6, wherein the first sector and the second sector are integral to at least one of: different dies in different processor packages, different dies of a single processor package, and different regions a processor die.

8. The processor of claim 7, wherein the first sector is integral to a first die of the single processor package and the second sector is integral to a second die of the single processor package, wherein the interconnect fabric comprises one or more die-to-die links to couple the first die and the second die.

9. The processor of claim 7, wherein the first sector is integral to a first die of a first processor package and the second sector is integral to a second die of a second processor package, wherein the interconnect fabric comprises one or more socket-to-socket links to couple the first die and the second die.

10. The processor of claim 9, further comprising a third sector integral to a third die of the first processor package, wherein the interconnect fabric comprises one or more die-to-die links to couple the first die and the third die.

11. A method, comprising:

decoding, by a first bridge device associated with a plurality of source fabric agents, a first plurality of

packets received from the plurality of source fabric agents of an interconnect fabric comprising a plurality of vertical interconnects coupled to a plurality of horizontal interconnects, wherein decoding is to identify one or more destination fabric agents associated with a second bridge device;

routing, by first routing circuitry, the first plurality of packets across the interconnect fabric to the second bridge device, the first routing circuitry to distribute the first plurality of packets across at least one of: multiple vertical interconnects of the plurality of vertical interconnects and multiple horizontal interconnects of the plurality of horizontal interconnects.

12. The method of claim 11, further comprising:

temporarily buffering, at the second bridge device, the first plurality of packets received over the interconnect fabric;

reordering, at the second bridge device, at least some of the first plurality of packets based on sequence identification fields in the first plurality of packets to produce an ordered sequence of the first plurality of packets; and

transmitting, through an interface of the second bridge device, each packet of the first plurality of packets in accordance with the ordered sequence to a corresponding destination fabric agent of a plurality of destination fabric agents coupled to the second bridge device.

13. The method of claim 12, further comprising:

tagging, at the first bridge device, the first plurality of packets with the sequence identification fields based on an order in which the first plurality of packets are received by the first bridge device.

14. The method of claim 11, further comprising:

temporarily storing, at the first bridge device, a second plurality of packets received over the interconnect fabric, the second plurality of packets addressed to one or more of the plurality of source fabric agents;

reordering, at the first bridge device, at least some of the second plurality of packets based on sequence identification fields in the second plurality of packets to produce an ordered sequence of the second plurality of packets; and

transmitting, through an interface of the first bridge device, each packet of the second plurality of packets in accordance with the ordered sequence to a corresponding source fabric agent of the plurality of source fabric agents.

15. The method of claim 11, further comprising:

performing, by the first bridge device and the second bridge device, credit-based flow control and bandwidth allocations, wherein each packet of the first plurality of packets is associated with a virtual channel or traffic class having a number of credits associated therewith, and wherein the interconnect fabric is to transmit each packet in accordance with a corresponding virtual channel or traffic class only if a sufficient number of corresponding credits are available.

16. The method of claim 11, wherein each of the first bridge device and the second bridge device is associated with a different sector of a plurality of sectors of a processor, the plurality of sectors including a first sector associated with the first bridge device and the plurality of source fabric agents and a second sector associated with the second bridge device and the one or more destination fabric agents.

17. The method of claim 16, wherein the first sector and the second sector are integral to at least one of: different dies in different processor packages, different dies of a single processor package, and different regions a processor die.

18. The method of claim 17, wherein the first sector is integral to a first die and the second sector is integral to a second die of the single processor package, wherein the interconnect fabric comprises one or more die-to-die links to couple the first die and the second die.

19. The method of claim 17, wherein the first sector is integral to a first die of a first processor package and the second sector is integral to a second die of a second processor package, wherein the interconnect fabric comprises one or more socket-to-socket links to couple the first die and the second die.

20. A machine-readable medium having program code stored thereon which, when executed by a machine, causes the machine to perform operations, comprising:

decoding, by a first bridge device associated with a plurality of source fabric agents, a first plurality of packets received from the plurality of source fabric agents of an interconnect fabric comprising a plurality of vertical interconnects coupled to a plurality of horizontal interconnects, wherein decoding is to identify one or more destination fabric agents associated with a second bridge device;

routing, by first routing circuitry, the first plurality of packets across the interconnect fabric to the second bridge device, the first routing circuitry to distribute the first plurality of packets across at least one of: multiple vertical interconnects of the plurality of vertical interconnects and multiple horizontal interconnects of the plurality of horizontal interconnects.

* * * * *